Содержание

§ 1. Интерфейсы и стоимость операций **4**
1.1. Сначала контракт, затем контейнер . . . 4
1.2. Что именно мы считаем . . . 4
1.3. Динамический массив: дорогая операция и дешёвая история . . . 5
1.4. Метод потенциалов без скрытых предположений . . . 6
1.5. Двоичный счётчик и стек с multipop . . . 7
1.6. Стек с минимумом . . . 7
1.7. Очередь на двух стеках . . . 8
1.8. Проверка и самостоятельная работа . . . 9
1.9. Разбор: минимум в скользящем окне . . . 10
1.10. Углубление: почему сжатие требует зазора . . . 11

§ 2. Дерево отрезков и проектирование агрегата **12**
2.1. Динамическая задача и статический ответ . . . 12
2.2. Интервалы становятся узлами . . . 12
2.3. Почему запрос логарифмический . . . 13
2.4. Полная реализация суммы . . . 14
2.5. Сначала алгебра объединения . . . 15
2.6. Максимальная сумма подотрезка . . . 15
2.7. Когда одного числа в узле мало . . . 16
2.8. Отладка и задания . . . 16
2.9. Разбор: сводка и порядок частей . . . 17
2.10. Углубление: поиск по сохранённым суммам . . . 18
2.11. От значений к координатам: подготовка частотного дерева . . . 18

§ 3. Интервальные изменения и отложенные операции **20**
3.1. Новый контракт требует нового инварианта . . . 20
3.2. Сначала прибавление . . . 21
3.3. Присваивание ломает коммутативность . . . 21
3.4. Реализация: один способ применять метку . . . 22
3.5. Изменение и запрос . . . 23
3.6. Подробная трасса смешанных операций . . . 24
3.7. Пример курса: чётные и нечётные суммы . . . 24
3.8. Геометрия, ограничения и проверка . . . 25
3.9. Разбор: что происходит при частичном запросе . . . 26
3.10. Углубление: какие преобразования допускает сводка . . . 26
3.11. Пример из видео: глобальное присваивание через версии . . . 27

§ 4. Битовые множества и массовые операции **28**
4.1. Интерфейс множества и размер универсума . . . 28
4.2. Почему вставка — OR, а не сложение . . . 28
4.3. Безопасная реализация на C++17 . . . 29
4.4. Сдвиг и достижимые суммы . . . 31
4.5. Поиск в ширину: целый слой как множество . . . 31
4.6. Транзитивное замыкание . . . 33
4.7. Линейная алгебра над GF(2) . . . 33
4.8. Проверка, границы применимости и задания . . . 34

(c) Aleksey Tolstikov

§ 5. Декомпозиция данных и запросов **35**
5.1. Интерфейс и две границы запроса . . . 35
5.2. Реализация суммы на C++17 . . . 35
5.3. Откуда берётся квадратный корень . . . 36
5.4. Сортировка внутри блока и изменение контракта . . . 37
5.5. Прыжки: сводка как переход в следующий блок . . . 37
5.6. Тяжёлые и лёгкие вершины . . . 39
5.7. Перестройка, перенос фрагментов и пределы метода . . . 39
5.8. Другой контракт: прибавление диапазону и чтение точки . . . 40
5.9. Углубление: декомпозиция порядка запросов . . . 41
5.10. Проверка и самостоятельная работа . . . 41

§ 6. Эффективный перебор: встреча посередине **42**
6.1. Контракт объединения двух половин . . . 42
6.2. Пример курса: четыре числа с нулевой суммой . . . 43
6.3. Подсчёт подмножеств с заданной суммой . . . 43
6.4. Рюкзак: почему недостаточно ближайшего веса . . . 44
6.5. Реализация рюкзака и её доказательство . . . 45
6.6. XOR-пути и общий элемент двух половин . . . 46
6.7. Углубление: большой и малый шаг . . . 46
6.8. Восстановление решения и бюджет памяти . . . 47
6.9. Двусторонний поиск и проверка границ метода . . . 48

§ 7. Персистентность: версии, стек и очередь **49**
7.1. Что означает «старая версия доступна» . . . 49
7.2. Неизменяемый список как представление стека . . . 50
7.3. Реализация стека и время жизни узлов . . . 50
7.4. Список: цена изменения в середине . . . 51
7.5. Очередь на двух персистентных стеках . . . 52
7.6. Почему потенциал нельзя потратить в каждой ветви . . . 53
7.7. Копирование пути и толстые узлы . . . 53
7.8. Персистентность, снимки и откат . . . 54
7.9. Проверка ветвящейся истории и задания . . . 54

§ 8. Персистентное дерево отрезков **56**
8.1. Контракт: какой массив представляет корень . . . 56
8.2. Копирование пути и два инварианта . . . 56
8.3. Реализация: пул узлов и присваивание . . . 57
8.4. Чтение без выделения памяти . . . 58
8.5. Префиксные частоты: другая ось дерева . . . 59
8.6. K-я порядковая статистика по двум корням . . . 60
8.7. Какие агрегаты можно вычитать . . . 62
8.8. Почему изменение массива не меняет один префикс . . . 62
8.9. Память, проверка и практические ошибки . . . 62
8.10. Вопросы для самопроверки . . . 63

§ 9. Персистентные деревья поиска и кучи **64**
9.1. Два интерфейса и два разных порядка . . . 64
9.2. Неизменяемые узлы BST . . . 64
9.3. Вставка, удаление и сохранение старого порядка . . . 65

(c) Aleksey Tolstikov

9.4. Где появляется логарифм, а где его нет . . . 67
9.5. Левосторонняя куча: инвариант короткой правой ветви . . . 67
9.6. Слияние — единственная содержательная операция . . . 68
9.7. Слияние родственных версий и кратности . . . 69
9.8. Биномиальные кучи и осторожность с амортизацией . . . 69
9.9. Ресурсы и выбор реализации . . . 70
9.10. Проверка и вопросы для самопроверки . . . 70

§ 10. Общие методы персистентности и дерево версий 72
10.1. Сначала фиксируем модель вычислений . . . 72
10.2. Полное копирование как эталон . . . 72
10.3. Толстая ячейка на линейной шкале времени . . . 73
10.4. Ограниченная коробка изменений и копирование узла . . . 73
10.5. Почему время создания не описывает ветвление . . . 74
10.6. Эйлерова нумерация готового дерева . . . 75
10.7. Динамический порядок: что нужно общему методу . . . 76
10.8. Сериализация: сначала однозначный формат . . . 77
10.9. Проверка обхода не равна восстановлению . . . 78
10.10. Итоговый выбор метода и самопроверка . . . 79

(c) Aleksey Tolstikov

§ 1. Интерфейсы и стоимость операций

Структура данных начинается с обещания пользователю: какие операции допустимы, что они возвращают и сколько ресурсов могут потребовать. Массив, дерево и связный список — способы выполнить это обещание. Одно и то же представление бывает удачным для одного набора операций и неудобным для другого.

После этой главы вы сможете отличать стоимость отдельной операции от стоимости последовательности, доказывать амортизированные оценки методом потенциалов и реализовать очередь с минимумом. Предполагается знакомство с массивами, указателями, рекурсией и базовыми контейнерами C++17.

1.1. Сначала контракт, затем контейнер

Рассмотрим последовательность целых чисел. Нужны чтение и изменение элемента по индексу, добавление и удаление последнего элемента. Назовём этот интерфейс динамическим массивом. Пока мы ничего не сказали о размещении памяти. Список тоже хранит последовательность, но доступ к произвольному индексу обычно требует пройти через предыдущие элементы.

Контракт включает не только названия методов. Надо определить индексацию, допустимость пустой структуры, поведение ошибочных вызовов и требования к устойчивости ссылок. В этой книге индексы начинаются с нуля; диапазон $[l, r)$ включает l, но не включает r. Пустой диапазон разрешён. Извлечение из пустой очереди — ошибка, а не молчаливое возвращение нуля: ноль может быть обычным элементом.

Требование	Массив	Связный список
Прочитать элемент по индексу	Постоянное число адресных операций	До линейного числа переходов
Вставить после известного узла	Возможен сдвиг хвоста	Перепривязка ссылок
Хранить элементы подряд	Да	Обычно нет
Перебрать все элементы	Линейное время, хорошая локальность	Линейное время, переходы по указателям

Последняя строка показывает ограниченность одной асимптотики. Две реализации могут иметь одинаковую оценку, но по-разному использовать кеш процессора. Сначала доказываем корректность и порядок роста, затем измеряем константы на характерной нагрузке.

Связь с видео. В записи ССД от 01.09.2021 около 15:19 на доске появляется `DS Interface`, а около 30:39 — размер, ёмкость и `push_back`. Сохраняем этот ход объяснения: сначала требование к операциям, затем представление. Числа в примерах ниже подобраны для учебника.

1.2. Что именно мы считаем

В модели машинных слов чтение ячейки, сравнение и сложение помещающихся в слово целых чисел считаем операциями постоянной стоимости. Это предположение надо проверять: сумма n чисел может не помещаться в тип элемента. В примерах с суммами используем `long long`; входные ограничения должны гарантировать, что все промежуточные результаты представимы. Переполнение знакового целого в C++ не даёт корректного «вычисления по модулю».

Размер входа также требует определения. Для массива естественный параметр — число элементов n. Для одного большого целого число его битов равно $\lfloor \log_2 n \rfloor + 1$, если $n > 0$. Цикл из n итераций линейный по числовому значению, но экспоненциальный по длине записи числа. В этой главе параметры массивов — числа элементов, а целочисленная арифметика ограничена машинным словом.

Различим три вида гарантий.

- **Худший случай одного вызова:** максимум стоимости по всем допустимым состояниям данного размера.
- **Амортизированная оценка:** оценка общей стоимости серии операций с учётом начального состояния.
- **Ожидаемая оценка:** математическое ожидание относительно явно заданной случайности — входов или внутренних случайных решений.

Амортизация не требует случайного входа. Если противник выбирает каждую следующую операцию, доказанная оценка последовательности всё равно действует. Для средней стоимости на случайных входах необходимо задать распределение; равномерность не возникает автоматически.

Например, при q добавлениях в растущий массив отдельный вызов может копировать почти все ранее добавленные элементы. Это не противоречит линейному времени всех q добавлений. Утверждения относятся к разным объектам: одному вызову и всей истории.

1.3. Динамический массив: дорогая операция и дешёвая история

Пусть `size` — число занятых ячеек, `capacity` — число выделенных. Пока `size < capacity`, записываем новый элемент в свободную ячейку. Когда места нет, выделяем массив удвоенной ёмкости, переносим элементы и дописываем новый.

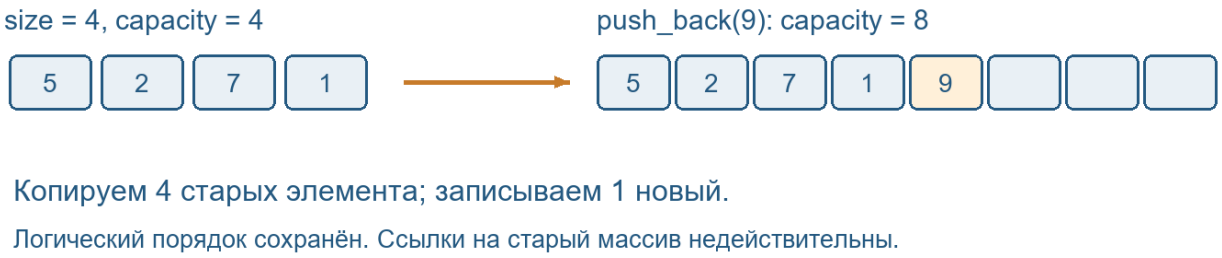


Рисунок 1: Расширение массива меняет ёмкость, но сохраняет логическую последовательность.

Начнём с нулевого размера и первой ёмкости 1. Добавим девять чисел. Копирование потребуется при переходах ёмкости $1 \to 2$, $2 \to 4$, $4 \to 8$, $8 \to 16$. Число скопированных элементов равно $1 + 2 + 4 + 8 = 15$. Ещё девять записей помещают новые элементы. Последний вызов дорогой, но весь процесс содержит только 24 записи или переноса в принятой модели.

Для произвольного q последняя скопированная ёмкость меньше q. Геометрическая сумма всех переносов меньше $2q$. Вместе с q записями получаем менее $3q$ действий, то есть $O(q)$ времени и $O(1)$ амортизированно на добавление. Если выделение блока требует времени, линейного по его длине, суммы выделяемых ёмкостей также образуют геометрическую прогрессию и порядок оценки сохраняется.

Увеличение ёмкости на одну ячейку даёт другую картину: $1 + 2 + \cdots + (q - 1) = \Theta(q^2)$. Причина улучшения при удвоении не в особом числе 2, а в геометрическом росте. Любой фиксированный множитель больше единицы даёт линейную суммарную стоимость, хотя расход памяти и константы меняются.

Пример из записи. Около 45:58 в лекции 01.09.2021 показано копирование массива в больший. Здесь мы продолжаем этот пример точным подсчётом переносов. Последовательность ёмкостей 1, 2, 4, 8, 16 — редакторская иллюстрация идеи, а не дословное восстановление доски.

В C++ `std::vector::push_back` имеет амортизированную константную сложность при обычных предпосылках о стоимости работы с элементом. Стандарт не обещает именно удвоение ёмкости. Перевыделение инвалидирует ссылки и итераторы на элементы. Если размер заранее известен, `reserve` может устранить перевыделения; вызов `reserve(size()+1)` перед каждым добавлением, напротив, способен разрушить геометрический рост.

Дополнение из новой расшифровки, 29:09–30:49. Лектор обсуждает более общую политику `new_capacity = α · capacity + β`: постоянная добавка помогает реже выделять совсем маленькие блоки в начале, а множитель больше единицы обеспечивает геометрический рост в дальнейшем. В записи приводятся добавки 16 или 32 элемента. Это пример из видео; он не задаёт обязательную политику `std::vector`. Для фиксированных $\alpha > 1$, $\beta \geq 0$ прежние ёмкости ограничиваются геометрической прогрессией относительно последней, поэтому суммарные переносы остаются линейными. Округление до целого и минимальная начальная ёмкость должны входить в контракт реализации.

1.4. Метод потенциалов без скрытых предположений

Потенциал $\Phi(S)$ — запас работы, приписанный состоянию структуры S. Пусть фактическая стоимость операции i равна c_i, а амортизированная —

$$\hat{c}_i = c_i + \Phi(S_i) - \Phi(S_{i-1}).$$

Просуммируем равенства. Внутренние потенциалы сокращаются:

$$\sum_{i=1}^{q} c_i = \sum_{i=1}^{q} \hat{c}_i + \Phi(S_0) - \Phi(S_q).$$

Если потенциал неотрицателен, то фактическая сумма не превосходит суммы амортизированных стоимостей плюс начальный потенциал. Поэтому достаточно ограничить $\hat{c}_i$ и отдельно учесть $\Phi(S_0)$. Само по себе название «потенциал» ничего не доказывает: отрицательный конечный запас мог бы скрыть большую стоимость.

Для растущего массива с непустым состоянием выберем $\Phi = 2s - c$, где s — размер, c — ёмкость; для пустого начального состояния зададим $\Phi = 0$. После первого добавления и при дальнейшем удвоении ёмкости $s \leq c < 2s$, поэтому потенциал неотрицателен.

Если расширения нет, фактическая стоимость равна 1, потенциал растёт на 2, значит амортизированная стоимость равна 3. Перед расширением $s = c$ и потенциал равен c. После копирования и добавления $s' = c+1$, $c' = 2c$, потенциал равен 2. Фактическая стоимость $c+1$, поэтому $\hat{c} = (c+1) + (2-c) = 3$. Первый вызов разбирается отдельно и имеет постоянную стоимость.

Это доказательство относится к добавлениям. Если разрешить удаления и немедленное уменьшение ёмкости, надо заново проверить неотрицательность и переходы. Простейшее чередование добавления и удаления около порога может вызвать постоянные дорогие копирования. Типичный способ избежать этого — разные пороги роста и сжатия, например расширение при заполнении и сжатие при четверти заполнения.

1.5. Двоичный счётчик и стек с multipop

У счётчика один инкремент меняет все младшие единицы на нули и ближайший ноль на единицу. Переход $0111 \to 1000$ меняет четыре бита, хотя предыдущие переходы могли быть дешевле. При q инкрементах от нуля бит 0 меняется q раз, бит 1 — $\lfloor q/2 \rfloor$, бит 2 — $\lfloor q/4 \rfloor$. До переполнения фиксированной разрядности сумма меньше $2q$.

То же можно доказать потенциалом: число единичных битов. Если сбрасываются k единиц и устанавливается один ноль, то $c = k + 1$, $\Delta\Phi = 1 - k$, следовательно $\hat{c} = 2$. Для произвольного начального состояния добавляется его потенциал. Поэтому формулировка «любые q инкрементов стоят не больше $2q$» без оговорки о старте неверна.

Переход	Изменено битов	Изменение потенциала	Амортизированная стоимость
0000 → 0001	1	1	2
0001 → 0010	2	0	2
0011 → 0100	3	−1	2
0111 → 1000	4	−2	2

В стеке `multipop(k)` удаляет до k элементов. Один вызов может стоить $\Theta(n)$, но каждый добавленный элемент удаляется не более одного раза. Начиная с пустого стека, суммарное число удалений не превосходит числа добавлений. Это агрегатное доказательство; потенциалом можно взять текущий размер стека.

Эти примеры полезны как шаблон поиска доказательства: найдите объект, который оплачивает дорогую работу и не может оплатить её повторно. Для счётчика это накопленные единицы, для стека — ещё не удалённые элементы, для массива — запас свободной ёмкости, созданный предыдущим расширением.

1.6. Стек с минимумом

Хранить один текущий минимум недостаточно. Если удалить элемент, равный минимуму, нужно узнать минимум оставшегося стека. Вместе с каждым элементом сохраним минимум всего префикса до него. Тогда верхняя пара содержит и значение, и ответ для текущего состояния.

Для добавлений 5, 2, 2, 7 пары будут `(5,5), (2,2), (2,2), (7,2)`. После удаления 7 минимум остаётся 2; после удаления одной двойки он также равен 2. Дубликаты не требуют отдельной логики. Инвариант доказывается индукцией: новая вторая компонента равна минимуму нового значения и предыдущего минимума; удаление восстанавливает уже корректный префикс.

(c) Aleksey Tolstikov

```cpp
class MinStack {
    std::vector<std::pair<long long, long long>> a;
public:
    bool empty() const { return a.empty(); }
    void push(long long x) {
        long long m = empty() ? x : std::min(x, a.back().second);
        a.emplace_back(x, m);
    }
    long long top() const {
        if (empty()) throw std::out_of_range("empty stack");
        return a.back().first;
    }
    long long minimum() const {
        if (empty()) throw std::out_of_range("empty stack");
        return a.back().second;
    }
    void pop() {
        if (empty()) throw std::out_of_range("empty stack");
        a.pop_back();
    }
};
```

На уровне абстрактного стека все действия постоянные. В данной реализации хранилище — `std::vector`, поэтому добавление пары имеет амортизированную константную стоимость, а перевыделение может быть линейным. Чтение и удаление последнего элемента постоянны для нашего простого типа. Дополнительная память на элемент — одно значение минимума.

1.7. Очередь на двух стеках

Входной стек `in` принимает новые элементы. Выходной `out` хранит начало очереди на вершине. Пока `out` не пуст, извлекаем из него. Когда он пуст, переносим в него все элементы из `in`: порядок переворачивается, и самый старый элемент оказывается сверху.

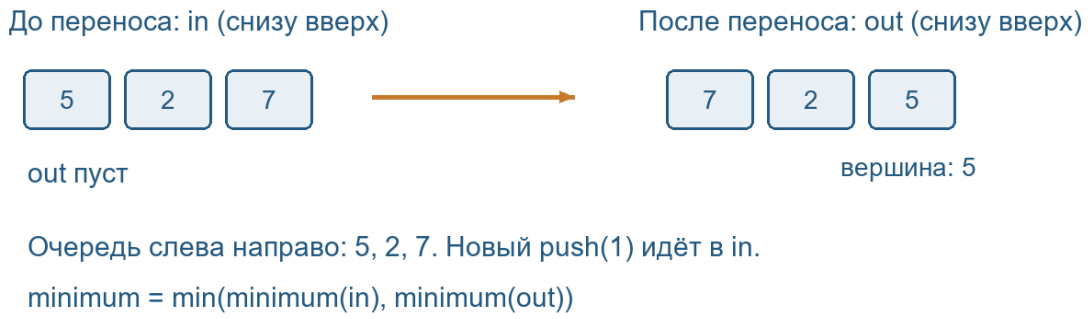


Рисунок 2: Два стека представляют одну очередь; перенос меняет физический порядок, сохраняя логический.

Например, после добавлений 5, 2, 7 входной стек снизу вверх содержит 5, 2, 7. Перенос создаёт выходной стек снизу вверх 7, 2, 5. Добавим 1 во входной стек. Логическая очередь теперь 5, 2, 7, 1; её минимум равен минимуму двух стековых минимумов.

(c) Aleksey Tolstikov

```
class MinQueue {
    MinStack in, out;
    void prepare() {
        if (out.empty()) {
            while (!in.empty()) {
                long long x = in.top();
                out.push(x);
                in.pop();
            }
        }
    }
public:
    bool empty() const { return in.empty() && out.empty(); }
    void push(long long x) { in.push(x); }
    long long front() {
        prepare();
        return out.top();
    }
    void pop() {
        prepare();
        out.pop();
    }
    long long minimum() const {
        if (in.empty()) return out.minimum();
        if (out.empty()) return in.minimum();
        return std::min(in.minimum(), out.minimum());
    }
};
```

Если оба стека пусты, `minimum`, `front` и `pop` заканчиваются исключением из `MinStack`. Мы не назначаем фиктивный минимум пустой очереди. Операция `front` тоже может инициировать перенос, поэтому её худшая стоимость линейная.

Каждый элемент один раз попадает во входной стек, не более одного раза переносится в выходной и один раз удаляется. За q вызовов суммарно выполняется $O(q)$ стековых операций. Амортизация внутренних векторов также даёт $O(q)$ суммарной работы, поэтому две оценки не конфликтуют. При моделировании стековых операций как единичных можно взять потенциал $2|in|$: перенос k элементов выполняет $2k$ действий и уменьшает потенциал на $2k$.

Важно переносить элементы **только в пустой** выходной стек. Иначе новые элементы окажутся перед старыми и нарушат FIFO. Очередь с минимумом не является решением для дека с удалениями с обоих концов: такой интерфейс требует другого баланса и отдельного доказательства.

1.8. Проверка и самостоятельная работа

Для очереди эталоном служит `std::deque`: добавление справа, удаление слева, минимум прямым просмотром. После каждой случайной операции сравнивайте начало и минимум. Проверяйте пустую очередь, единственный элемент, одинаковые значения, отрицательные числа, несколько переносов и длинную серию чтений без изменений. Случайные проверки дополняют инвариант, но не заменяют его.

Для временной сложности тесты должны считать переносы, а не только секунды. На маленьких входах таймер больше измеряет накладные расходы. При удвоении числа операций линейный счётчик переносов объясняет поведение лучше одного удачного замера.

1. Сформулируйте контракт очереди с минимумом: допустимые вызовы, результат каждого метода и поведение пустой структуры.
2. Почему линейная стоимость одного `push_back` совместима с амортизированной константной?
3. Выведите число переносов при добавлении 17 элементов в массив с удвоением ёмкости.
4. Что изменится при увеличении ёмкости на 10 ячеек?
5. Получите телескопическую формулу метода потенциалов и назовите нужное ограничение на конечный потенциал.
6. Сколько битов меняется при переходе $31 \to 32$? Как это согласуется с оценкой последовательности?
7. Почему начальное состояние нельзя игнорировать в анализе счётчика?
8. Проследите очередь после `push(5), push(2), pop(), push(1), front()`.
9. Приведите пример нарушения FIFO при переносе в непустой выходной стек.
10. Объясните, почему наша реализация не даёт гарантии $O(1)$ на каждый вызов `front`.

Практика. Реализуйте очередь с минимумом по приведённому контракту и независимую проверку на прямой модели. Для углубления рассмотрите iRunner 4185 «Добавить один много раз» и 4022 «Минимум в списке». У второй задачи операции разрешены на обоих концах; это сознательное усложнение, а не прямое применение листинга.

Что переносится в следующие главы. Мы будем выбирать между дешёвыми обновлениями и дешёвыми запросами, явно считать объём дополнительной памяти и доказывать сохранение инварианта после каждой операции. Дерево отрезков даст логарифмические гарантии для обоих типов действий.

1.9. Разбор: минимум в скользящем окне

Пусть измерения поступают последовательно, а после каждого нового значения нужен минимум последних k измерений. При $k = 3$ и потоке $[5, 2, 7, 1, 4]$ полные окна имеют минимумы $[2, 1, 1]$. Это применение очереди с минимумом: добавляем новое значение, удаляем самое старое, если размер превысил k, и читаем минимум, когда окно заполнено. Числа этого разбора выбраны для учебника.

Получено	Очередь после удаления лишнего	Ответ
5	5	Окно ещё не заполнено
2	5, 2	Окно ещё не заполнено
7	5, 2, 7	2
1	2, 7, 1	1
4	7, 1, 4	1

Внешний алгоритм хранит размер очереди отдельным счётчиком. При добавлении увеличивает его, при удалении уменьшает. Контракт требует $1 \le k \le n$; при другом соглашении надо отдельно определить, существуют ли ответы для неполного окна. Каждое измерение добавляется ровно один раз и удаляется не более одного раза, поэтому весь проход занимает $O(n)$, а занятые элементы — $O(k)$ памяти. Даже если один перенос стеков стоит $\Theta(k)$, он не происходит заново для каждого окна.

Есть другая структура — монотонная очередь индексов. Из её хвоста удаляют индексы элементов, не меньших нового, а из головы — вышедшие за левую границу окна. Оставшиеся значения возрастают. Почему можно забыть удалённый из хвоста элемент? Новый элемент не больше него и покинет окно позже. Во всех будущих окнах, где старый мог бы быть полезен, новый будет не хуже.

Это доказательство использует специальную последовательность запросов: границы окна движутся только вправо. Произвольный запрос к прошлому диапазону разрушил бы аргумент. Очередь на двух стеках хранит все живые элементы и реализует обычный FIFO; монотонная очередь хранит только

кандидатов на минимум. Их оценки одинаковы по порядку, но интерфейсы и основания корректности различаются. Чтобы выбрать структуру, сначала определите допустимые операции.

1.10. Углубление: почему сжатие требует зазора

Предположим, массив растёт вдвое при переполнении и сжимается вдвое, как только заполнение падает до половины. Возьмём состояние размера $c/2 + 1$ и ёмкости c. Удаление одного элемента вызывает сжатие до $c/2$ с переносом $c/2$ элементов. Следующее добавление требует снова расшириться до c. Повторяя эти два действия, получаем линейную стоимость почти каждой операции, хотя логический размер меняется всего на единицу.

Исправим правило: сжимаем только при размере не больше четверти ёмкости. После сжатия вдвое массив заполнен примерно наполовину. До следующего расширения нужны многие добавления, а до следующего сжатия — многие удаления. Этот промежуток позволяет накопить плату за переносы. Разные пороги переключения называют гистерезисом.

Для ёмкости $c \geq 1$ и размера s рассмотрим потенциал

$$\Phi(s, c) = \begin{cases} 2s - c, & s \geq c/2, \\ c/2 - s, & s < c/2. \end{cases}$$

Он неотрицателен и равен нулю при половинном заполнении. Вставка без расширения меняет потенциал не более чем на 2, удаление без сжатия — не более чем на 1. Рост из полного состояния разобран выше: уменьшение потенциала оплачивает перенос старых элементов.

Для сжатия удобно разобрать точный порог: до удаления размер $c/4 + 1$, после него $c/4$, новая ёмкость $c/2$. До операции потенциал равен $c/4 - 1$, после — нулю. Если стоимость удаления и копирования равна $1 + c/4$, амортизированная стоимость равна 2. Малые ёмкости и округление целых размеров меняют лишь константу; практически задают минимальную ёмкость и не сжимают ниже неё.

Полученная гарантия относится к нашей политике контейнера. Она не означает, что метод `pop_back` у `std::vector` автоматически уменьшает выделенную память. Разделяйте логический размер, занятую ёмкость и контракт конкретной библиотеки. Для очереди со стеками, реализованными векторами, резерв памяти может зависеть от максимального достигнутого размера, а не от числа элементов в текущий момент.

Самостоятельный эксперимент. Считайте отдельно число операций пользователя и число переносов элементов. Сравните увеличение ёмкости на единицу, удвоение и удвоение со сжатием при половине заполнения. В последнем случае используйте описанное чередование, а не случайный вход. Эксперимент должен иллюстрировать уже доказанную границу, а не заменять её измерением одного удачного запуска.

§ 2. Дерево отрезков и проектирование агрегата

Пусть массив меняется, а запросы требуют суммы произвольных диапазонов. Прямой просмотр делает обновления дешёвыми, а запросы дорогими. Префиксные суммы меняют роли: запрос становится постоянным, зато изменение одного элемента затрагивает длинный хвост. Дерево отрезков хранит информацию на нескольких масштабах и даёт логарифмическую стоимость обеих операций.

После главы вы сможете построить дерево, доказать оценку запроса, реализовать точечные изменения и спроектировать сведения узла для максимальной суммы подотрезка. Предполагается понимание рекурсии и оценок первой главы.

2.1. Динамическая задача и статический ответ

Контракт: `set(p,x)` присваивает элементу a_p значение x; `sum(l,r)` возвращает $\sum_{i=l}^{r-1} a_i$. Индексы удовлетворяют $0 \le p < n$ и $0 \le l \le r \le n$. На пустом диапазоне сумма равна нулю. Размер массива не меняется. Добавление элемента в конец — другой интерфейс, которого текущая реализация не обещает.

Для статического массива определим $P_0 = 0$, $P_{i+1} = P_i + a_i$. Тогда ответ равен $P_r - P_l$. Однако присваивание в позиции p меняет все P_j с $j > p$. Если обновления часты, предвычисления приходится слишком часто чинить.

Представление	Построение	Присваивание	Сумма диапазона
Только массив	$O(n)$	$O(1)$	$O(r-l)$
Префиксные суммы	$O(n)$	$O(n)$	$O(1)$
Дерево отрезков	$O(n)$	$O(\log n)$	$O(\log n)$

Таблица не назначает победителя без знания нагрузки. При одном запросе после миллиона присваиваний обычный массив может быть выгоднее. Структуру выбирают по полному набору операций и памяти, а не по наличию слова «логарифм».

Ход из видео. В записи 13.09.2022 первые девять минут посвящены интерфейсу и конфликту префиксных сумм с обновлениями. Далее массив разбивается на интервалы разных размеров. Именно эту последовательность сохраняем. Числовой массив ниже — редакторский пример, а не восстановленные значения с доски.

2.2. Интервалы становятся узлами

Пусть $a = [2, 7, 1, 4, 3, 6, 5, 8]$. Сохраним суммы соседних пар, затем четвёрок и всего массива. Получится двоичное дерево. Лист отвечает за одну позицию; внутренний узел — за объединение двух соседних интервалов. Корень хранит 36, его дети — 14 и 22.

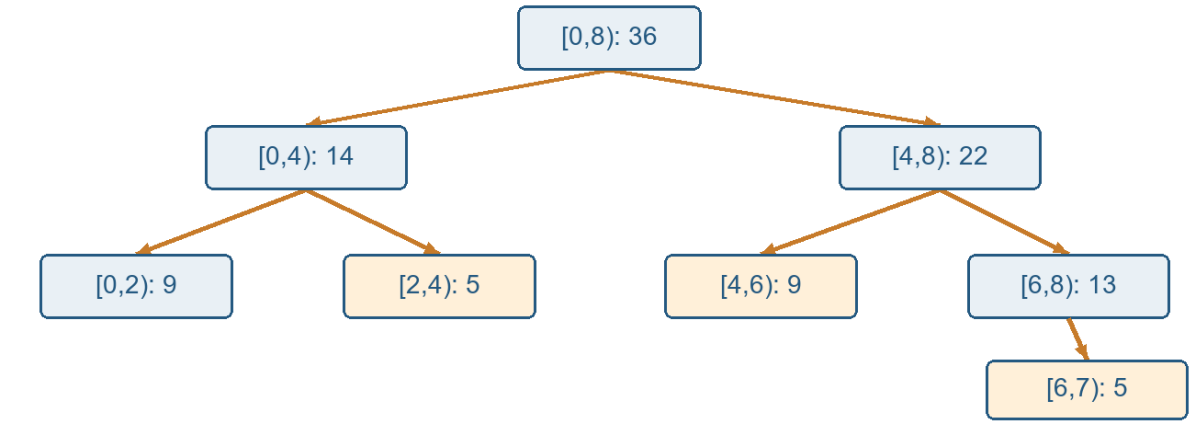


Рисунок 3: Узел хранит сумму своего диапазона; запрос собирается из непересекающихся узлов.

Инвариант: значение узла равно сумме элементов его фиксированного интервала. Оно не описывает диапазон **значений** элементов: индексы остаются прежними, даже если содержимое массива сильно меняется.

Для запроса $[2, 7)$ берём интервалы $[2, 4)$, $[4, 6)$ и $[6, 7)$. Их суммы 5, 9 и 5 дают 19. Элементы не теряются и не учитываются дважды. Запрос $[0, 8)$ совпадает с корнем и не требует спуска, а $[3, 4)$ в конце спуска совпадает с одним листом.

Если размер не является степенью двойки, есть два удобных подхода. Рекурсивное дерево делит фактический диапазон пополам. Итеративное дополняет массив до ближайшей степени двойки нейтральными элементами. В этой главе выберем второй вариант: он делает хранение и код компактными.

Пусть `base` — минимальная степень двойки не меньше n. В массиве `t` корень имеет индекс 1, дети узла v — $2v$ и $2v + 1$, листья начинаются с `base`. Для $n > 0$ выполняется $n \le base < 2n$, значит размер `2*base` линейный. При $n = 0$ оставляем `base=1`; единственный допустимый запрос — пустой.

2.3. Почему запрос логарифмический

Рекурсивное объяснение выделяет три случая. Если интервал узла не пересекается с запросом, возвращаем ноль. Если целиком лежит внутри, возвращаем сохранённую сумму. При частичном пересечении спускаемся в детей и складываем их ответы.

Нельзя обосновывать $O(\log n)$ только высотой: дерево высоты $\log n$ содержит $\Theta(n)$ узлов. Важен характер посещённых вершин. На каждом уровне не более двух узлов пересекают границы запроса частично. Остальные посещённые узлы либо целиком принимаются, либо сразу отбрасываются. От каждого граничного пути отделяется не более одного целиком принимаемого поддерева на уровне. Поэтому число существенных посещений ограничено константой на уровень.

Итеративный алгоритм выражает ту же идею через два указателя на границы полуинтервала. Если левая граница — правый ребёнок, её узел нельзя объединить с левым братом: тот уже вне запроса. Забираем узел и двигаем границу вправо. Если правая граница нечётна, узел непосредственно слева от неё целиком входит в запрос, а его правый брат уже не входит. Забираем этот узел. После этого обе границы поднимаются к родителям.

Для $[2, 7)$ при `base=8` начинаем с $(l, r) = (10, 15)$. Узел 14 соответствует $[6, 7)$, он попадает в правый накопитель. После подъёма границы равны $(5, 7)$. Узел 5 даёт $[2, 4)$, узел 6 — $[4, 6)$. Следующий подъём даёт равные границы и завершает запрос.

Шаг	Границы в массиве дерева	Добавленные интервалы
Листья	10, 15	$[6, 7)$
Уровень пар	5, 7	$[2, 4)$ и $[4, 6)$
Завершение	3, 3	Ничего

На каждом уровне выполняется постоянное число операций, а число подъёмов логарифмическое. Это доказательство относится к одному запросу в худшем случае, а не к амортизированной стоимости серии запросов.

2.4. Полная реализация суммы

```
class SumTree {
    int n, base = 1;
    std::vector<long long> t;
public:
    explicit SumTree(const std::vector<long long>& a) : n(int(a.size())) {
        while (base < n) base *= 2;
        t.assign(2 * base, 0);
        for (int i = 0; i < n; ++i) t[base + i] = a[i];
        for (int v = base - 1; v > 0; --v)
            t[v] = t[2 * v] + t[2 * v + 1];
    }
    void set(int p, long long x) {
        assert(0 <= p && p < n);
        p += base;
        t[p] = x;
        while (p > 1) {
            p /= 2;
            t[p] = t[2 * p] + t[2 * p + 1];
        }
    }
    long long sum(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        long long left = 0, right = 0;
        l += base; r += base;
        while (l < r) {
            if (l & 1) left = left + t[l++];
            if (r & 1) right = t[--r] + right;
            l /= 2; r /= 2;
        }
        return left + right;
    }
};
```

Полный исходник с заголовками находится в `code/02-segment-tree.hpp`. Размер и индексы представлены `int`: учебный контракт предполагает $n \le 10^6$, что также исключает переполнение `base` и индексов. Значения и суммы используют `long long`; вызывающая сторона гарантирует отсутствие переполнения сумм. `assert` проверяет предусловия в отладочной сборке и не заменяет валидацию недоверенного ввода в приложении.

Построение заполняет листья, после чего обходит внутренние узлы от последнего к корню. Дети всегда уже готовы, поэтому инвариант устанавливается индукцией по высоте. Это $O(base) = O(n)$ работы, а не $O(n \log n)$: мы не делаем n отдельных вызовов `set`.

Присваивание меняет один лист и всех его предков. Для нашего массива `set(3,6)` меняет сумму $[2, 4)$ с 5 на 7, сумму $[0, 4)$ с 14 на 16 и общую сумму с 36 на 38. Все остальные узлы остаются правильными: их диапазоны не содержат позицию 3.

Запрос не изменяет дерево, поэтому помечен `const`. Два накопителя выглядят избыточными для суммы, но сохраняют порядок частей. Это пригодится при обобщении на операцию, которая ассоциативна, но не коммутативна.

2.5. Сначала алгебра объединения

Дереву нужна операция `merge`, которая получает сведения двух соседних диапазонов и строит сведения их объединения. Для запросов с пустыми частями нужен нейтральный элемент. Ассоциативность означает, что разные способы расставить скобки не меняют результат, пока порядок частей сохранён.

Для суммы нейтральный элемент — 0; для минимума — достаточно большое значение или отдельный признак пустоты; для конкатенации строк — пустая строка. Коммутативность не обязательна. Если хранить конкатенацию, левый накопитель пополняют справа, а правый — слева. Перестановка частей дала бы другой ответ.

Не всякая операция подходит в исходном виде. Среднее двух средних не равно среднему объединения, если размеры частей различаются. Правильные сведения — пара «сумма, число элементов», а среднее вычисляется после объединения. Аналогично один минимум не позволяет узнать, сколько раз он встретился: необходимо хранить минимум вместе с кратностью.

Для пары (m, c) при объединении выбираем меньший минимум и его счётчик; если минимумы равны, складываем счётчики. Нейтральная запись имеет счётчик 0 и специальный признак пустоты либо безопасно выбранное значение. Не используйте фиктивное бесконечное значение в арифметике, где его сложение может переполниться.

Дерево отрезков полезно тем, что позволяет отделить доказательство структуры от доказательства сводки. Сначала покажите, что `merge` точно описывает конкатенацию двух последовательностей. Затем алгоритм покрытия запроса автоматически объединит правильные сведения нужных частей.

2.6. Максимальная сумма подотрезка

Одна сумма недостаточна, чтобы найти лучший подотрезок. Например, последовательности $[5, -5]$ и $[0, 0]$ имеют одинаковую сумму 0, но разные максимальные суммы непустых подотрезков. При склейке нужен ответ на вопрос, выгодно ли продолжать подотрезок через границу.

Разрешим **пустой** подотрезок с суммой 0. Для каждого диапазона храним четыре числа: общую сумму `sum`, лучшую сумму префикса `pref`, суффикса `suff` и произвольного подотрезка `best`. Для одиночного значения x три последних поля равны $\max(0, x)$.

Если сведения левой и правой частей обозначены L и R, то

$$\begin{aligned} S &= L.sum + R.sum, \\ P &= \max(L.pref, L.sum + R.pref), \\ Q &= \max(R.suff, R.sum + L.suff), \\ B &= \max(L.best, R.best, L.suff + R.pref). \end{aligned}$$

Префикс либо заканчивается слева, либо включает всю левую часть и префикс правой. Суффикс разбирается симметрично. Лучший подотрезок расположен целиком в одной части либо пересекает границу и складывается из суффикса слева и префикса справа. Эти случаи исчерпывают все возможности.

```
struct Summary {
    long long sum, pref, suff, best;
};
Summary leaf(long long x) {
    long long p = std::max(0LL, x);
    return {x, p, p, p};
}
Summary merge(Summary l, Summary r) {
    return {l.sum + r.sum,
            std::max(l.pref, l.sum + r.pref),
            std::max(r.suff, r.sum + l.suff),
            std::max({l.best, r.best, l.suff + r.pref})};
}
```

Нейтральная запись — `{0,0,0,0}`. Для массива $[-2, 3, -1, 4, -5]$ ответ равен 6: подотрезок $[3, -1, 4]$. Если все элементы отрицательны, ответ 0 соответствует разрешённому пустому подотрезку. Для задачи с обязательным непустым ответом нужно изменить базу и обработку пустых узлов, а не только последнюю строку вывода.

Связь с видео. В лекции 08.09.2021 около 1:19:27 видны поля суммы и граничных оптимумов. Это источник идеи расширенной сводки. Обозначения, соглашение о пустом ответе и приведённые формулы здесь сформулированы и доказаны заново; они не являются цитатой неразборчивой доски.

2.7. Когда одного числа в узле мало

Рассмотрим запрос: сколько элементов на диапазоне меньше x? Сумма, минимум и максимум по отдельности не дают ответа. Один из вариантов для статического массива — хранить в каждом узле отсортированный список всех его элементов. Запрос раскладывается на $O(\log n)$ узлов, в каждом выполняется `lower_bound`.

Получаем $O(\log^2 n)$ времени запроса, $O(n \log n)$ памяти и столько же времени построения при слиянии дочерних списков. Каждый элемент присутствует один раз на каждом уровне. Название merge-sort tree отражает это сходство со слиянием, но дерево используется для запросов по исходным позициям.

Точечные изменения усложняют такой вариант. Если список узла — `std::vector`, вставка или удаление может сдвигать линейное число элементов. Поэтому нельзя механически объявить обновление логарифмическим только потому, что меняется логарифмическое число узлов. Стоимость локального действия в узле должна входить в итоговую оценку.

Количество **различных** значений также не является суммой количества различных в двух половинах: одинаковое значение может встретиться по обе стороны границы. Подход зависит от ограничений. Маленький универсум допускает битовые множества; статические запросы допускают специальные offline-методы; персистентные частотные деревья появятся позднее. Эта граница защищает от неверного универсального обещания «любая статистика за логарифм».

2.8. Отладка и задания

Начните с массива длины 0, 1, 3 и 8. Проверяйте пустой диапазон, весь массив, первый и последний элементы. Размер 3 обнаруживает ошибки в дополнении до степени двойки. После серии присваиваний сравнивайте все диапазоны с прямым суммированием на маленьком массиве.

Для сводки максимальной суммы независимо переберите все подотрезки, включая пустой. Кроме правильного ответа полезно проверять ассоциативность `merge` на коротких случайных массивах: объединения $(A + B) + C$ и $A + (B + C)$ должны совпасть по всем четырём полям. Такой тест проверяет смысл сводки, а не только конкретный маршрут запроса.

1. Зафиксируйте отличие `set(p,x)` от `add(p,x)`.
2. Какие префиксные суммы меняются при изменении позиции 0?
3. Разложите $[1, 7)$ в дереве из восьми листьев на максимальные целиком покрытые узлы.
4. Почему высоты дерева недостаточно для доказательства сложности запроса?
5. Какие узлы меняются после присваивания последнему элементу?
6. Почему построение снизу вверх линейно?
7. Приведите ассоциативную некоммутативную операцию и объясните порядок накопления.
8. Почему среднее следует хранить как пару?
9. Докажите формулу `best` и определите ответ для всех отрицательных значений.
10. Почему вектор отсортированных значений в узле не гарантирует быстрых изменений?

Практика. Реализуйте интерфейс суммы и затем замените сводку на «минимум и число его вхождений». Для углубления добавьте запрос максимальной суммы подотрезка. Это учебные задания данной главы, а не заявления о новых опубликованных задачах iRunner.

В следующей главе изменение затронет целый диапазон. Пересчитать все его листья было бы слишком дорого. Чтобы сохранить логарифмическое время, придётся хранить не только ответ, но и ещё не переданное детям преобразование.

2.9. Разбор: сводка и порядок частей

Проследим четыре поля максимальной суммы на массиве $[-2, 3, -1, 4, -5]$. Разделим его на $L = [-2, 3]$ и $R = [-1, 4, -5]$. Для левой части сумма 1, лучший префикс 1, лучший суффикс 3 и лучший подотрезок 3. Для правой части сумма -2, лучший префикс 3, лучший суффикс 0 и лучший подотрезок 4. Нулевой суффикс здесь пустой: любой непустой суффикс правой части имеет отрицательную сумму.

Часть	sum	pref	suff	best
$[-2, 3]$	1	1	3	3
$[-1, 4, -5]$	-2	3	0	4
Всё вместе	-1	4	1	6

Пересекающий границу кандидат равен $3 + 3 = 6$: суффикс $[3]$ слева продолжается префиксом $[-1, 4]$ справа. Лучший префикс всего массива равен $1 + 3 = 4$, а лучший суффикс — $-2 + 3 = 1$. Так четыре поля описывают разные оптимумы; ни одно нельзя без доказательства заменить другим.

Почему операция ассоциативна, хотя формулы содержат несколько максимумов? Сводка однозначно определяется последовательностью. Корректная `merge` возвращает сводку конкатенации. Поэтому `merge(merge(A,B),C)` и `merge(A,merge(B,C))` описывают одну последовательность и совпадают по всем полям. Это семантическое доказательство проще прямого раскрытия вложенных максимумов. Оно действует для корректных сводок последовательностей; произвольные четыре числа могут не удовлетворять их инвариантам.

Теперь рассмотрим строки как наглядную модель некоммутативной операции. Для запроса к буквам `abcdefgh` на $[1, 7)$ ответ должен быть `bcdefg`. При обходе снизу вверх правые части обнаруживаются от правого края к левому. Поэтому очередной кусок добавляется перед уже собранной правой строкой.

Если добавлять его после, можно получить переставленные блоки, хотя набор выбранных листьев будет правильным.

Строковая модель годится для проверки порядка, но буквальная конкатенация не имеет постоянной стоимости. Если дерево хранит реальные строки, время слияний и объём скопированных символов надо учесть отдельно. Оценка $O(\log n)$ предполагает сводку постоянного размера и постоянную стоимость её объединения. Четыре числа этому условию удовлетворяют, строка переменной длины — нет.

2.10. Углубление: поиск по сохранённым суммам

Дерево может отвечать не только на запрос известного диапазона. Пусть все элементы неотрицательны, а их общая сумма положительна. Требуется найти первую позицию p, для которой сумма $[0, p+1)$ не меньше заданного k, где $1 \le k \le sum(0, n)$. Если элементы равны 0 или 1, это поиск позиции k-й единицы.

Начинаем в корне. Если сумма левого ребёнка не меньше k, ответ находится слева. Иначе весь левый диапазон можно пропустить: вычитаем его сумму из k и переходим вправо. Доходим до листа, который и даёт искомый индекс. На каждом уровне выбирается ровно один ребёнок, поэтому время $O(\log n)$; двоичный поиск с отдельными запросами сумм занял бы $O(\log^2 n)$.

Для массива $[0, 2, 0, 1, 3, 0, 0, 1]$ и $k = 4$ сумма левой половины равна 3. Переходим вправо с остатком $k = 1$. В диапазоне $[4, 8)$ левая половина $[4, 6)$ имеет сумму 3, поэтому переходим туда. Затем выбираем лист 4 со значением 3. Действительно, сумма до позиции 4 включительно равна 6, а сумма до неё — 3.

Инвариант спуска: искомый элемент находится в текущем диапазоне, а k — порог относительно его начала после вычета пропущенных слева сумм. Неотрицательность обеспечивает монотонность префиксов и позволяет сравнивать порог с полной суммой левой части. С отрицательными числами это неверно: у $[5, -5]$ сумма 0, хотя первый префикс уже достигает 4. Такое левое поддерево нельзя пропустить по одной сумме.

Практический контракт. Не вызывайте этот поиск при $k \le 0$, при k больше общей суммы или после обновления, создавшего отрицательный элемент. Можно вместо запрета определить специальный результат «не найдено», но его смысл следует зафиксировать до реализации. Нулевые дополненные листья не создают ложного ответа при допустимом положительном пороге.

Самостоятельная реализация. Добавьте поиск k-й единицы и точечное переключение 0/1. Сравните ответы с линейным проходом, включая массив из одних нулей, единственную единицу в конце и удаление последней единицы. Объясните, почему основное дерево из этой главы подходит без изменения сводки, а изменяется только алгоритм запроса.

2.11. От значений к координатам: подготовка частотного дерева

В предыдущем поиске листья соответствовали позициям исходного массива. Есть другой полезный выбор: лист соответствует возможному значению, а в нём хранится число его вхождений. Тогда сумма префикса — число элементов меньше порога, а поиск по суммам находит порядковую статистику мультимножества. Структура дерева та же, но смысл координаты изменился.

Пусть заранее известны все значения, которые могут появиться: $[100, -7, 100, 40]$. После сортировки и удаления повторов получаем координаты $[-7, 40, 100]$. Значениям сопоставляем ранги 0, 1 и 2. При добавлении 100 увеличиваем частоту листа 2, при удалении уменьшаем её, предварительно проверив, что она положительна. Для мультимножества $\{100, -7, 100, 40\}$ частоты равны $[1, 1, 2]$. Третьему элементу в отсортированном порядке соответствует ранг 2, а исходный ответ — 100.

Сжатие сохраняет порядок и равенство, но не расстояния. Разность рангов значений 100 и 40 равна 1, хотя разность самих значений равна 60. Поэтому в задачах о длине покрытия координатная ячейка должна хранить настоящую длину промежутка, а не единицу. Это различие понадобится в геометрическом приложении следующей главы.

Для запроса «сколько элементов строго меньше 50» сам порог 50 не обязан присутствовать среди координат. `lower_bound` в массиве $[-7, 40, 100]$ возвращает позицию 2; запрашиваем сумму частот на $[0, 2)$ и получаем 2. Для «не больше 100» используем `upper_bound`, получаем позицию 3 и суммируем весь диапазон. Граница принадлежит пространству рангов, а не исходному массиву.

Если все будущие значения известны из входных операций, их можно собрать до построения дерева. Это предварительное чтение не переставляет сами операции: ответы по-прежнему выдаются в их исходном порядке. Если новые значения приходят в неизвестном заранее потоке, фиксированное сжатие уже не подходит без перестройки; нужны другой интерфейс или другая структура.

Задание на выбор модели. Сравните два запроса: «найти третье по величине значение среди всех добавленных» и «найти третье по величине значение только на позициях $[l, r)$ исходного массива». Первому достаточно одного дерева частот. Второй одновременно ограничивает позиции и спрашивает о порядке значений; переход к частотным листьям теряет сведения о позициях. Именно здесь возникает необходимость в более сложных структурах последующих недель.

(c) Aleksey Tolstikov

§ 3. Интервальные изменения и отложенные операции

Если прибавить число ко всему массиву, его сумма изменится предсказуемо: к ней добавится произведение числа на длину. Нет необходимости немедленно переписывать каждый элемент. Дерево отрезков позволяет применить это наблюдение к любому диапазону: изменить несколько целиком покрытых узлов и отложить передачу изменений ниже.

Цель главы — не запомнить функцию `push`, а понять, какое состояние она представляет. Мы реализуем присваивание и прибавление на диапазоне, докажем порядок композиции меток и разберём пример с чётными и нечётными суммами из старого конспекта курса.

3.1. Новый контракт требует нового инварианта

Поддерживаем `add(l,r,d)`, `set(l,r,x)` и `sum(l,r)` на полуинтервалах $[l, r)$. Размер массива постоянен; пустой диапазон допустим и ничего не меняет. Значения могут быть отрицательными. Для примеров предполагаем $n \le 10^6$ и представимость всех значений, меток, произведений на длину и промежуточных сумм в `long long`.

После прибавления d к диапазону длины k его сумма возрастает на dk. После присваивания x она становится xk. Значит, для полного покрытия узла ответ можно обновить за постоянное время. Но дети продолжают хранить старые сведения. У узла появляется обязанность: перед будущим спуском передать детям преобразование, которое они ещё не учли.

Обычное равенство «сумма родителя равна сумме хранимых сумм детей» теперь не всегда истинно. Для узла с меткой f верно другое: его сумма соответствует результату применения f к объединению состояний детей. Метка уже учтена в собственной сумме узла, но ещё не учтена в его детях. Вдоль пути могут существовать несколько таких отложенных преобразований; доступ к поддереву должен учитывать их в правильном порядке.

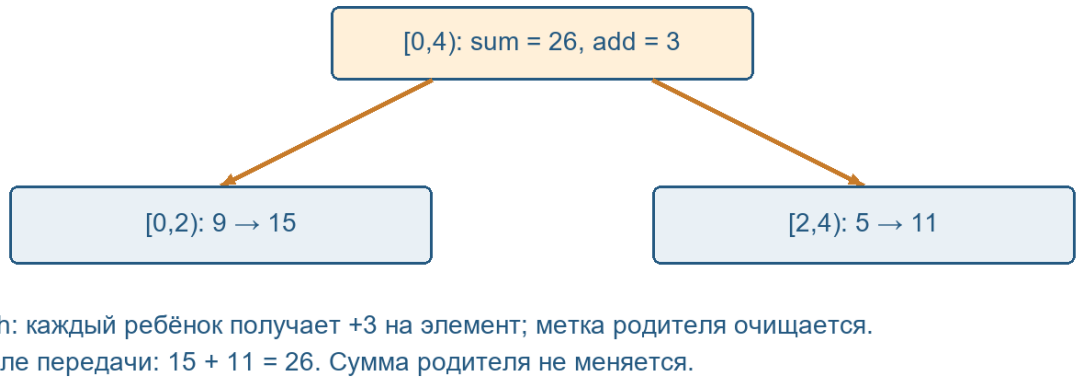


Рисунок 4: Полное покрытие меняет сумму и метку узла; перед частичным спуском метка передаётся детям.

Это соглашение устраняет типичную двойную прибавку: при полном покрытии возвращаем сохранённую сумму, не прибавляя метку ещё раз. Оно же объясняет, почему перед рекурсивным запросом нужно протолкнуть метку, даже если запрос не меняет логический массив.

3.2. Сначала прибавление

Для прибавлений достаточно метки `pending`: сколько ещё нужно добавить каждому элементу детей. Чтобы применить d к узлу длины k, выполняем `sum += d*k` и `pending += d`. Чтобы протолкнуть накопленное p, применяем p к каждому ребёнку с его собственной длиной, затем обнуляем метку родителя.

Длины детей могут отличаться, если длина родительского интервала нечётна. У рекурсивного дерева на фактическом диапазоне $[l, r)$ середина равна $m = l + (r - l)/2$, поэтому длины — $m - l$ и $r - m$. Использование половины длины без проверки потеряет один элемент.

Пусть исходный массив $[2, 7, 1, 4, 3, 6, 5, 8]$. Добавим 3 к $[0, 4)$. Сумма левого ребёнка изменится с 14 до 26, его метка станет 3, общая сумма — 48. Листья этой половины пока не переписаны. Запрос всей половины сразу вернёт 26.

Для запроса $[2, 3)$ надо спуститься. Метка левого ребёнка передаётся его детям, а затем по нужному пути — листу. Значение позиции 2 становится доступно как 4. Другие поддеревья могут продолжать хранить собственные метки: проталкивание нужно по требованию, а не во всём дереве после каждого изменения.

Интервальная модификация не обещает более дешёвого вывода всего массива: вывод n значений сам требует $\Omega(n)$ времени. Выигрыш относится к операциям, результат которых можно описать компактной сводкой.

3.3. Присваивание ломает коммутативность

Добавления можно суммировать в любом порядке. С присваиванием это неверно. Для элемента $x = 2$ операции «присвоить 5, прибавить 3» дают 8. Операции «прибавить 3, присвоить 5» дают 5. Поэтому метка должна хранить именно функцию преобразования, а правило её обновления — порядок исполнения.

Есть удобное общее представление:

$$f(x) = ax + b, \qquad a \in \{0, 1\}.$$

Прибавление d — пара $(1, d)$; присваивание c — $(0, c)$; отсутствие изменения — $(1, 0)$. Если раньше отложена $g(x) = a_g x + b_g$, а теперь приходит $f(x) = a_f x + b_f$, детям позднее нужно применить сначала g, затем f:

$$f(g(x)) = (a_f a_g)x + (a_f b_g + b_f).$$

Порядок аргументов — часть контракта. Будем говорить «новая функция после старой». При полном покрытии обновим сумму как $S' = a_f S + b_f k$, а метку заменим композицией $f \circ g$.

Старая метка	Новая операция	Итоговая метка
Добавить p	Добавить d	Добавить $p + d$
Присвоить c	Добавить d	Присвоить $c + d$
Добавить p	Присвоить c	Присвоить c
Присвоить c	Присвоить z	Присвоить z

Нулевое присваивание — полноценная операция $(0, 0)$. Его нельзя путать с отсутствием метки $(1, 0)$. Использование одного числа с соглашением «ноль означает нет присваивания» сделало бы невозможным корректное присваивание нуля.

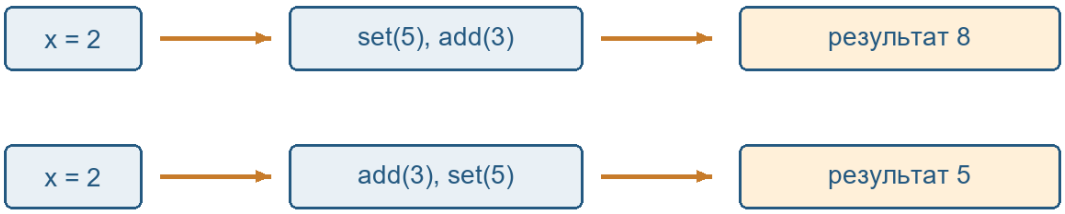


Рисунок 5: Композиция зависит от порядка: set(5), add(3) и add(3), set(5) дают разные результаты.

3.4. Реализация: один способ применять метку

Весь код, меняющий сумму и отложенную функцию узла, сосредоточен в `apply`. Такое устройство важно для корректности: обновление из пользовательской операции и обновление при проталкивании обязаны использовать одинаковое правило.

```
class LazySumTree {
    struct Node {
        long long sum = 0, mul = 1, add = 0;
    };
    int n;
    std::vector<Node> t;
    void build(int v, int l, int r, const std::vector<long long>& a) {
        if (r - l == 1) { t[v].sum = a[l]; return; }
        int m = l + (r - l) / 2;
        build(v * 2, l, m, a);
        build(v * 2 + 1, m, r, a);
        pull(v);
    }
    void pull(int v) { t[v].sum = t[v*2].sum + t[v*2+1].sum; }
    // Apply NEW transformation x -> a*x+b after the stored one.
    void apply(int v, int len, long long a, long long b) {
        t[v].sum = a * t[v].sum + b * len;
        t[v].mul = a * t[v].mul;
        t[v].add = a * t[v].add + b;
    }
    void push(int v, int l, int r) {
        int m = l + (r - l) / 2;
        apply(v*2, m-l, t[v].mul, t[v].add);
        apply(v*2+1, r-m, t[v].mul, t[v].add);
        t[v].mul = 1; t[v].add = 0;
    }
```

`push` вызывается только при частичном пересечении. Лист длины 1 при целочисленных границах не может пересекаться с запросом частично: он либо внутри, либо снаружи. Поэтому у него проталкивание не вызывается. Это свойство алгоритма, а не предположение о том, что обращение к несуществующим детям безвредно.

Проталкивание сохраняет сумму родителя. Новые суммы детей после одного и того же аффинного преобразования в сумме равны прежней актуальной сумме родителя. После очистки родительской метки обычное равенство сумм вновь выполняется.

3.5. Изменение и запрос

Следующий листинг продолжает класс из пункта 3.4. Полный компилируемый файл с заголовками — `code/03-lazy-tree.hpp`.

```
    void update(int v, int l, int r, int ql, int qr,
                long long a, long long b) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) { apply(v, r-l, a, b); return; }
        push(v, l, r);
        int m = l + (r-l)/2;
        update(v*2, l, m, ql, qr, a, b);
        update(v*2+1, m, r, ql, qr, a, b);
        pull(v);
    }
    long long query(int v, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return t[v].sum;
        push(v, l, r);
        int m = l + (r-l)/2;
        return query(v*2, l, m, ql, qr)
             + query(v*2+1, m, r, ql, qr);
    }
    void bounds(int l, int r) const { assert(0 <= l && l <= r && r <= n); }
public:
    explicit LazySumTree(const std::vector<long long>& a)
        : n(int(a.size())), t(4 * (n > 0 ? n : 1)) {
        if (n) build(1, 0, n, a);
    }
    void add(int l, int r, long long x) {
        bounds(l,r);
        if (l < r) update(1,0,n,l,r,1,x);
    }
    void set(int l, int r, long long x) {
        bounds(l,r);
        if (l < r) update(1,0,n,l,r,0,x);
    }
    long long sum(int l, int r) {
        bounds(l,r);
        return l == r ? 0 : query(1,0,n,l,r);
    }
};
```

При полном покрытии обновление заканчивается вызовом `apply`. При частичном сначала делаем детей актуальными относительно родителя, затем рекурсивно изменяем нужные части и пересчитываем сумму. Пересчёт **до** передачи метки затёр бы ещё не отражённые в детях изменения.

Запрос также проталкивает метки, поэтому в реализации он не `const`. Логический массив не меняется; меняется только распределение отложенной работы внутри представления. Публичный интерфейс не обещает неизменности внутренней памяти при чтении.

Для пустого массива конструктор не строит дерево. Пустые операции завершаются в публичных методах и не запускают рекурсию по диапазону нулевой длины. Это предотвращает бесконечное деление пустого интервала.

Доказательство обновления. Вне запроса состояние не меняется. При полном покрытии формула суммы точна для всех элементов диапазона, а композиция сохраняет обязанность перед детьми. При частичном покрытии `push` восстанавливает нужное соответствие детей текущему состоянию; индуктивное предположение применяется к обоим рекурсивным вызовам; `pull` объединяет правильные суммы. Корень поэтому всегда представляет весь логический массив.

Как и в предыдущей главе, на каждом уровне не более двух частично пересечённых узлов. Полностью покрытые поддеревья не обходятся, `apply` и `push` постоянны. Значит, каждая из трёх операций имеет $O(\log n)$ времени в худшем случае, построение и память — $O(n)$.

3.6. Подробная трасса смешанных операций

Начнём с $[2, 7, 1, 4]$. Таблица описывает **логическое** содержимое; листья дерева не обязаны сразу принимать эти значения.

Операция	Логический массив	Сумма всего
Исходное состояние	2, 7, 1, 4	14
`set(0,4,5)`	5, 5, 5, 5	20
`add(1,3,2)`	5, 7, 7, 5	24
`set(2,4,0)`	5, 7, 0, 0	12
`add(0,4,-3)`	2, 4, −3, −3	0

После первого шага только корень обязательно имеет сумму 20 и метку $(0, 5)$. Второй шаг частично пересекает корень, поэтому присваивание передаётся обеим половинам. Дальше аналогичное проталкивание происходит в половинах, чтобы отдельно изменить позиции 1 и 2.

Третий шаг целиком покрывает правую половину. Новая метка $(0, 0)$ уничтожает накопленную историю изменений этой половины: всем её элементам назначено нулевое значение независимо от прошлого. Последнее прибавление можно оставить у корня; запрос `sum(1,3)` протолкнёт его по нужным путям и вернёт $4 + (-3) = 1$.

Эта короткая трасса содержит отрицательное прибавление, присваивание нуля, полное и частичное покрытия. Она полезнее теста только из положительных добавлений: в нём неверная композиция присваиваний вообще не проявилась бы.

3.7. Пример курса: чётные и нечётные суммы

В старом конспекте `segment_tree.tex`, раздел 2.6.2, дан интерфейс: присвоить значение элементу, прибавить 1 на диапазоне и запросить отдельно суммы чётных и нечётных элементов. Это же содержательное усложнение встречается в iRunner 3764. Здесь одной общей суммы недостаточно.

Храним четыре величины: число чётных c_e, число нечётных c_o, сумму чётных s_e и сумму нечётных s_o. При прибавлении единицы каждый чётный становится нечётным и наоборот:

$$s'_e = s_o + c_o, \quad s'_o = s_e + c_e, \quad c'_e = c_o, \quad c'_o = c_e.$$

Все правые части должны использовать **старые** значения. Если сначала перезаписать s_e, а затем использовать его для вычисления s_o, получится двойное применение изменения.

Например, на $[2, 3, 4]$ начальные сведения: $c_e = 2$, $s_e = 6$, $c_o = 1$, $s_o = 3$. После прибавления 1 массив равен $[3, 4, 5]$; новые чётная и нечётная суммы равны 4 и 8. Формулы дают $3 + 1 = 4$ и $6 + 2 = 8$.

Для произвольного прибавления d сначала увеличиваются суммы на d умножить на соответствующие количества. Если d нечётно, две категории меняются местами. В C++ для проверки нечётности отрицательного числа используйте `d % 2 != 0`, а не `d % 2 == 1`. Композиция таких прибавлений по-прежнему складывает d.

Точечное присваивание спускается до листа, предварительно проталкивая метки предков. Лист получает одну ненулевую категорию по чётности нового значения. После этого сводки предков объединяются покомпонентным сложением. Реализацию этого расширения оставляем самостоятельной: она проверяет понимание инварианта, а не копирование готового шаблона.

Другой пример старого конспекта — перекраска полоски с запросом цвета точки. Если нужны только присваивания диапазона и чтение одной позиции, хранить сумму не требуется. Более слабый интерфейс допускает более простую сводку. Не добавляйте поля только потому, что они были в предыдущем решении.

3.8. Геометрия, ограничения и проверка

Интервальные структуры естественно появляются в методе сканирующей прямой. Например, при подсчёте площади объединения прямоугольников вертикальные границы создают события добавления и удаления интервалов по y. Между соседними событиями площадь увеличивается на длину покрытого объединения, умноженную на расстояние по x.

Однако длина объединения не является обычной суммой счётчиков покрытий. Нужны число покрывающих весь узел интервалов и покрытая длина. Если счётчик положителен, покрыт весь геометрический интервал; иначе длина складывается из детей. После сжатия координат длина листа равна разности соседних координат, а не единице. Это отдельная сводка с отдельным доказательством, а не автоматическое применение приведённого класса.

Для основного класса эталон — обычный вектор. Выполняйте каждую модификацию поэлементно, после неё сравнивайте диапазонные суммы. Обязательно включите последовательности `set→add`, `add→set`, `set→set`, присваивание нуля, изменение всего массива перед изменением одного элемента, вложенные и непересекающиеся диапазоны.

1. Что уже учтено в сумме узла, а что ещё не учтено в детях?
2. Почему запрос при полном покрытии не должен прибавлять метку ещё раз?
3. Когда равенство суммы родителя и сумм детей временно неверно?
4. Получите формулу композиции новой функции после старой.
5. Чем отличаются пары $(0, 0)$ и $(1, 0)$?
6. Почему частичный запрос может менять внутреннее состояние дерева?
7. Проследите запрос $[1, 3)$ после всех шагов таблицы.
8. Выведите формулы обновления чётных и нечётных сумм.
9. Почему высоты дерева недостаточно для оценки, если полное покрытие не останавливает спуск?
10. Какие дополнительные сведения нужны для длины объединения геометрических интервалов?

Происхождение. Основной механизм и примеры перекраски и чётности опираются на локальный конспект дерева отрезков. Числовая трасса и единая аффинная запись меток — редакторская разработка для этой главы. В записи 20.09.2022, 45:00–48:00, рассмотрен другой приём — глобальное присваивание через версии ячеек; его разбираем отдельно ниже.

3.9. Разбор: что происходит при частичном запросе

Начнём с $[2, 7, 1, 4]$ и выполним присваивание 5 на всём массиве. В корне теперь сумма 20 и метка $(0, 5)$. Значения детей могут по-прежнему содержать старые суммы 9 и 5. Это допустимо: корень уже отвечает правильно за весь диапазон, а метка объясняет, какое преобразование ещё не отражено ниже.

Запрос $[1, 3)$ не совпадает с корнем. Сначала `push` применяет присваивание к обоим детям. Их суммы становятся по 10, их метки — $(0, 5)$, а метка корня очищается до $(1, 0)$. Сумма корня остаётся 20. Затем запрос спускается в обе половины и аналогично передаёт присваивание листьям. Листья 1 и 2 возвращают по 5; результат равен 10.

Теперь выполним прибавление 2 на $[1, 3)$. Листья 1 и 2 становятся равны 7. При возврате пересчитываются суммы половин: обе равны 12, сумма корня — 24. Если бы мы пересчитали корень сразу после исходного полного присваивания, до передачи метки, получили бы старую сумму 14. Ошибка возникает не в сложении, а в нарушении условия, при котором сведения детей уже актуальны.

Момент	Сумма корня	Суммы детей	Метка корня
Исходный массив	14	9, 5	$(1, 0)$
Полное присваивание 5	20	9, 5	$(0, 5)$
Передача к детям	20	10, 10	$(1, 0)$
Частичное прибавление 2	24	12, 12	$(1, 0)$

Эта таблица — редакторская трасса реализации. Её полезно воспроизвести отладочным выводом узлов, а затем убрать вывод перед измерением времени. Не сравнивайте каждый внутренний узел напрямую с массивом без учёта меток предков: ещё не переданное преобразование может законно делать его локальную запись устаревшей относительно всего массива.

Для независимой проверки сравнивайте публичные ответы с простым вектором. Чтобы дополнительно проверить внутреннее представление, можно сначала передать все метки до листьев, затем проверить их значения и восстановленные суммы. Такой полный обход стоит $O(n)$ и относится к диагностике, а не к обычному запросу структуры.

3.10. Углубление: какие преобразования допускает сводка

Наличие функции над отдельным элементом ещё не означает, что её можно быстро применить к узлу. Требуется вычислять новую сводку по старой сводке и метке, не читая все элементы диапазона. Для суммы и прибавления это возможно, если известна длина: $S' = S + d \cdot len$. Без длины суммы одинаковых значений не отличают массив $[4]$ от $[2, 2]$, но после прибавления единицы их суммы различаются.

Для присваивания достаточно той же длины: $S' = x \cdot len$. Для аффинного преобразования $x \mapsto ax + b$ получаем $S' = aS + b \cdot len$. В нашей реализации используются только $a = 0$ и $a = 1$. Разрешение произвольного умножения потребует пересмотра границ чисел: многократная композиция быстро увеличивает коэффициенты. В модульной арифметике можно хранить коэффициенты по модулю, но это уже другой числовой контракт.

У корректного действия меток есть три свойства. Единичная метка оставляет сводку неизменной. Применение старой, затем новой метки совпадает с применением их композиции. Наконец, обработка объединённого диапазона совпадает с объединением обработанных половин с их собственными длинами. Последнее свойство связывает алгебру меток с алгеброй `merge` и объясняет, почему `push` сохраняет сумму родителя.

Контрпример особенно полезен для новых операций. Пусть требуется заменить каждое значение на $\min(x, 3)$, а узел хранит только сумму и длину. Массивы $[0, 6]$ и $[3, 3]$ имеют одинаковую длину 2 и сумму 6. После преобразования их суммы равны 3 и 6. Значит, по прежней сводке невозможно восстановить ответ: никакая формула от этих двух чисел не исправит потерю информации.

Это не доказательство невозможности эффективной структуры вообще. Можно хранить дополнительные сведения и применять преобразование только при специальных условиях; такие конструкции требуют отдельного анализа. Но обычный шаблон lazy propagation не превращает произвольное поэлементное изменение в логарифмическую операцию.

Похожая ловушка — одновременно поддерживать прибавление на диапазоне и максимальную сумму подотрезка из второй главы. Кандидат длины k изменяет сумму на $d \cdot k$, поэтому после прибавления может победить другой подотрезок. Четырёх полей `sum`, `pref`, `suff`, `best` недостаточно для механического обновления прежнего оптимума. Сначала докажите замкнутость сводки относительно преобразования; только затем пишите `apply`.

Самостоятельная проверка модели. Для суммы, минимума и пары «минимум, число вхождений» определите, какие из операций `add`, `set`, умножение на положительное число и умножение на ноль допускают обновление за $O(1)$. Для каждого разрешённого случая выпишите формулу. Обратите внимание: умножение на ноль делает все элементы минимальными, поэтому прежней кратности минимума недостаточно без знания длины.

3.11. Пример из видео: глобальное присваивание через версии

В расшифрованном фрагменте лекции 20.09.2022, примерно 46:07–47:58, предлагаются два массива: значения и версии ячеек. Структура хранит также текущую версию и общее значение по умолчанию. Это особенно простой пример отложенного изменения: операция затрагивает весь массив, а чтение обращается только к одной позиции.

Обозначим текущую версию $epoch$, значение по умолчанию $default$, локальные записи $value[i]$ и $stamp[i]$. После `assign_all(x)` увеличиваем $epoch$ и устанавливаем $default = x$. После `set(i,x)` записываем $value[i] = x$ и $stamp[i] = epoch$. При `get(i)` возвращаем локальное значение, если версии совпадают, иначе — общее значение по умолчанию. Последнее правило следует из инварианта; конец выбранного звукового фрагмента обрывается во время его объяснения.

Числовой пример ниже редакторский. Пусть размер равен 3, начальное значение 0. После `set(1,7)` логический массив равен $[0, 7, 0]$. Затем `assign_all(5)` делает его равным $[5, 5, 5]$, хотя число 7 физически остаётся в старой ячейке. Версия этой ячейки больше не актуальна, поэтому чтение не возвращает 7. После `set(2,9)` получаем $[5, 5, 9]$; только позиция 2 имеет локальную запись текущей версии.

Инвариант разделяет два случая: ячейка либо записана после последнего глобального присваивания, либо наследует его значение. Присваивание всему массиву меняет границу актуальности сразу для всех ячеек, а точечная запись переводит одну ячейку в первый случай. После начального выделения и инициализации $O(n)$ все три операции постоянны по времени, память линейна.

Счётчик версий не должен переполняться: иначе старый `stamp` может снова совпасть с текущим. Контракт ограничивает число глобальных присваиваний. Полный сброс штампов при исчерпании счётчика возможен, но стоит $O(n)$ и меняет гарантию отдельной операции.

Приём обслуживает глобальное присваивание и точечное чтение. Произвольные перекрывающиеся диапазоны требуют отдельных отложенных состояний в разных частях — такую иерархию создаёт дерево. Общая идея сохраняется: логическое изменение не обязано немедленно переписывать все ячейки.

§ 4. Битовые множества и массовые операции

Когда элементы принадлежат небольшому известному универсуму, множество можно хранить как последовательность флагов. Машинное слово объединяет десятки таких флагов: одна операция OR или AND обрабатывает сразу несколько элементов. Выигрыш возникает не от изменения математического смысла задачи, а от упаковки однотипной работы.

После главы вы сможете реализовать динамический битсет на C++17, оценивать его стоимость по числу слов, применять сдвиги к достижимым суммам и строить битовые варианты BFS, транзитивного замыкания и исключения Гаусса над GF(2).

4.1. Интерфейс множества и размер универсума

Пусть $U = \{0, 1, \dots, m-1\}$. Нужны вставка, удаление, проверка принадлежности, объединение, пересечение и разность. Число элементов множества k может быть намного меньше m. Эти параметры нельзя смешивать: битсет расходует память на весь универсум, даже если в нём содержится только один элемент.

Массив логических флагов уже даёт постоянный доступ по номеру. Упакуем флаги в слова ширины $w = 64$. Для элемента x номер слова равен $\lfloor x/64 \rfloor$, номер бита — $x \bmod 64$. Всего нужно $M = \lceil m/64 \rceil$ слов.

Инвариант представления: бит $x \bmod 64$ слова $x/64$ равен единице тогда и только тогда, когда x принадлежит множеству. Неиспользуемые старшие биты последнего слова всегда нулевые. Последнее условие особенно важно после сдвигов и дополнения: иначе подсчёт элементов может включить объекты вне универсума.



Рисунок 6: Адрес элемента распадается на номер слова и позицию бита; граница слова не является особым элементом множества.

Для учебной ширины $w = 8$ и $m = 20$ элемент 13 находится в слове 1, бите 5; элемент 16 — в слове 2, бите 0. Элемент 20 недопустим. В реальном коде ширина 64, но рассуждение остаётся тем же.

Пример из видео. В записи 28.09.2021 около 16:13 видны `Insert(x)` и `Test(x)` с номером слова и маской. Ключевой переход — от интерфейса множества к расположению бита. Значения 13 и 16 здесь выбраны для ясного показа границы, а не скопированы с доски.

4.2. Почему вставка — OR, а не сложение

Маска элемента — `std::uint64_t{1} << (x % 64)`. Вставка выполняет OR с маской, удаление — AND с инверсией маски, проверка — выделение соответствующего бита. Повторная вставка должна быть идемпотентной: множество не меняется, если элемент уже присутствует.

Возьмём восьмибитное слово `00000100`, представляющее элемент 2. OR с той же маской оставит `00000100`. Арифметическое сложение даст `00001000`: элемент 2 исчез, появился элемент 3. Перенос разрушил смысл множества. Это не редкий граничный случай, а нарушение основного контракта.

Именно такой смысл обсуждается в коротком фрагменте около 15-й минуты записи 28.09.2021: сложение безопасно лишь при дополнительной гарантии, что бит ещё не установлен. В интерфейсе обычного множества такой гарантии нет, поэтому используем OR.

Для $A = \{0, 2, 5\}$ и $B = \{2, 3, 5\}$ маски равны `00100101` и `00101100`. OR даёт `00101101`, то есть $\{0, 2, 3, 5\}$; AND даёт `00100100`, то есть $\{2, 5\}$; XOR даёт $\{0, 3\}$. Порядок записи двоичного числа идёт от старшего бита слева к младшему справа: индекс элемента не равен позиции символа в строке.

Операция	Одно слово	Всё множество
Вставка или удаление элемента	Одна маска и логическая операция	$O(1)$
Проверка принадлежности	Сдвиг или AND	$O(1)$
Объединение, пересечение, разность	OR, AND, AND-NOT	$O(M)$
Сдвиг на заданное число позиций	Возможен перенос между словами	$O(M)$

Фраза «битовые операции работают за константу» относится к одному машинному слову. Множество на миллион возможных элементов занимает много слов, и целиком обработать его за одну инструкцию нельзя.

4.3. Безопасная реализация на C++17

Класс `Bits` разделён на несколько последовательных листингов; метод сдвига в пункте 4.4 завершает тот же класс. Полный исходник с заголовками — `code/04-bitset.hpp`.

Полный класс находится в `code/04-bitset.hpp`. Используем беззнаковый тип фиксированной ширины. В отличие от `1 << bit`, запись `std::uint64_t{1} << bit` сразу выбирает нужную ширину левого операнда. Сдвиг на 64 для 64-битного слова недопустим, поэтому все такие выражения требуют проверки.

```
class Bits {
    int n;
    std::vector<std::uint64_t> w;
    void trim() {
        if (n % 64) w.back() &= (std::uint64_t{1} << (n % 64)) - 1;
    }
public:
    explicit Bits(int size) : n(size), w((size + 63) / 64, 0) { assert(size >= 0); }
    int size() const { return n; }
    bool test(int x) const {
        assert(0 <= x && x < n);
        return (w[x/64] >> (x%64)) & 1;
    }
    void set(int x) {
        assert(0 <= x && x < n);
        w[x/64] |= std::uint64_t{1} << (x%64);
    }
    void reset(int x) {
        assert(0 <= x && x < n);
        w[x/64] &= ~(std::uint64_t{1} << (x%64));
    }
```

Операции над двумя множествами требуют одинаковых универсумов. Их нельзя молча объединять, сравнивая только общие слова: это скрыло бы ошибку контракта. Учебный класс использует `assert`; предусловия — $0 \le m \le 10^6$, допустимые индексы и неотрицательные сдвиги.

```
    void unite(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] |= b.w[i];
    }
    void intersect(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] &= b.w[i];
    }
    void subtract(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] &= ~b.w[i];
    }
    void xor_with(const Bits& b) {
        assert(n == b.n);
        for (std::size_t i=0; i<w.size(); ++i) w[i] ^= b.w[i];
    }
    int count() const {
        int result=0;
        for (auto x:w) while (x) { x &= x-1; ++result; }
        return result;
    }
```

`count` использует тождество `x & (x-1)`: оно удаляет младший установленный бит. Поэтому внутренний цикл выполняется по числу единиц, а не по числу всех позиций. Для M слов и k единиц получаем $O(M + k)$ элементарных действий; при фиксированном $w = 64$ это также $O(M)$ с константой до 64. Это не обязательно одна аппаратная инструкция на слово.

В C++20 есть `std::popcount`, а в C++17 можно встретить расширение компилятора `__builtin_popcountll`. Здесь выбран переносимый вариант, чтобы контракт не зависел от расширения компилятора. Замена локальной функции подсчёта не меняет логику множества.

4.4. Сдвиг и достижимые суммы

Сдвиг на d переводит каждый элемент x в $x+d$, отбрасывая результаты вне универсума. Если $d = 64q+r$, то слово перемещается на q слов, а при $r \neq 0$ делится между двумя соседними словами результата.

```
    Bits shifted(int d) const {
        assert(d >= 0);
        Bits b(n);
        if (d >= n) return b;
        int whole=d/64, part=d%64;
        for (int i=0; i<int(w.size()); ++i) {
            int j=i+whole;
            if (j<int(w.size())) b.w[j] |= w[i] << part;
            if (part && j+1<int(w.size()))
                b.w[j+1] |= w[i] >> (64-part);
        }
        b.trim();
        return b;
    }
};
```

При `part == 0` вторую запись пропускаем: выражение `>> (64-part)` тогда означало бы сдвиг на 64. При $d \geq m$ результат пуст. Последний вызов `trim` убирает биты за границей универсума. Исходный объект не меняется, поэтому сдвиг пригоден для перехода динамического программирования без смешивания старого и нового состояния.

Пусть бит s означает, что сумма s достижима некоторым подмножеством уже обработанных предметов. Изначально достижима только сумма 0. Для нового неотрицательного веса d обновление имеет вид

```
Bits shifted = reachable.shifted(d);
reachable.unite(shifted);
```

Результат содержит либо старую сумму без нового предмета, либо старую сумму плюс его вес. Все новые биты вычислены из **старого** состояния, поэтому каждый предмет используется не более одного раза. Если обновлять обычный массив достижимости слева направо, один и тот же предмет мог бы использоваться повторно.

Для весов 3, 2, 3 и максимальной суммы 8 получаем состояния $\{0\}$, затем $\{0,3\}$, затем $\{0,2,3,5\}$ и наконец $\{0,2,3,5,6,8\}$. Два предмета веса 3 различны, поэтому сумма 6 допустима. Нулевой вес не меняет множество достижимых сумм. Отрицательные веса требуют другого диапазона индексов и здесь не поддерживаются.

Для n предметов и суммы до W время равно $O(n\lceil (W+1)/64 \rceil)$, память — $O(\lceil (W+1)/64 \rceil)$ слов, включая временный сдвиг с постоянным множителем. При огромном W такой подход непригоден, даже если предметов мало: в главе 6 используем meet-in-the-middle.

В расшифровке видео соответствующий сюжет находится примерно на 33–38-й минутах. Числовая трасса выше — редакторское упражнение; она проверяет смысл сдвига независимо от ошибок ASR.

4.5. Поиск в ширину: целый слой как множество

В ориентированном невзвешенном графе обычный BFS строит слои $L_d = \{v : dist(s,v) = d\}$. Пусть `adj[v]` — битовое множество исходящих соседей. Тогда следующий слой равен объединению соседей текущего слоя за вычетом уже посещённых вершин.

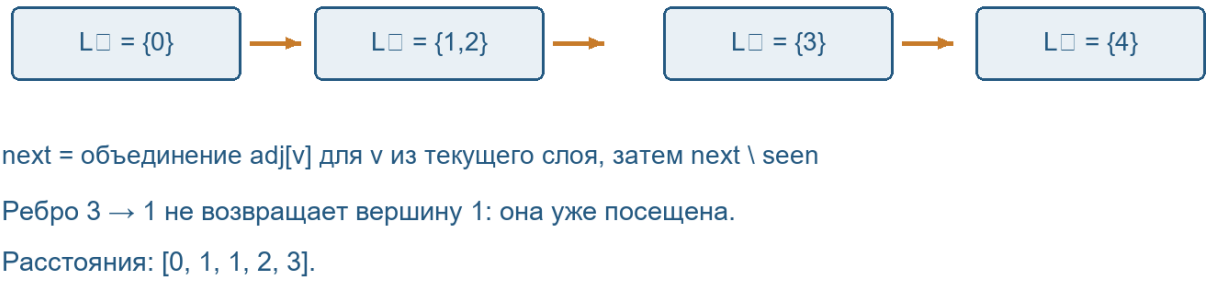


Рисунок 7: BFS собирает соседей всего слоя и исключает посещённые вершины до назначения новых расстояний.

Возьмём рёбра $0 \to 1, 0 \to 2, 1 \to 3, 2 \to 3, 3 \to 1, 3 \to 4$. Получаем $L_0 = \{0\}$, $L_1 = \{1, 2\}$, $L_2 = \{3\}$, $L_3 = \{4\}$. Обратное ребро $3 \to 1$ не возвращает вершину 1 в очередь: она уже посещена. Два пути к 3 также не создают двух копий вершины, потому что OR объединяет множества.

```
inline std::vector<int> bit_bfs(const std::vector<Bits>& adj, int s) {
    int n=int(adj.size());
    assert(0 <= s && s < n);
    for (const auto& row:adj) assert(row.size()==n);
    Bits seen(n);
    seen.set(s);
    std::vector<int> frontier{s}, dist(n,-1);
    dist[s]=0;
    int depth=0;
    while (!frontier.empty()) {
        Bits next(n);
        for (int v:frontier) next.unite(adj[v]);
        next.subtract(seen);
        seen.unite(next);
        frontier.clear();
        ++depth;
        for (int v=0; v<n; ++v)
            if (next.test(v)) { frontier.push_back(v); dist[v]=depth; }
    }
    return dist;
}
```

Инвариант: перед итерацией `frontier` содержит ровно вершины текущего слоя, а `seen` — все уже обнаруженные вершины до этого слоя включительно. Каждый новый сосед имеет путь длины на единицу больше; вершины с более коротким путём уже исключены. Поэтому первое присвоенное расстояние кратчайшее.

Важно честно посчитать реализацию. Строка каждой достижимой вершины объединяется один раз: $O(nM)$ словных операций в худшем случае. Здесь новый слой перечисляется простым просмотром всех n позиций. Если число непустых слоёв равно D, это ещё $O(nD)$. Итого $O(nM + nD)$, а при длинной цепи остаётся $O(n^2)$. Упаковка не даёт автоматического ускорения любого графа.

Перечисление установленных битов по словам может сократить стоимость выделения новых вершин, но требует дополнительного аккуратного кода. Для разреженного графа обычный BFS за $O(n+e)$ часто лучше, а матрица битсетов расходует $O(nM)$ слов памяти. Этот вариант полезен прежде всего при плотных графах и массовых операциях над соседями.

В видео около 48:40 видны слои BFS, посещённые вершины и граф. Сохраняем именно послойное объяснение; наш граф и код — новая проверяемая иллюстрация. Задача iRunner 3911 «Сумма расстояний» требует дополнительного чтения условия: это не просто вывод массива расстояний от одного источника.

4.6. Транзитивное замыкание

Теперь хотим ответить на все вопросы достижимости, не вычисляя длины путей. Пусть строка `reach[i]` содержит вершины, достижимые из i. На шаге k разрешаем вершину k как промежуточную: если i уже достигает k, добавляем к строке i всю строку k.

```
inline std::vector<Bits> closure(std::vector<Bits> reach) {
    int n=int(reach.size());
    for (int i=0; i<n; ++i) { assert(reach[i].size()==n); reach[i].set(i); }
    for (int k=0; k<n; ++k)
        for (int i=0; i<n; ++i)
            if (reach[i].test(k)) reach[i].unite(reach[k]);
    return reach;
}
```

В данной версии вершина достижима из себя по пути длины 0. Поэтому перед циклом ставим диагональные биты. Если задача требует только непустые пути, это соглашение надо изменить осознанно и отдельно проверить циклы.

Индуктивный инвариант алгоритма Уоршалла: перед шагом k учтены пути, внутренние вершины которых лежат среди $0, \dots, k-1$. Новый путь либо уже существовал, либо распадается в k на путь из i в k и из k в j. Объединение строк реализует вторую возможность сразу для всех j.

Обновление на месте допустимо. На шаге k строка k объединяется только с самой собой и не меняется; изменение бита достижения k в других строках не создаёт новой возможности, которой не было до шага. Стоимость — $O(n^2 M)$ словных операций и $O(nM)$ слов памяти. Это ускорение внутреннего цикла классического кубического алгоритма, а не переход к линейному времени.

4.7. Линейная алгебра над GF(2)

В поле GF(2) сложение и вычитание совпадают с XOR, а коэффициенты равны 0 или 1. Строку двоичной матрицы естественно хранить битсетом. Вычитание опорной строки из другой превращается в XOR целых слов.

```
inline int gf2_rank(std::vector<Bits> rows, int columns) {
    int rank=0, m=int(rows.size());
    for (const auto& row:rows) assert(row.size()==columns);
    for (int c=0; c<columns && rank<m; ++c) {
        int p=rank;
        while (p<m && !rows[p].test(c)) ++p;
        if (p==m) continue;
        std::swap(rows[p],rows[rank]);
        for (int i=rank+1; i<m; ++i)
            if (rows[i].test(c)) rows[i].xor_with(rows[rank]);
        ++rank;
    }
    return rank;
}
```

После каждого найденного ведущего столбца одна новая строка становится опорной. Из нижних строк устраняется единица в этом столбце; прежние ведущие столбцы остаются нулевыми, поскольку опорная строка также имеет там нули. Число найденных ведущих позиций равно рангу.

Для строк `110`, `011`, `101` третья равна XOR первых двух, поэтому ранг равен 2. Независимость проверяется математическим равенством строк, а не количеством единиц. Порядок битов при печати надо согласовать с порядком столбцов: в примере строки записаны как обычные тройки коэффициентов слева направо.

Для системы $Ax = b$ добавляют последний бит правой части, но ведущие позиции ищут только среди столбцов переменных. Строка с нулевыми коэффициентами и правой частью 1 означает несовместность. Если противоречия нет, число свободных переменных равно числу переменных минус ранг; обратный ход восстанавливает решение после выбора свободных значений. Приведённая функция вычисляет только ранг коэффициентной матрицы и не выдаёт решение системы.

При r строках и c столбцах поиск ведущих строк занимает до $O(rc)$ проверок битов; исключение — $O(r \min(r, c)\lceil c/64 \rceil)$ словных операций. Логические операции сокращают стоимость обработки строки, но не отменяют поиск ведущих позиций.

4.8. Проверка, границы применимости и задания

Особенно полезны размеры универсума 0, 1, 63, 64, 65, 127 и 129. Они ловят переходы между словами и маскирование последнего слова. Для сдвига проверьте 0, 1, 63, 64, 65, m и $m + 1$. Для каждой операции сравнивайте результат с массивом флагов; проверяйте все позиции, а не только `count`: одинаковое количество единиц не означает одинаковое множество.

BFS сравнивайте с обычной очередью на случайных маленьких графах; замыкание — с запуском обычного BFS из каждой вершины; достижимые суммы — с полным перебором подмножеств. Для ранга маленькой матрицы независимо посчитайте число разных XOR-комбинаций строк: оно равно 2^{rank}. Это действительно иной способ проверки, а не переписывание того же исключения Гаусса.

1. Чем отличаются размер множества и размер универсума?
2. Где находятся элементы 63 и 64 в словном представлении?
3. Покажите ошибку повторной вставки через сложение.
4. Почему инверсия последнего слова требует внимания к лишним битам?
5. Объясните отдельный случай сдвига, кратного 64.
6. Почему временная копия сдвига предотвращает многократное использование предмета?
7. Зачем исключать посещённые вершины до формирования следующего слоя BFS?
8. Выведите стоимость именно приведённой реализации BFS, включая перечисление слоя.
9. Какое соглашение о достижимости вершины из себя принято в `closure`?
10. Чем ранг матрицы отличается от решения системы с правой частью?

Практика. Реализуйте множество и независимую модель на флагах, затем добавьте задачу достижимой суммы. BFS и GF(2) образуют углублённую часть недели. В следующей главе к вопросу выбора представления добавится порог: часть данных выгодно обрабатывать непосредственно, а часть — через заранее подготовленные сводки.

§ 5. Декомпозиция данных и запросов

В дереве отрезков иерархия делит диапазон на всё более мелкие части. У блочной структуры обычно только два масштаба: отдельные элементы и блоки примерно одинакового размера. Она выигрывает, когда сводку блока легко пересчитать, а более сложное дерево требует неудобного объединения или дорого поддерживает изменения.

Цель параграфа — научиться выводить размер блока из стоимости операций, различать разбиение по позициям и по значениям, ускорять прыжки и отделять тяжёлые объекты от лёгких. Нужны полуинтервалы, двоичный поиск, асимптотика и метод амортизации из § 1. Сначала рассмотрим статичное расположение элементов, затем перейдём к изменению самой последовательности.

5.1. Интерфейс и две границы запроса

Поддержим массив длины n, точечное присваивание `set(i,x)` и сумму `sum(l,r)` на $[l, r)$. Выберем целое $B \geq 1$. Блок b содержит позиции $[bB, \min(n, (b+1)B))$; последний блок может быть короче. Сохраним сам массив и сумму каждого блока. Инвариант: сводка блока равна сумме его текущих элементов.

Запрос состоит из неполного левого края, нескольких целых блоков и неполного правого края. Края читаем поэлементно, целые блоки — по сводке. Если обе границы лежат внутри одного блока, целых блоков нет; элементы нельзя учитывать дважды. Пустой запрос возвращает ноль.

Для $A = [2, 7, 1, 4, 3, 6, 5, 8, 9, 0, 2, 1]$, $B = 4$ суммы блоков равны 14, 22 и 12. Запрос $[2, 11)$ берёт элементы 1 и 4, целый блок с суммой 22 и элементы 9, 0, 2. Ответ равен 38. Присваивание `set(5,10)` меняет только второй блок: его сумма становится 26; тот же запрос теперь возвращает 42.

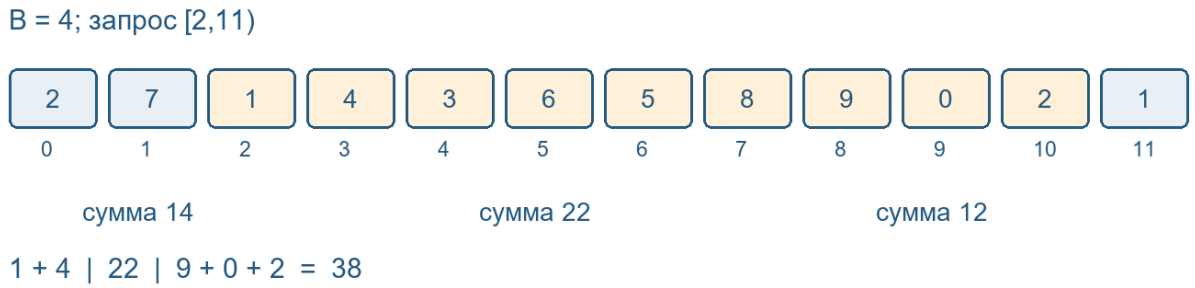


Рисунок 8: Два неполных края читаются поэлементно; середина запроса покрывается сводкой.

Стоимость запроса $O(B + n/B)$: на краях меньше $2B$ элементов, в середине не больше $\lceil n/B \rceil$ сводок. Построение и память линейны. Для суммы точечное присваивание обновляет блок разностью нового и старого значений, поэтому стоит $O(1)$. Это свойство именно суммы: для минимума удаление единственного минимального значения может потребовать просмотра всего блока.

5.2. Реализация суммы на C++17

Полный код находится в `code/05-decomposition.hpp`. Конструктор получает размер блока явно: это параметр модели нагрузки, а не обязательная скрытая константа. Массив может быть пустым. Все значения, разности и суммы должны помещаться в `long long`; длина ограничена миллионом элементов.

```
class BlockSum {
    int n, width;
    std::vector<long long> a, total;
public:
    BlockSum(std::vector<long long> values, int b)
        : n(int(values.size())), width(b), a(std::move(values)) {
        assert(0 <= n && n <= 1000000 && width > 0);
        total.assign(n / width + (n % width != 0), 0);
        for (int i=0; i<n; ++i) total[i/width] += a[i];
    }
    void set(int i, long long x) {
        assert(0 <= i && i < n);
        total[i/width] += x-a[i];
        a[i] = x;
    }
    long long sum(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        long long answer=0;
        while (l<r && l%width) answer += a[l++];
        while (width <= r-l) {
            answer += total[l/width]; l += width;
        }
        while (l<r) answer += a[l++];
        return answer;
    }
};
```

Первый цикл доводит левую границу до границы блока либо до конца запроса. Второй цикл берёт только полные блоки: условие `width <= r-l` не допускает выхода за правую границу. Третий дочитывает остаток. Все три цикла последовательно увеличивают одну границу, так что каждый элемент относится ровно к одной части разложения.

При присваивании к сумме блока прибавляется $x - a[i]$. Разность также должна быть представима в типе: одной гарантии, что исходные значения помещаются в `long long`, недостаточно. Здесь входные ограничения должны ограничивать промежуточную арифметику, как и в дереве отрезков.

Блоки не улучшают асимптотику суммы относительно дерева, но дают короткую реализацию и основу для менее удобных сводок. Выбор структуры нельзя делать только по длине кода: при большом количестве запросов суммы логарифмический алгоритм обычно предпочтительнее.

5.3. Откуда берётся квадратный корень

Для одного запроса оценка $B + n/B$ минимальна по порядку при $B \approx \sqrt{n}$. Это следует из равенства двух слагаемых или из неравенства $B + n/B \geq 2\sqrt{n}$. Не требуется, чтобы n было точным квадратом: округление до ближайшего положительного целого меняет лишь константу.

Но корень возникает из конкретной функции затрат. Пусть сводка требует $O(B)$ при обновлении и $O(1)$ при чтении полного блока. Для u обновлений и q запросов получаем

$$T(B) = O(n + (u + q)B + qn/B).$$

Баланс даёт $B \approx \sqrt{qn/(u+q)}$, с ограничением $1 \leq B \leq \max(1, n)$. Если обновлений гораздо больше запросов, полезны меньшие блоки. Для суммы обновление постоянно и этого дополнительного слагаемого uB нет. Если $q = 0$, оптимизация запросов вообще не определяет выбор параметра.

Реальные коэффициенты тоже различаются. Прямой просмотр последовательной памяти дешевле переходов по указателям; двоичный поиск внутри блока добавляет логарифм; копирование больших объектов дороже копирования целых. Сначала выпишите число действий, затем измеряйте несколько размеров блока на характерной нагрузке. Один удачный замер не доказывает универсальность $B = 700$.

Проверка понимания. Для $n = 10^6$, $q = 10^4$, $u = 10^6$ сравните модель с $B = 1000$ и с $B = 100$. Не требуется точный прогноз секунд: объясните, какое слагаемое уменьшилось и какое выросло. Затем повторите расчёт для суммы с постоянным обновлением.

5.4. Сортировка внутри блока и изменение контракта

Теперь запрос спрашивает число элементов на $[l, r)$, меньших x. Кроме исходного порядка сохраним отсортированную копию каждого блока. На краях сравним отдельные элементы с x; в полном блоке позиция `lower_bound(x)` равна числу подходящих значений. Время запроса $O(B + (n/B)\log(B + 1))$, память $O(n)$.

После изменения одного элемента можно заново отсортировать его блок за $O(B \log(B + 1))$. Более аккуратная реализация удаляет одно вхождение старого значения и вставляет новое в отсортированный вектор. Поиски логарифмические, но сдвиги элементов линейны: итоговое обновление $O(B)$. При дубликатах удаляют ровно одно вхождение, а не все равные значения.

Пусть исходный блок $[7, 2, 7, 1]$, сортированная копия $[1, 2, 7, 7]$. После замены позиции 0 на 3 копия становится $[1, 2, 3, 7]$. Запрос «меньше 7» меняется с 2 на 3. Индексы запроса относятся к исходному порядку; отсортированная копия нужна только для сводки. Сортировка самого массива разрушила бы смысл $[l, r)$.

В лекции 25.10.2022 около 15–17 минут обсуждается близкий, но другой интерфейс: коллекция из отсортированных блоков с вставкой и удалением элементов. В ней размеры блоков меняются, а поиск по позиции требует учитывать длины предыдущих блоков. Не переносите формулу `i/B` в такую структуру: она работает только для фиксированных границ.

Разбор по мотивам видео. При вставке нового значения сдвигается хвост одного блока. Если блок слишком вырос, его делят; слишком маленький присоединяют к соседнему и при необходимости делят результат. Это локализует перенос элементов. В новой расшифровке 16:20–16:41 лектор приводит опасный сценарий: всё время добавлять значения по убыванию, каждый раз в начало одного блока. Без разделения он растёт и перестаёт быть маленьким. Числовая адаптация: вставки 9, 8, 7, 6, 5 в один исходно пустой блок. Ограничение размера должно сработать независимо от того, насколько удобны остальные блоки.

5.5. Прыжки: сводка как переход в следующий блок

Пусть из позиции i выполняется прыжок в $i + a[i]$, причём $a[i] > 0$. Запрос начинает движение в заданной позиции и просит последнюю посещённую позицию и число прыжков до выхода за массив. Разрешено менять одно значение $a[i]$. Это учебная модель темы «Лунки» из плана курса; здесь используется нулевая индексация и явно зафиксирован контракт.

Прямой запрос может посетить весь массив. Для каждой позиции сохраним `next` — первую позицию вне её блока либо n, `count` — число прыжков до неё, `last` — последнюю позицию перед этим выходом. Сводка останавливается на границе блока и не пытается предсказать всю дальнейшую траекторию.

Для $a = [1, 2, 1, 1, 2, 1, 1, 3]$ и $B = 3$ траектория из 0: $0 \to 1 \to 3 \to 4 \to 6 \to 7 \to 10$. Первый блок заменяет два прыжка $0 \to 1 \to 3$ одним переходом сводки; второй — два прыжка $3 \to 4 \to 6$; последний — два прыжка $6 \to 7 \to 10$. Ответ: последняя позиция 7, всего 6 прыжков.

Построим один блок справа налево. Если $j = i + a[i]$ уже вне блока, то `count[i]=1`, `last[i]=i`, а `next[i]` равен j или внешнему маркеру n. Если j внутри, его сводка уже готова, поскольку $j > i$: добавляем один к числу прыжков и наследуем выход и последнюю позицию.

```
class JumpBlocks {
    int n, width;
    std::vector<int> a, next, count, last;
    void rebuild(int block) {
        int l=block*width, r=std::min(n,l+width);
        for (int i=r-1; i>=l; --i) {
            long long j=1LL*i+a[i];
            if (j>=r) {
                next[i]=int(std::min<long long>(n,j));
                count[i]=1; last[i]=i;
            } else {
                next[i]=next[int(j)];
                count[i]=1+count[int(j)];
                last[i]=last[int(j)];
            }
        }
    }
```

Изменение позиции может повлиять на более ранние позиции того же блока, поэтому перестраиваем весь блок за $O(B)$. Другие блоки менять не нужно: их сводки заканчиваются при первом выходе из собственного блока и не зависят от дальнейшей траектории. Во время запроса актуальная сводка следующего блока будет прочитана отдельно.

```
public:
    JumpBlocks(std::vector<int> jumps, int b)
        : n(int(jumps.size())), width(b), a(std::move(jumps)),
          next(n), count(n), last(n) {
        assert(n<=1000000 && 1<=width && width<=1000000);
        for (int x:a) assert(1<=x && x<=1000000000);
        for (int l=0; l<n; l+=width) rebuild(l/width);
    }
    void set(int i,int x) {
        assert(0<=i && i<n && 1<=x && x<=1000000000);
        a[i]=x; rebuild(i/width);
    }
    std::pair<int,int> exit_from(int i) const {
        assert(0<=i && i<n);
        int steps=0, final=i;
        while (i<n) {
            steps+=count[i]; final=last[i]; i=next[i];
        }
        return {final,steps};
    }
};
```

Листинги продолжают один класс. Полная реализация проверяет положительность прыжков. При нулевых или отрицательных значениях возможны циклы, зависимости перестают идти справа налево, а запрос уже не обязан завершаться. Это изменение постановки, а не редкий случай для небольшой правки цикла.

Каждый переход сводки увеличивает номер блока. Поэтому запрос стоит $O(1 + n/B)$, обновление $O(B)$, построение $O(n)$. Для u обновлений и q запросов балансируем $uB + qn/B$. При $u = 0$

можно предварительно посчитать переход до выхода из всего массива; динамическая структура тогда избыточна.

5.6. Тяжёлые и лёгкие вершины

Корневая декомпозиция не обязательно делит массив на отрезки. В простом неориентированном графе с m рёбрами сумма степеней равна $2m$. Если тяжёлой назвать вершину степени не меньше B, тяжёлых вершин не больше $2m/B$. Лёгких может быть много, зато их локальные списки коротки.

Рассмотрим подсчёт треугольников. Зададим строгий порядок вершин по паре `(степень, номер)` и направим каждое ребро от меньшей пары к большей. Вершина малой степени имеет мало исходящих рёбер. У вершины большой степени исходящие соседи тоже имеют большую степень, а таких вершин мало. При пороге около $\sqrt{2m}$ исходящая степень любой вершины равна $O(\sqrt{m})$.

Для каждой вершины u пометим её исходящих соседей. Затем для каждого ребра $u \to v$ просмотрим исходящих соседей w вершины v; если w помечен, найден треугольник. Наконец снимем метки. В каждом треугольнике есть единственная наименьшая вершина u и единственная средняя v, поэтому он учитывается один раз.

На пометки тратится $O(m)$, на двойные проходы — $O(m\sqrt{m})$, память списков и меток — $O(n + m)$. Строгий порядок по номеру при равенстве степеней важен: без него ориентация не определена однозначно. Петли и кратные рёбра заранее исключены контрактом; иначе «число треугольников» требует другого определения.

Здесь порог служит доказательству оценки, а не обязательному массиву с пометкой heavy. Общая схема полезна шире: дорогое действие выполняется либо над маленьким объектом, либо над небольшим числом больших объектов. Это другой источник квадратного корня, хотя алгебра баланса похожа.

5.7. Перестройка, перенос фрагментов и пределы метода

В той же лекции 25.10.2022 примерно с 27-й минуты появляется перенос фрагмента последовательности через `rotate`. Для $[a, b, c, d, e, f, g]$ и границ $(1, 4, 6)$ перестановка $[1, 4)$ и $[4, 6)$ даёт $[a, e, f, b, c, d, g]$. Буквы и границы этого примера редакторские. Порядок значений здесь не связан с сортировкой.

Блочное представление разрезает блоки, содержащие три границы, переставляет целые блоки и восстанавливает размеры около разрезов. Но цена перестановки самих описателей зависит от контейнера. Вектор указателей сдвигает описатели линейно по числу блоков; связный список быстро перепривязывает известные границы, зато ещё надо найти их по номеру позиции. Дешёвая локальная операция не делает весь запрос константным.

Если после каждой операции оставлять всё больше маленьких блоков, число сводок перестаёт быть $O(n/B)$. Нужен инвариант минимального размера либо периодическая полная перестройка. Например, если одна операция создаёт лишь постоянное число дополнительных кусков, после B операций их станет на $O(B)$ больше. Перестройка за $O(n)$ раз в B операций добавляет $O(n/B)$ амортизированной стоимости; её надо включить в баланс.

В расширенной практике задача «Архив» требует именно управления порядком последовательности. Она не сводится к нашему классу `BlockSum`. Сравните блочный подход с неявным деревом поиска: в дереве позиция задаётся размерами поддеревьев, а разрез и склейка действуют на всю последовательность. Сначала сравнивают интерфейсы и гарантии, затем выбирают структуру.

5.8. Другой контракт: прибавление диапазону и чтение точки

Сменим роли дорогого и дешёвого действия: теперь запрос читает один элемент, а обновление прибавляет число ко всему диапазону. Храним базовые значения элементов и одну отложенную прибавку на блок. Инвариант: логическое значение позиции i равно $base[i] + lazy[i/B]$.

При полном покрытии блока увеличиваем только его метку. При частичном покрытии увеличиваем базовые значения затронутых элементов, не трогая метку. В отличие от дерева с присваиваниями и прибавлениями, здесь все отложенные действия — сложения; они коммутируют. Поэтому можно изменить базовую часть и оставить прежнюю общую прибавку: итог будет тем же.

Для $[2, 7, 1, 4, 3, 6]$ и $B = 3$ выполним `add(0,3,5)`. Базовый массив не изменится, метки будут $[5, 0]$, а логический массив станет $[7, 12, 6, 4, 3, 6]$. Затем `add(1,5,2)` изменит базы позиций 1–4: получим базы $[2, 9, 3, 6, 5, 6]$ с теми же метками. Логический результат — $[7, 14, 8, 6, 5, 6]$.

```
class BlockAddPoint {
    int n, width;
    std::vector<long long> base, lazy;
public:
    BlockAddPoint(std::vector<long long> a,int b)
        : n(int(a.size())), width(b), base(std::move(a)) {
        assert(n<=1000000 && width>0);
        lazy.assign(n/width+(n%width!=0),0);
    }
    void add(int l,int r,long long x) {
        assert(0<=l && l<=r && r<=n);
        while(l<r && l%width) base[l++]+=x;
        while(width<=r-l) {lazy[l/width]+=x; l+=width;}
        while(l<r) base[l++]+=x;
    }
    long long get(int i) const {
        assert(0<=i && i<n); return base[i]+lazy[i/width];
    }
    void set(int i,long long x) {
        assert(0<=i && i<n); base[i]=x-lazy[i/width];
    }
};
```

Точечное присваивание также можно сделать без проталкивания: новое базовое значение равно $x - lazy[block]$. Например, `set(1,0)` в приведённом состоянии записывает в базу -5; чтение возвращает $-5 + 5 = 0$. Записав просто 0, мы получили бы ошибочный ответ 5. Отрицательная база допустима, даже если логические значения задачи неотрицательны.

Прибавление стоит $O(B + n/B)$, чтение и присваивание точки — $O(1)$; построение и память линейны. Все суммы и разности должны помещаться в `long long`. Структура не хранит суммы блоков и потому не даёт быстрых запросов суммы диапазона. Чтобы добавить такой запрос, надо поддерживать ещё один инвариант, как в § 3.

Этот пример объясняет, почему нельзя выбирать алгоритм по словам «диапазонные операции». Прибавление диапазону и чтение точки, присваивание диапазону и сумма диапазона, точечное изменение и статистика диапазона имеют разные удобные представления. Переносите прежде всего контракт, а не знакомый фрагмент кода.

5.9. Углубление: декомпозиция порядка запросов

Иногда изменять данные вообще не нужно, а все запросы известны заранее. Пусть требуется число различных значений на каждом $[l, r)$ статического массива. Если умеем добавлять или удалять один элемент текущего окна, то можно не строить ответ заново, а перемещать две его границы. Частоты значений и число ненулевых частот позволяют менять ответ за $O(1)$ после сжатия координат.

Алгоритм Мо сортирует запросы сначала по номеру блока левой границы, затем по правой границе. В соседних блоках направление сортировки правой границы можно чередовать, чтобы уменьшить возвраты. Отвечаем в новом порядке, а результаты записываем под исходными номерами. В отличие от online-структуры, этот метод использует знание всего набора запросов.

Внутри одной группы левые границы лежат в блоке длины B, поэтому их перемещения дают $O(qB)$ за все запросы. Правая граница в каждой группе идёт монотонно по диапазону длины не больше n; групп $O(n/B)$, значит её перемещения стоят $O(n^2/B)$. Переходы между группами не меняют этот порядок. Вместе с сортировкой получаем $O(q \log(q+1) + qB + n^2/B + n)$.

При q порядка n подходит $B \approx \sqrt{n}$. При другом числе запросов баланс даёт $B \approx n/\sqrt{q}$, с отсечением до диапазона $[1, \max(1, n)]$. Если запросов нет, выполнять сортировку и перемещения незачем. Выбранный порядок не уменьшает цену добавления элемента: если она не постоянна, её надо умножить на число перемещений.

Для массива $[1, 2, 1, 3, 2]$ окно $[0, 3)$ содержит два различных значения. При переходе к $[1, 5)$ удаляется одна единица, но другая остаётся; затем добавляются 3 и 2, итог — три значения. Уменьшать число различных при каждом удалении неверно: оно меняется только при переходе частоты через ноль. Это минимальный контрпример для проверки операций окна.

Базовый алгоритм Мо предполагает статичный массив. Если между запросами есть обновления, одних двух координат окна недостаточно; добавляется измерение времени и другой анализ. Для суммы с точечными обновлениями дерево из § 2 обычно проще, а Мо полезен для статистики с дешёвым локальным изменением и неудобным объединением больших сводок.

5.10. Проверка и самостоятельная работа

Для суммы сравнивайте все ответы с прямым сложением. Включите размеры 0 и 1, длины $B-1$, B, $B+1$, запрос внутри одного блока, пустой запрос и неполный последний блок. Для прыжков эталоном служит буквальное движение по массиву; положительность значений гарантирует не более n шагов. Особенно полезно менять прыжок непосредственно перед границей блока.

1. Почему запрос внутри одного блока нельзя разложить на два независимо просматриваемых края?
2. Для какой сводки точечное присваивание постоянно, а для какой требует перестройки?
3. Выведите размер блока при разных количествах запросов и обновлений.
4. Почему сортированная копия блока не заменяет исходный порядок?
5. Что именно означают `next`, `count` и `last`?
6. Почему изменение прыжка не требует перестраивать предыдущие блоки?
7. Как нарушится доказательство при разрешении прыжка нулевой длины?
8. Докажите оценку числа тяжёлых вершин.
9. Почему ориентированный треугольник учитывается ровно один раз?
10. Как периодическая перестройка удерживает число маленьких кусков под контролем?

Практика. Реализуйте подсчёт значений меньше порога с точечными изменениями. Затем решите модель прыжков и сравните её с наивным симулятором. Для углубления докажите оценки выбранного варианта переноса фрагмента: явно назовите контейнер блоков и стоимость поиска границ. Следующий параграф делит уже не данные на блоки, а пространство перебора на две половины.

§ 6. Эффективный перебор: встреча посередине

Экспоненциальный перебор иногда остаётся лучшей отправной точкой. При $n = 40$ полный просмотр подмножеств содержит около 10^{12} вариантов. Две половины по 20 элементов дают примерно по миллиону состояний. Выигрыш появляется, только если половины можно объединить без перебора всех пар: миллион на миллион вернул бы исходную проблему.

Метод meet-in-the-middle, или встреча посередине, сохраняет результаты одной части поиска и организует быстрый поиск совместимого дополнения. В этом параграфе мы отделим перечисление состояний от их объединения, разберём кратности, доминирование, XOR-пути и большой и малый шаг. Основной язык реализаций — C++17; из предыдущих параграфов нужны двоичный поиск, битовые маски и оценка памяти.

6.1. Контракт объединения двух половин

Разделим индексы на непересекающиеся множества L и R. Каждое подмножество исходных индексов единственным образом представляется парой подмножеств этих половин. Это даёт основу корректности: мы не теряем решения и не создаём лишних, если для пары правильно проверено исходное условие.

Для суммы условие $sum(X) + sum(Y) = T$ превращается в поиск $sum(Y) = T - sum(X)$. Для ограничения веса нужен поиск лучшей ценности при $weight(Y) \le W - weight(X)$. Для путей половины должны встретиться в одной вершине и согласовать значение общего состояния. Сам факт разбиения не говорит, какую сводку надо хранить.

Пусть числа состояний равны P и Q. Сортировка правой части и двоичный поиск для каждого левого состояния дают $O(Q \log Q + P \log Q)$ времени и $O(P + Q)$ памяти при хранении обоих списков. Если левую часть генерировать потоком, её список можно не хранить. Хеш-таблица часто даёт ожидаемое $O(P + Q)$, но гарантия зависит от модели хеширования; сортировка даёт детерминированную границу.

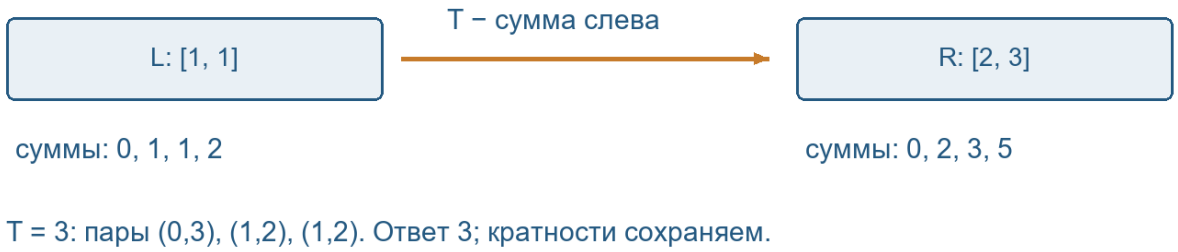


Рисунок 9: Состояние левой половины ищет совместимое дополнение в подготовленной правой половине.

При $|L| = \lfloor n/2 \rfloor$ и $|R| = \lceil n/2 \rceil$ размеры списков близки. Перекос делает большую половину дороже: при 10 и 30 элементах одна сторона содержит уже около миллиарда состояний. Если доступная память ограничивает сохранённую сторону, допускается несимметричное разбиение, но это осознанный обмен памяти на время.

6.2. Пример курса: четыре числа с нулевой суммой

В старом конспекте дан массив $A = (2, 3, 1, 0, -4, -1)$ и разрешено использовать один элемент несколько раз. Например, $3 + 1 + 0 - 4 = 0$ и $0 + 0 + 0 + 0 = 0$. Эта оговорка принципиальна: она определяет допустимые пары и делает повтор индекса корректным.

Перепишем равенство как $a + b = -(c + d)$. Сохраним все n^2 сумм упорядоченных пар, отсортируем их, затем для каждой второй пары найдём противоположную сумму. Время $O(n^2 \log(n + 1))$, память $O(n^2)$. Чтобы восстановить ответ, вместе с суммой сохраним индексы пары. Для задачи существования можно остановиться при первом совпадении.

Если нужны четыре различных индекса, простая проверка наличия суммы уже недостаточна. Найденная пара может использовать тот же индекс, что и текущая. Нельзя объявить задачу решённой, проверив только одну произвольную пару на сумму: другая пара с той же суммой могла бы подойти. Такое ограничение требует дополнительной организации поиска или другого порядка обработки пар.

Для упорядоченных четвёрок кратности пар перемножаются. Подсчёт наборов индексов без учёта порядка — другая задача: нужно учитывать симметрии. Не делите ответ механически на $4!$: повторяющиеся индексы и значения создают разные размеры классов симметрии. Сначала зафиксируйте объект подсчёта.

Этот пример сохраняет постановку и числа письменного материала курса. Он удобен тем, что показывает метод без экспоненты: встреча посередине может уменьшать и степень полиномиального перебора.

6.3. Подсчёт подмножеств с заданной суммой

Считаем подмножества **индексов**, включая пустое, с суммой T. Равные значения на разных позициях дают разные выборы. Разрешены отрицательные числа: нельзя отбрасывать промежуточную сумму только потому, что она уже превышает T.

Для одной половины начинаем со списка `[0]`. Каждый новый элемент x создаёт копию прежних состояний с прибавленным x. Размер старой части запоминаем до добавлений: иначе новые состояния могли бы снова использовать тот же элемент. Суммарное число создаваемых записей равно $1 + 2 + \cdots + 2^k = O(2^k)$, а не $O(k2^k)$, как при отдельном просмотре всех битов каждой маски.

```
inline std::vector<long long> subset_sums(
        const std::vector<long long>& a,int l,int r) {
    assert(0<=l && l<=r && r<=int(a.size()) && r-l<=20);
    std::vector<long long> sums{0};
    for (int i=l; i<r; ++i) {
        auto old=sums.size();
        for (std::size_t j=0; j<old; ++j)
            sums.push_back(sums[j]+a[i]);
    }
    return sums;
}
```

Контракт полной функции: $n \le 40$, $|a_i| \le 10^9$, $|T| \le 10^{12}$. Эти ограничения гарантируют безопасность сумм и `target-x` в `long long`. Ответ не больше 2^{40} и также помещается в тип. Вспомогательный генератор предполагает не больше 20 элементов на половину.

```
inline long long count_subsets(const std::vector<long long>& a,
                               long long target) {
    assert(a.size()<=40);
    int n=int(a.size()), mid=n/2;
    auto left=subset_sums(a,0,mid);
    auto right=subset_sums(a,mid,n);
    std::sort(right.begin(),right.end());
    long long answer=0;
    for (long long x:left) {
        auto range=std::equal_range(right.begin(),right.end(),target-x);
        answer+=range.second-range.first;
    }
    return answer;
}
```

`equal_range` возвращает весь диапазон элементов с нужной суммой. Его длина — число правых подмножеств, дополняющих текущее левое. Замена отсортированного вектора множеством уникальных сумм сохранила бы ответы о существовании, но уничтожила бы кратности.

Разберём $[1, 1, 2, 3]$ и $T = 3$, разделив после второй позиции. Левые суммы: $[0, 1, 1, 2]$, правые: $[0, 2, 3, 5]$. Ноль слева сочетается с тройкой справа; каждая из двух единиц слева — с двойкой справа. Ответ 3. Выбор одного из двух элементов 1 различает два подмножества, хотя их суммы совпадают.

При пустом массиве генераторы возвращают по одному состоянию 0. Поэтому ответ равен 1 для $T = 0$ и 0 иначе. Это корректная обработка пустого подмножества, а не специальная ошибка подсчёта. Для сорока нулей и $T = 0$ ответ 2^{40} — хороший тест типа счётчика.

6.4. Рюкзак: почему недостаточно ближайшего веса

Теперь каждый предмет имеет неотрицательные вес и ценность; вместимость $W \geq 0$. Нужно максимизировать суммарную ценность при весе не больше W, каждый предмет используется не более одного раза. Обычное DP по вместимости неудобно, если веса порядка 10^9, но предметов не больше сорока.

Пример из видео 11.10.2022, 20:07–21:59. Лектор рассматривает размещение виртуальных машин на сервере: у заявок есть требования по оперативной памяти и CPU, а у сервера — отдельные пределы каждого ресурса. Приоритет заявки задаёт ценность. Это многомерный рюкзак, и его нельзя незаметно заменить одним ограничением на сумму весов. Наш код ниже решает одномерную учебную модель. Для двух ресурсов состояние имеет вид «память, CPU, ценность»; правило доминирования сравнивает оба расхода, а один двоичный поиск по весу уже не выбирает лучшее допустимое дополнение.

Состояние половины — пара (w, v). Для левого состояния нужно найти максимальное v_R среди правых с $w_R \leq W - w_L$. Простого двоичного поиска по весу недостаточно: наиболее тяжёлое допустимое состояние не обязано быть самым ценным.

Состояние (w_2, v_2) можно удалить, если есть (w_1, v_1) с $w_1 \leq w_2$ и $v_1 \geq v_2$: первая пара не тяжелее и не хуже по ценности. Для любого допустимого дополнения она остаётся допустимой и даёт не меньший ответ. Это доказательство доминирования относится к оптимизации значения; если требуется число оптимальных подмножеств, удаление состояния может потерять кратность.

Отсортируем пары по возрастанию веса, а при равном весе — по убыванию ценности. Оставим только строгие рекорды ценности. Тогда веса и ценности сохранённого фронта строго возрастают. Последнее состояние с допустимым весом одновременно имеет максимальную ценность среди допустимых.

Например, пары $(0,0),(2,4),(3,3),(3,5),(5,5),(6,9)$ после сортировки и фильтрации дают $(0,0),(2,4),(3,5),(6,9)$. Пара $(3,3)$ хуже $(2,4)$, а $(5,5)$ хуже $(3,5)$. При остатке вместимости 5 надо выбрать $(3,5)$, а не искать ровно вес 5.

6.5. Реализация рюкзака и её доказательство

Генератор может сразу не добавлять состояние тяжелее W, поскольку все дальнейшие веса неотрицательны. Для задачи с отрицательными весами это отсечение было бы неверным. Нулевая масса допустима; старый размер списка всё равно гарантирует однократность использования предмета.

```
struct Item { long long weight,value; };
using State=std::pair<long long,long long>; // weight, value
inline std::vector<State> enumerate_states(const std::vector<Item>& a,
                                           int l,int r,long long capacity) {
    std::vector<State> states{{0,0}};
    for (int i=l; i<r; ++i) {
        auto old=states.size();
        for (std::size_t j=0; j<old; ++j) {
            auto [w,v]=states[j];
            if (a[i].weight<=capacity-w)
                states.push_back({w+a[i].weight,v+a[i].value});
        }
    }
    return states;
}
```

В полной реализации $n \le 40$, веса и ценности лежат от 0 до 10^9, а $W \le 4\cdot 10^{10}$. Промежуточные суммы представимы в `long long`. Утверждения `assert` проверяют только часть предусловий и не заменяют ограничения входа.

```
inline long long best_knapsack(const std::vector<Item>& a,
                               long long capacity) {
    assert(a.size()<=40 && capacity>=0);
    for (auto item:a) assert(item.weight>=0 && item.value>=0);
    int mid=int(a.size())/2;
    auto right=enumerate_states(a,mid,int(a.size()),capacity);
    std::sort(right.begin(),right.end(),[](State x,State y) {
        return x.first!=y.first ? x.first<y.first : x.second>y.second;
    });
    std::vector<State> frontier;
    long long best=-1;
    for (auto state:right) if (state.second>best) {
        frontier.push_back(state); best=state.second;
    }
    auto left=enumerate_states(a,0,mid,capacity);
    long long answer=0;
    for (auto [w,v]:left) {
        long long room=capacity-w;
        auto it=std::upper_bound(frontier.begin(),frontier.end(),room,
            [](long long limit,State s){return limit<s.first;});
        assert(it!=frontier.begin());
        --it; answer=std::max(answer,v+it->second);
    }
    return answer;
}
```

Пустое подмножество гарантирует непустой список допустимых правых состояний. После фильтрации может остаться более ценное состояние нулевого веса; это тоже корректно. Поэтому при неотрицательном остатке вместимости `upper_bound` всегда имеет хотя бы одного предшественника.

Корректность. Любое допустимое решение распадается на перечисленные половины. Если его правая часть удалена, её доминирует сохранённая часть, которая совместима с той же левой и не уменьшает ценность. Для каждого левого состояния двоичный поиск находит лучший допустимый сохранённый вариант справа. Максимум по всем левым состояниям поэтому не меньше оптимума и сам является допустимым решением.

Время $O(n2^{\lceil n/2 \rceil})$ с учётом сортировки и поиска, память $O(2^{\lceil n/2 \rceil})$. Хранение масок для восстановления увеличивает константу памяти, но не порядок оценки. В этой реализации возвращается только значение оптимума; нельзя обещать восстановление состава предметов по одному числу.

6.6. XOR-пути и общий элемент двух половин

Пусть в прямоугольной таблице разрешены движения вправо и вниз. Нужно посчитать пути из левого верхнего угла в правый нижний, у которых XOR всех посещённых значений равен K. Каждый путь имеет $L = h + w - 2$ шагов. Выберем диагональ $i + j = \lfloor L/2 \rfloor$ и перечислим префиксы от старта и суффиксы от финиша до этой диагонали.

Для каждой клетки встречи храните частоты XOR префиксов. Смешивать состояния разных клеток нельзя: одинаковое XOR не означает, что части соединяются в один путь. Обратный перебор использует движения вверх и влево. Каждая полная траектория пересекает диагональ ровно один раз, поэтому разложение единственно.

Если обе половины включают значение клетки встречи a, то XOR целого пути равен $P \oplus R \oplus a$: в $P \oplus R$ общий элемент исчезает, и его надо вернуть один раз. Для данного R ищем частоту $P = K \oplus R \oplus a$. Можно иначе определить одну из половин без общей клетки, но соглашение должно быть единым в генерации и объединении.

Для таблицы с первой строкой $[1, 2, 3]$ и второй $[4, 5, 6]$ существуют три пути. Их XOR равны 6, 0 и 6. При $K = 6$ ответ 2. Через клетку со значением 2 префикс имеет $P = 1 \oplus 2 = 3$. Суффикс через верхний правый угол имеет $R = 2 \oplus 3 \oplus 6 = 7$; получаем $3 \oplus 7 \oplus 2 = 6$. Ошибочная формула без общей клетки дала бы 4.

Число частичных траекторий ограничено $O(2^{\lceil L/2 \rceil})$, хотя границы таблицы часто уменьшают его. При хешировании объединение имеет ожидаемую линейную стоимость по числу состояний; при сортировке частот добавляется логарифм. Счётчик путей должен вмещать число полных траекторий: заранее оцените биномиальный коэффициент или используйте заданный модуль. Это тематический разбор к задаче «Xor-пути», а не утверждение о прохождении серверных тестов конкретной реализации.

6.7. Углубление: большой и малый шаг

В мультипликативной группе по простому модулю p даны ненулевые g и h. Требуется найти $x \in [0, p-2]$, для которого $g^x = h \pmod{p}$, либо установить, что решения нет. Основание не обязано порождать всю группу; отсутствие решения возможно.

Положим $m = \lceil \sqrt{p-1} \rceil$ и запишем $x = im + j$, где $0 \le j < m$. Тогда $g^j = h(g^{-m})^i$. Сохраним малые шаги g^j, а затем последовательно вычислим правую часть для $i = 0, \dots, m-1$. Совпадение даёт кандидата $im + j$, который надо проверить на границу $p - 2$. При повторяющихся малых шагах для поиска наименьшего решения сохраняем минимальный j и выбираем минимальный допустимый кандидат.

Для $p = 23$, $g = 5$, $h = 8$ получаем $m = 5$. Малые шаги: $1, 5, 2, 10, 4$. Значение $5^5 = 20 \pmod{23}$ имеет обратный элемент 15. Первый большой шаг даёт $8 \cdot 15 = 5 \pmod{23}$, что совпадает с малым шагом $j = 1$. Получаем $x = 5 + 1 = 6$; действительно, $5^6 = 8 \pmod{23}$.

Таблица содержит $O(\sqrt{p})$ элементов. Сортировка и двоичный поиск дают $O(\sqrt{p} \log p)$ времени, хеширование — ожидаемое $O(\sqrt{p})$ после вычисления обратного множителя. Все умножения выполняются по модулю с защитой от переполнения. Для составного модуля и необратимого g эта формула без изменений не подходит: обратного элемента может не существовать.

Общий мотив тот же, что у сумм подмножеств: дорогой параметр разбивается на два меньших, одна сторона сохраняется, другая ищет совпадение. Однако доказательство использует свойства группы, а не разбиение множества предметов.

6.8. Восстановление решения и бюджет памяти

Для восстановления подмножества мало знать, что совместимая сумма существует. Сохраняйте вместе с суммой маску индексов своей половины. При расширении состояния новым элементом устанавливайте его локальный бит. Совпавшая пара масок однозначно восстанавливает исходные индексы: для правой половины к локальному номеру добавляется размер левой.

Для рюкзака маска должна перемещаться вместе с парой веса и ценности при сортировке. При удалении доминируемого состояния сохраняется хотя бы один не худший представитель. После двоичного поиска запоминаем обе маски, если найден лучший ответ. Если требуется определённый выбор среди равных оптимумов, например лексикографически минимальный список индексов, правило сравнения состояний надо дополнить; произвольный представитель может не выполнить такое требование.

Для подсчёта и для восстановления нужны разные объёмы информации. Все одинаковые суммы можно сжать в пару «сумма, частота», если нужен только счётчик. Для одного восстановленного ответа достаточно одной маски на сумму. Для перечисления всех решений могут понадобиться все маски или повторная генерация; сам вывод способен занимать экспоненциальное время.

Посчитаем память при половине из 20 элементов. Число состояний — $2^{20} = 1\,048\,576$. Список `long long` занимает 8 МиБ полезных данных. Две такие половины — 16 МиБ. Пара 64-битных веса и ценности обычно занимает 16 байт на состояние, то есть 16 МиБ на одну половину; структура с дополнительной маской может из-за выравнивания занимать 24 или 32 байта. Проверяйте `sizeof` конкретного типа, а не складывайте размеры полей на глаз.

Контейнер дополнительно хранит ёмкость. При геометрическом росте временно существуют старый и новый буферы; хеш-таблица имеет корзины и служебные данные. Фронт рюкзака может в худшем случае содержать все состояния: нельзя планировать память, надеясь, что доминирование всегда удалит почти всё. Потоковая генерация левой части экономит её список, но не уменьшает размер подготовленной правой стороны.

Отличайте ограничение числа предметов от величины весов. Для ста предметов meet-in-the-middle с двумя половинами по 50 непрактичен даже при маленьких весах. Тогда DP по сумме или битсет из § 4 может быть лучше. Для сорока предметов с огромными весами ситуация обратная: перебор половин зависит от числа предметов, а не от численного значения вместимости.

Контрольный расчёт. Сравните $n = 30$, $n = 40$ и $n = 50$. При увеличении n на десять каждая половина получает пять дополнительных элементов, поэтому число состояний увеличивается в 32 раза. Уменьшение константы вдвое не компенсирует такой рост. Именно поэтому перед реализацией оценивают не только асимптотику, но и конкретное число записей.

6.9. Двусторонний поиск и проверка границ метода

Поиск в графе тоже можно вести с двух концов. Если известна верхняя граница D длины искомого пути, полные BFS до глубин $\lfloor D/2 \rfloor$ от старта и $\lceil D/2 \rceil$ от финиша найдут общую вершину для любого пути длины не больше D. В ориентированном графе обратный поиск идёт по обратным рёбрам. Минимум суммы расстояний в пересечении даёт кратчайший путь в заданном ограничении.

При ограниченном ветвлении $b > 1$ это может сократить дерево поиска от порядка b^D до порядка $b^{\lceil D/2 \rceil}$. Оценка относится к числу достижимых состояний в модели; граф с большим пересечением путей и граф с огромной входящей степенью ведут себя иначе. Остановка при первом случайно найденном пересечении не заменяет доказательства для конкретной очередности расширения фронтов.

Для проверки сумм сравните результат с полным перебором при $n \leq 18$. Для рюкзака переберите все подмножества независимо и проверьте нулевые веса, одинаковые состояния, равные ценности при разных весах и предмет тяжелее вместимости. Для XOR-путей перечислите все пути на маленьких таблицах, включая таблицу 1×1.

1. Почему простое разбиение пополам не помогает, если затем перебираются все пары состояний?
2. Какое свойство обеспечивает единственность разложения подмножества?
3. Почему нельзя удалять одинаковые суммы при подсчёте?
4. Что изменится в примере четырёх чисел при требовании различных индексов?
5. При каких предпосылках можно отсекать слишком тяжёлые состояния?
6. Докажите безопасность удаления доминируемой пары для максимума ценности.
7. Почему для подсчёта оптимальных решений того же доказательства недостаточно?
8. Откуда в формуле XOR появляется значение клетки встречи?
9. Где в алгоритме большого и малого шага используется обратимость?
10. Почему обратный BFS в ориентированном графе идёт по обратным рёбрам?

Практика. Дополните рюкзак восстановлением индексов, сохраняя маски половин. Для XOR-путей явно выберите соглашение об общей клетке и напишите тест 1×1. Сравните память списка сумм, списка пар и списка пар с масками до запуска больших случаев. В § 7 мы будем сохранять не состояния перебора, а доступные версии одной структуры данных.

§ 7. Персистентность: версии, стек и очередь

Обычная структура изменяет текущее состояние. Персистентная сохраняет доступ к старым состояниям после изменений. Можно прочитать прежнюю версию, продолжить вычисление от неё и одновременно оставить доступной другую ветвь истории. Для этого вовсе не обязательно копировать все данные после каждого действия.

В этом параграфе мы введём явный интерфейс версий, реализуем полностью персистентный стек и разберём очередь, для которой корректность сохраняется, а привычная амортизированная оценка — нет. Затем сравним копирование пути, толстые узлы и откат. Нужны связные списки, индексы узлов и доказательство очереди на двух стеках из § 1.

7.1. Что означает «старая версия доступна»

Пусть версия v_0 содержит последовательность $[5, 2]$. Из неё создадим v_1 добавлением 7 и независимо v_2 удалением последнего элемента. Получаются $[5, 2, 7]$ и $[5]$, но v_0 по-прежнему содержит $[5, 2]$. Команда изменения должна назвать исходную версию; команда чтения — нужную версию ответа.

Различают частичную персистентность: читать можно любую версию, изменять только последнюю; полную: изменять можно любую существующую версию; и конфлюэнтную: одна операция может объединять несколько версий. В последнем случае история зависимостей становится ориентированным ациклическим графом, а не деревом с единственным родителем каждой новой версии.

Копия номера версии не обязательно создаёт новые узлы. Две логические версии могут ссылаться на один корень и иметь одинаковое состояние. И наоборот, два разных корня могут представлять одинаковые последовательности. Разделяйте идентификатор истории, корень представления и логическое содержимое.

В этой главе все старые корни остаются доступными. Операции возвращают новый корень, вызывающий код хранит его под нужным номером. Мы не добавляем операцию удаления исторической версии и не обещаем автоматического освобождения переставших быть нужными узлов.

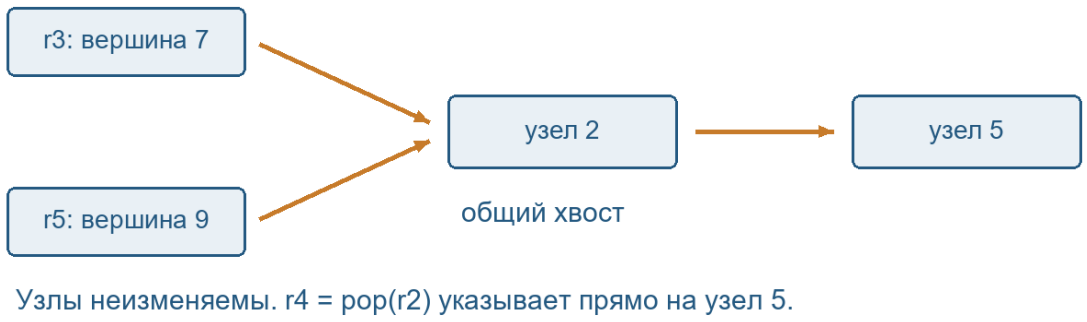


Рисунок 10: История операций ветвится; неизменяемые узлы разных версий могут иметь общий хвост.

Связь с видео. Запись 30.11.2022 посвящена сохранению версий; около 14:56 на кадре показано ветвление, а фрагмент 44:00–46:00 разбирает стек и расход памяти. Ниже формализуем интерфейс на C++17. Числа 5, 2 и 7 и имена версий выбраны для учебника.

7.2. Неизменяемый список как представление стека

Стек хранится односвязным списком от вершины к основанию. Узел содержит значение и ссылку на следующий узел. Корень — ссылка на вершину, нулевой корень обозначает пустой стек. После создания поля узла не изменяются.

Для `push(r,x)` создаём один узел `(x,r)` и возвращаем его номер. Старый корень остаётся прежним: он не указывает на новый узел и не видит добавление. Для `pop(r)` возвращаем ссылку `next` узла r. Узлы не удаляются, поэтому старая версия продолжает читать прежнюю вершину.

Проследим историю. $r_0 = 0$; $r_1 = push(r_0, 5)$; $r_2 = push(r_1, 2)$; $r_3 = push(r_2, 7)$; $r_4 = pop(r_2)$; $r_5 = push(r_2, 9)$. Корни r_3 и r_5 ведут к разным вершинам 7 и 9, но дальше используют одни и те же узлы 2 и 5. Корень r_4 совпадает с r_1.

Версия	Последовательность от основания к вершине	Новых узлов
r_1	5	1
r_2	5, 2	1
r_3	5, 2, 7	1
r_4	5	0
r_5	5, 2, 9	1

Во всей истории создано четыре узла, хотя суммарная длина перечисленных непустых версий равна десяти. Экономия возникает из разделения общего хвоста. Узел не знает, сколько корней до него доходят: для чтения это не нужно.

Инвариант. Каждый ненулевой корень задаёт конечную цепочку неизменяемых узлов с нулём на конце. Новый узел ссылается только на ранее существовавший корень, поэтому цикл не возникает. Изменение возвращает новую цепочку или её хвост и не меняет цепочки старых корней. Индукция по числу созданных узлов доказывает корректность всех ветвей одновременно.

7.3. Реализация стека и время жизни узлов

Пул хранит узлы по целочисленным номерам. Это освобождает внешний интерфейс от сырых указателей. Нулевой элемент пула — служебный; действительные корни должны принадлежать именно этому экземпляру структуры. В примере используется `std::deque`, добавление в конец которого не копирует весь прежний массив узлов.

```
class PersistentStack {
public:
    using Root=std::size_t;
private:
    struct Node { long long value; Root next; std::size_t length; };
    std::deque<Node> pool{{0,0,0}}; // index 0 is the empty stack
public:
    bool empty(Root r) const { assert(r<pool.size()); return r==0; }
    std::size_t size(Root r) const {
        assert(r<pool.size()); return pool[r].length;
    }
```

В узле хранится длина хвоста вместе с вершиной. Поэтому `size` читает готовое число, а при добавлении достаточно увеличить длину исходной версии на единицу. Следующий листинг продолжает тот же класс; старые поля узлов не переписываются.

```
    Root push(Root r,long long value) {
        assert(r<pool.size());
        pool.push_back({value,r,pool[r].length+1});
        return pool.size()-1;
    }
    long long top(Root r) const {
        if (empty(r)) throw std::out_of_range("empty persistent stack");
        return pool[r].value;
    }
    Root pop(Root r) const {
        if (empty(r)) throw std::out_of_range("empty persistent stack");
        return pool[r].next;
    }
    std::size_t allocated() const { return pool.size()-1; }
};
```

Заголовок `code/07-persistence.hpp` содержит все необходимые включения. Исключение при чтении или удалении из пустого стека является частью контракта. `assert` контролирует принадлежность номера диапазону пула в отладочной сборке; он не отличит случайный допустимый номер из другого пула, поэтому такие номера запрещены предусловием.

`push` создаёт ровно один узел, `pop` не создаёт ни одного, `top`, `size` и `empty` читают постоянное число полей. Каждая операция имеет постоянное число действий на уровне узлов. Память после p добавлений — $O(p)$ узлов, плюс $O(v)$ для внешнего массива корней, если сохранено v именованных версий. Память нельзя оценивать только длиной текущего стека.

Пул живёт столько же, сколько объект `PersistentStack`. После уничтожения объекта все его номера теряют смысл. В нашей модели узлы не освобождаются по отдельности. Это простое и предсказуемое решение для конечной истории; система с долгим временем работы может потребовать сборки мусора или подсчёта ссылок.

Не следует хранить ссылку на элемент `std::vector`, затем добавлять в тот же вектор новые узлы и продолжать обращаться по старой ссылке: перевыделение делает её недействительной. Индексы сохраняют логическую идентичность узлов, но чтение по индексу всё равно должно происходить после потенциального перевыделения. Неизменяемость логических узлов не отменяет правил времени жизни объектов C++.

7.4. Список: цена изменения в середине

Стек особенно удобен, потому что его операции затрагивают голову списка. Если нужно изменить значение на расстоянии k от головы, старый узел переписывать нельзя: он доступен из прежних корней. Создадим новый узел с изменённым значением, а затем копии всех k предшествующих узлов с новыми ссылками. Неизменённый суффикс после позиции изменения разделяется со старой версией.

Таким образом, изменение по индексу требует $O(k+1)$ времени и новых узлов. Вставка и удаление в известной позиции также копируют путь от корня до места изменения. Наличие указателя на узел не позволяет бесплатно изменить входящую ссылку его предшественника во всех версиях.

Объединение двух списков через конкатенацию также зависит от представления. Чтобы приписать список B к хвосту A, нужно скопировать узлы A, изменив связь последнего узла, а затем разделить B. Цена $O(|A|)$, даже если оба входных корня уже существуют. Это пример операции с двумя версиями, но он не даёт универсальной константной конфлюэнтности.

Полное копирование списка перед каждым изменением было бы корректно, но стоило бы $O(n)$ независимо от позиции. Копирование пути сохраняет только изменённую часть достижимости. Для

сбалансированного дерева этот путь короток; для длинного списка — нет. Персистентность не исправляет плохую высоту исходного представления.

7.5. Очередь на двух персистентных стеках

Представим очередь парой стеков: `front` хранит начало очереди на вершине, `back` принимает новые элементы. Логическая последовательность — элементы `front` от вершины к основанию, затем элементы `back` от основания к вершине. Добавление создаёт новую вершину `back` и новую пару корней.

Если `front` пуст, перед чтением или удалением надо перевернуть весь `back` в новый стек. Старый входной стек сохраняется: операция создаёт новые узлы выходного стека, не переписывая старых. Версия очереди передаётся по значению, поэтому изменения локальной пары не меняют ранее сохранённые версии.

```
class PersistentQueue {
public:
    struct Version { PersistentStack::Root front=0, back=0; };
private:
    PersistentStack stack;
    Version normalize(Version v) {
        if (!stack.empty(v.front)) return v;
        while (!stack.empty(v.back)) {
            v.front=stack.push(v.front,stack.top(v.back));
            v.back=stack.pop(v.back);
        }
        return v;
    }
```

Публичные методы продолжают тот же класс. `normalize` — внутренний способ получить эквивалентное представление с непустым передним стеком, если очередь не пуста.

```
public:
    bool empty(Version v) const {
        return stack.empty(v.front) && stack.empty(v.back);
    }
    Version push(Version v,long long x) {
        v.back=stack.push(v.back,x); return v;
    }
    long long front(Version v) {
        v=normalize(v); return stack.top(v.front);
    }
    Version pop(Version v) {
        v=normalize(v); v.front=stack.pop(v.front); return v;
    }
    std::size_t allocated() const { return stack.allocated(); }
};
```

Код корректен как персистентная очередь: каждая возвращённая версия имеет нужную FIFO-последовательность, старые версии сохраняются. Но это **не** очередь с постоянной амортизированной стоимостью по произвольной истории версий. Даже чтение может создавать узлы, поскольку нормализованная пара из `front(v)` не сохраняется наружу.

На линейной цепочке добавлений и удалений, где результат каждого удаления используется дальше, переносы можно оплачивать как в § 1. Повторные чтения из одной ненормализованной версии в данном коде, однако, каждый раз выполняют перенос заново. Для них и для ветвящейся истории нужно отдельное рассуждение, которого обычная оценка двух стеков не предоставляет.

7.6. Почему потенциал нельзя потратить в каждой ветви

Создадим версию v с n элементами в `back` и пустым `front`. Теперь m раз выполним `pop(v)`, каждый раз начиная с одного и того же корня, и сохраним результаты как разные дочерние версии. Каждый вызов переворачивает n элементов, поэтому суммарная работа и число новых узлов равны $\Theta(mn)$.

В обычной очереди потенциал пропорционален размеру входного стека. Один дорогой перенос уменьшает его, и повторить этот перенос из прежнего состояния уже нельзя. При персистентности прежнее состояние доступно снова: одинаковый локальный запас пытаются использовать для оплаты работы во многих ветвях.

В формуле потенциалов из § 1 внутренние состояния сокращались вдоль одной цепочки. В дереве версий одна вершина может иметь несколько исходящих рёбер. Если независимо просуммировать разности потенциалов по рёбрам, потенциал ветвления будет вычтен несколько раз. Телескопическое сокращение больше не имеет прежнего вида.

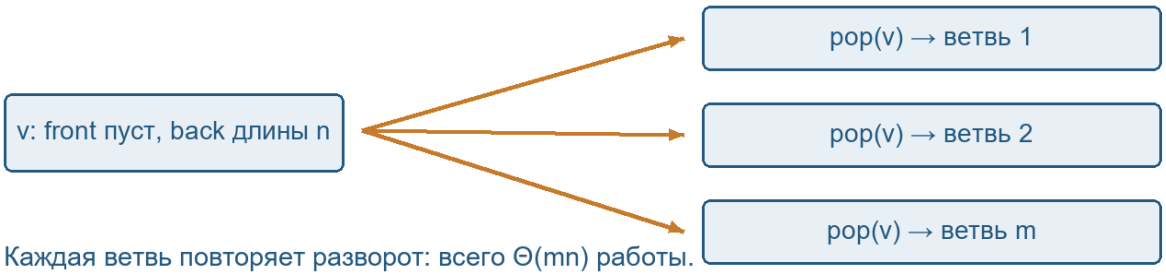


Рисунок 11: Несколько операций от одной версии повторяют один и тот же дорогой разворот.

Полезный тест производительности здесь детерминированный: построить описанную версию и многократно разветвиться именно от неё. Случайная история может почти никогда не выбирать такую вершину. В проверочном коде считаем новые узлы, а не только время; это непосредственно наблюдаемая стоимость, которую объясняет доказательство.

Мемоизация результата разворота может устранить повторение для одного и того же объекта, если кэш действительно разделяется между ветвями. Но тогда представление включает кэш, его заполнение, время жизни и гарантии однократного вычисления. В многопоточной системе добавляются вопросы синхронизации. Слова «лениво вычислим» без этой модели не являются доказательством.

Другой путь — очередь с постепенной перестройкой: дорогой разворот распределяется между будущими операциями, а состояние хранит расписание оставшейся работы. В конструкциях с несколькими стеками нужно доказать, что перестройка начинается достаточно рано, операция выполняет ограниченное число шагов, а начало очереди доступно до её завершения. Название «пять стеков» или «шесть стеков» само по себе не задаёт такой алгоритм. Здесь это направление углубления; приведённый код намеренно показывает простую корректную очередь и точную границу её эффективности.

7.7. Копирование пути и толстые узлы

Для дерева изменение одного листа затрагивает путь от корня. Создаём новый лист и новые копии его предков; неизменённые соседние поддеревья оставляем общими. Каждый скопированный узел указывает на правильные детей новой версии. Старый корень по-прежнему достигает только старых узлов или неизменённых общих поддеревьев.

Если высота равна h, число новых узлов и время изменения обычно $O(h)$ при постоянной работе в узле. В следующем параграфе дерево отрезков даст $h = O(\log n)$ без случайных предположений. Для обычного BST без балансировки высота может быть n; механическое копирование пути не даёт логарифмической гарантии.

Альтернативный подход — толстый узел: вместо перезаписи поля хранить историю его значений с номерами версий. При частичной персистентности и линейной истории можно искать последнее изменение не позже времени запроса. Однако поле с k изменениями требует поиска в его истории, а не бесплатного доступа к «старому значению».

Для полной персистентности линейного времени недостаточно. Пусть v_0 имеет поля $x = 0, y = 0$, из него создана v_1 с $x = 1$, а затем независимо v_2 с $y = 2$. Последнее хронологическое изменение x перед созданием v_2 произошло в v_1, но в v_2 значение x всё ещё равно 0: v_1 не является её предком. Нужно учитывать отношение предок–потомок, а не только сравнение номеров.

Общие преобразования структур в персистентные требуют дополнительных ограничений на число полей, входящие ссылки, способы копирования и поиск подходящих модификаций. Не следует переносить оценки простого стека на произвольный граф указателей. В этой главе доказана конкретная неизменяемая реализация, а не универсальный преобразователь.

7.8. Персистентность, снимки и откат

Полное копирование массива при каждом изменении создаёт снимки за $O(n)$ времени и памяти на версию. Это простой эталон корректности и иногда разумное решение для маленьких данных. Разделение узлов уменьшает стоимость локальных изменений, но требует дисциплины неизменяемости.

Откат хранит журнал изменений и умеет отменять последние действия. Если дерево истории известно заранее, можно обойти его в глубину: применить изменение при входе в вершину и откатить при выходе. В каждый момент существует одно текущее состояние. Это удобный offline-приём, но он не даёт произвольного одновременного доступа ко всем версиям через независимые корни.

Снимок диска, сохранение объекта в файл и персистентность в смысле структур данных — разные понятия. Наши версии существуют в оперативной памяти и исчезают вместе с процессом. Чтобы пережить сбой, нужны отдельные правила записи и восстановления; из неизменяемости узлов это автоматически не следует.

При выборе представления выпишите, какие версии будут читаться, разрешено ли ветвление, сколько операций ожидается и можно ли забывать историю. Если нужен только возврат на один шаг, журнал может быть проще. Если нужны независимые ветви и быстрые локальные изменения, корни и разделение подструктур дают естественный интерфейс.

7.9. Проверка ветвящейся истории и задания

Независимый эталон для стека — отдельный `std::vector` значений на каждую версию. Генератор выбирает любую старую версию, копирует эталон, выполняет изменение и сравнивает новую вершину, размер и содержимое. После создания потомка повторно проверяйте предка: иначе ошибка с изменением общего узла может долго оставаться незамеченной.

Для очереди используйте отдельную `std::deque` на версию. Проверяйте ветвление из пустой очереди, несколько удалений от одной версии, смешивание старых и новых ветвей и одинаковые значения. Для эффективности отдельно создайте контрпример из пункта 7.6; функциональный тест FIFO не обнаружит лишние копирования.

1. Чем полная персистентность отличается от частичной?
2. Почему копия корня не равна полному копированию структуры?
3. Докажите, что новый узел стека не создаёт цикл.
4. Сколько новых узлов создают `push` и `pop`?
5. Почему память зависит от всей истории, а не от длины последней версии?
6. Сколько узлов нужно скопировать для изменения середины односвязного списка?
7. Почему очередь из листинга корректна, но не имеет заявленной для обычной очереди амортизации?
8. В каком месте ломается телескопическое суммирование по дереву версий?
9. Почему хронологический номер не заменяет отношение предка при чтении толстого узла?
10. Чем обход дерева истории с откатом отличается от доступных одновременно персистентных корней?

Практика. Добавьте стеку минимум, сохраняя его в каждом новом узле, и проверьте ветвление из версии с повторяющимися минимумами. Затем реализуйте копирование пути для точечного изменения в дереве из § 2. До написания кода укажите, какие узлы должны быть новыми, а какие разрешено разделить: это основа следующего параграфа о персистентном дереве отрезков.

(c) Aleksey Tolstikov

§ 8. Персистентное дерево отрезков

В § 2 дерево отрезков хранило единственное состояние массива. В § 7 мы научились сохранять старые корни и разделять неизменяемые узлы. Теперь соединим эти идеи. Одно точечное изменение затрагивает только путь от корня к листу, поэтому сохранение версии не требует копирования всего массива.

У этого приёма два разных применения. Первое — изменяемый массив с ветвящейся историей. Второе — статический массив, для каждого префикса которого строится частотное дерево по значениям. Во втором случае разность двух корней описывает диапазон позиций. Смешивать эти модели нельзя: одинаковый тип узла ещё не означает одинаковый смысл версии.

8.1. Контракт: какой массив представляет корень

Пусть длина $n > 0$ фиксирована. Интерфейс `set(root,pos,x)` возвращает новый корень с присвоенным значением в позиции `pos`; `sum(root,l,r)` возвращает сумму на полуинтервале $[l, r)$. Допускаются отрицательные значения и пустой запрос. Индексы принадлежат $[0, n)$, границы запроса удовлетворяют $0 \le l \le r \le n$. Все промежуточные суммы должны помещаться в `long long`.

Корень относится к конкретному экземпляру структуры и к одной фиксированной длине. Нельзя передавать корень другого пула, даже если его числовой номер существует в первом. Класс хранит узлы, а вызывающий код — нужные корни: эта граница ответственности позволяет оставить только интересующие версии.

Для начала используем нулевой массив. Корень 0 означает, что все значения равны нулю; его дети тоже имеют номер 0. Один служебный узел представляет любой полностью нулевой диапазон, потому что границы передаются отдельно. Это не полноценное дерево из $2n - 1$ узлов, а неявное представление. Оно особенно удобно для частот.

Редакторский пример. Для $n = 4$ создадим $r_1 = set(0, 1, 7)$, затем $r_2 = set(r_1, 3, 5)$ и независимо $r_3 = set(r_1, 1, -2)$. Получаются массивы $[0, 7, 0, 0]$, $[0, 7, 0, 5]$ и $[0, -2, 0, 0]$. Запрос $[1, 4)$ возвращает 7, 12 и −2. После создания r_3 корень r_2 по-прежнему возвращает 12.

Корень	Родитель	Изменение	Сумма всего массива
0	—	нулевой массив	0
r_1	0	позиция 1 получает 7	7
r_2	r_1	позиция 3 получает 5	12
r_3	r_1	позиция 1 получает −2	−2

Это пример полной персистентности: новая версия может продолжать любой старый корень. Хронологически последний созданный корень не обязан быть родителем следующего.

8.2. Копирование пути и два инварианта

Узел хранит сумму своего диапазона и номера детей. При присваивании сначала читаем старый узел в локальную переменную. Если это лист, заменяем значение только в локальной копии. Иначе рекурсивно получаем нового ребёнка со стороны изменяемой позиции, второго ребёнка оставляем прежним и пересчитываем сумму. Затем сохраняем новый узел в пуле.

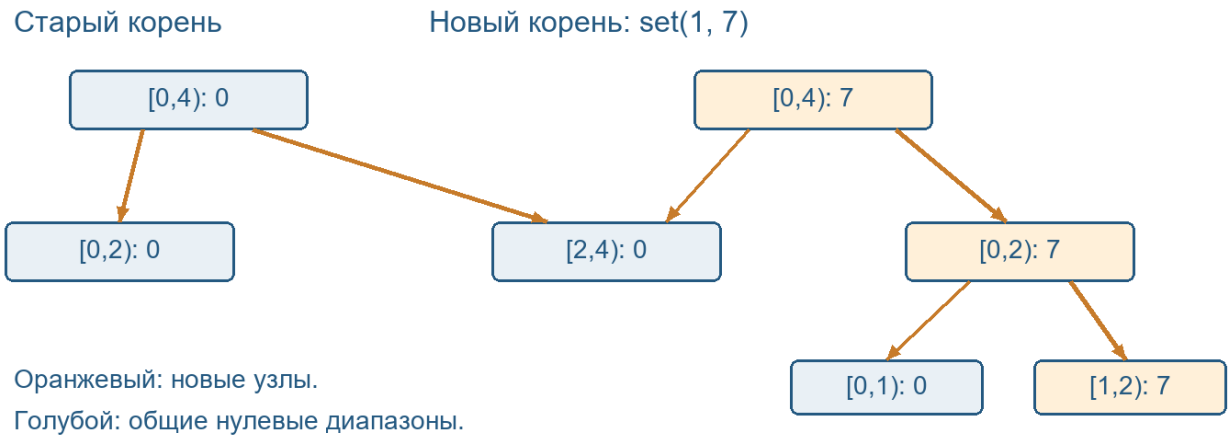


Рисунок 12: При изменении позиции 1 копируются три узла; правое поддерево используется обеими версиями.

Первый инвариант — обычный: сумма внутреннего узла равна сумме детей, а лист хранит значение соответствующей позиции. Второй — персистентный: после публикации корня ни один достижимый из него узел не изменяется. Недостаточно правильно пересчитать новый корень: нужно ещё доказать сохранность всех старых.

Доказательство идёт по длине диапазона. В листе новый узел содержит нужное значение, старый не тронут. Во внутреннем узле рекурсивный вызов по предположению индукции построил правильное изменённое поддерево; второе поддерево уже правильно и не меняется. Сумма их корней правильна. Новая родительская запись не меняет старые ссылки. Индукция доказывает оба утверждения одновременно.

Для $n = 2^h$ присваивание создаёт ровно $h + 1$ узлов, включая лист и корень. Для произвольного n — не более $\lceil \log_2 n \rceil + 1$. В нашей реализации даже присваивание прежнего значения создаёт путь: проверка равенства могла бы сэкономить память, но для оценки не требуется.

Корень 0 не надо раскрывать заранее. Копия служебного узла содержит нулевые ссылки; рекурсия создаст только нужную ветвь. Доказательство сохраняется, если считать отсутствующее поддерево полным нулевым массивом на переданном диапазоне.

8.3. Реализация: пул узлов и присваивание

Полный заголовок `code/08-persistent-segment.hpp` включает необходимые стандартные библиотеки. Ниже последовательные части одного класса. Пул `deque` позволяет добавлять узлы без переноса всех прежних записей; внешние ссылки всё равно представлены индексами. Локальная копия `Node t` исключает запись в общий узел.

```
class PersistentSum {
public:
    using Root = std::size_t;
private:
    struct Node { long long sum=0; Root left=0, right=0; };
    std::deque<Node> pool{Node{}};
    int n;
    Root change(Root r, int lo, int hi, int pos, long long x) {
        Node t=pool[r];
        if (hi-lo==1) t.sum=x;
        else {
            int mid=lo+(hi-lo)/2;
            if (pos<mid) t.left=change(t.left,lo,mid,pos,x);
            else t.right=change(t.right,mid,hi,pos,x);
            t.sum=pool[t.left].sum+pool[t.right].sum;
        }
        pool.push_back(t);
        return pool.size()-1;
    }
```

Проверки `assert` выражают предусловия учебного интерфейса. Это не обработчик произвольного пользовательского ввода. В прикладном API ошибки индексов надо либо проверять явно, либо исключать условиями вызывающей программы. При `n=0` этот вариант класса не создаётся.

Почему новый узел добавляется после рекурсивного вызова? Тогда все его дети уже существуют. Вдоль каждой ссылки номер узла строго уменьшается, кроме служебного нуля. Следовательно, цикл в пуле невозможен. Номера задают топологический порядок графа представления, но не порядок версий: один корень может разделять узлы с несколькими историческими ветвями.

8.4. Чтение без выделения памяти

Запрос совпадает с обычным запросом суммы. При отсутствии пересечения возвращаем 0, при полном покрытии — готовую сумму, иначе складываем ответы детей. Нулевой корень тоже сразу возвращает 0. Запрос не создаёт и не меняет ни одного узла.

```
long long query(Root r,int lo,int hi,int l,int h) const {
        if (!r || h<=lo || hi<=l) return 0;
        if (l<=lo && hi<=h) return pool[r].sum;
        int mid=lo+(hi-lo)/2;
        return query(pool[r].left,lo,mid,l,h)
             + query(pool[r].right,mid,hi,l,h);
    }
```

Следующие методы завершают публичный интерфейс присваивания и суммы; позже к тому же классу добавим поиск порядковой статистики.

```
public:
    explicit PersistentSum(int size): n(size) { assert(n>0); }
    Root set(Root r,int pos,long long x) {
        assert(r<pool.size() && 0<=pos && pos<n);
        return change(r,0,n,pos,x);
    }
    long long sum(Root r,int l,int h) const {
        assert(r<pool.size() && l>=0 && h>=0 && h<=n);
        assert(static_cast<unsigned>(l)<=static_cast<unsigned>(h));
        return query(r,0,n,l,h);
    }
    Root add(Root r,int pos,long long delta) {
        assert(0<=pos && pos<n);
        return set(r,pos,sum(r,pos,pos+1)+delta);
    }
```

Метод `add` читает старое значение и вызывает присваивание. Это два прохода высоты $O(\log n)$ и одно выделение нового пути; асимптотика остаётся прежней. При частых прибавлениях можно сделать один проход, но сначала полезнее иметь легко проверяемый контракт.

Для произвольного начального массива можно выполнить n присваиваний за $O(n \log n)$ времени и узлов. Это удобный, но не оптимальный способ. Обычное рекурсивное построение создаёт $2n - 1$ узлов за $O(n)$, после чего используется тот же алгоритм изменений. Для частотного дерева исходное состояние действительно нулевое, поэтому начальные $O(n)$ узлов вообще не нужны.

После q присваиваний от нулевого корня занято $O(1 + q \log n)$ узлов и $O(q)$ корней, если сохранять все версии. Запрос занимает $O(\log n)$ времени, включая пустой диапазон, и $O(\log n)$ стека рекурсии. Отдельный запрос ко всему массиву выполняется за $O(1)$.

8.5. Префиксные частоты: другая ось дерева

Теперь массив a статичен. Отсортируем различные значения и получим массив координат c. Значение a_i заменяется своим рангом в c. Сжатие сохраняет отношения порядка и равенства, но не расстояния: разность рангов нельзя считать разностью исходных значений.

Дерево строится уже не по позициям, а по рангам значений. Корень p_i хранит частоты первых i элементов: $a_0, \dots, a_{i-1}$. Поэтому $p_0 = 0$, а $p_{i+1} = add(p_i, rank(a_i), 1)$. В каждом листе число увеличивается на единицу при очередном появлении данного значения.

Для редакторского массива $a = [5, 1, 5, 3, 1]$ координаты равны $c = [1, 3, 5]$.

Префикс	Добавленное значение	Частоты значений 1, 3, 5
p_0	—	0, 0, 0
p_1	5	0, 0, 1
p_2	1	1, 0, 1
p_3	5	1, 0, 2
p_4	3	1, 1, 2
p_5	1	2, 1, 2

Диапазону позиций $[1, 5)$ соответствует разность $p_5 - p_1$: частоты $(2, 1, 1)$. Значения на этих позициях — $[1, 5, 3, 1]$. Мы не создаём физическое дерево разности: во время запроса читаем два дерева и вычитаем их агрегаты.

Для подсчёта элементов с $x \le a_i < y$ на позициях $[l, r)$ найдём $L = lower_bound(c, x)$ и $R = lower_bound(c, y)$. Ответ:

$$sum(p_r, L, R) - sum(p_l, L, R).$$

Пустой диапазон значений даёт ноль. Если нужна закрытая верхняя граница $a_i \le y$, для неё используется `upper_bound(c,y)`. Преобразование `y+1` опасно на максимальном целочисленном значении; двоичный поиск не требует этой арифметики.

Задача 3769 из плана курса относится к подсчёту в диапазонах позиций и значений. Это мотивация метода; приведённый интерфейс не подменяет точное условие iRunner, его индексацию и формат ввода.

8.6. K-я порядковая статистика по двум корням

Пусть $1 \le k \le r - l$. В левом ребёнке разности $p_r - p_l$ находится d элементов. Если $k \le d$, ответ лежит слева. Иначе пропускаем эти d элементов и ищем справа номер $k - d$. В листе получаем ранг, который затем переводится обратно в значение через c.

В нашем примере диапазон $[1, 5)$ в отсортированном виде равен $[1, 1, 3, 5]$. При разбиении трёх рангов на $[0, 1)$ и $[1, 3)$ слева находятся две единицы. Для $k = 3$ переходим вправо с $k = 1$, затем выбираем ранг значения 3. Повторы считаются отдельными элементами; заменять частоты булевыми флагами нельзя.

```
// Precondition: every leaf of after-before is nonnegative.
    int kth(Root before,Root after,long long k) const {
        assert(before<pool.size() && after<pool.size());
        long long total=pool[after].sum-pool[before].sum;
        if (k<1 || k>total) throw std::out_of_range("rank");
        int lo=0,hi=n;
        while (hi-lo>1) {
            int mid=lo+(hi-lo)/2;
            Root a=pool[before].left, b=pool[after].left;
            long long count=pool[b].sum-pool[a].sum;
            if (k<=count) { before=a; after=b; hi=mid; }
            else {
                k-=count;
                before=pool[before].right;
                after=pool[after].right;
                lo=mid;
            }
        }
        return lo;
    }
    std::size_t allocated() const { return pool.size()-1; }
};
```

Предусловие поиска сильнее, чем у суммы: частота в каждом листе разности должна быть неотрицательной. Проверка общего количества в коде этого не доказывает. Префиксная конструкция обеспечивает условие автоматически, когда $l \le r$. Для двух произвольных версий массива разность может содержать отрицательные листья, и такое ветвление уже не является поиском по мультимножеству.

Почему алгоритм корректен? Левый диапазон значений целиком меньше правого. Поэтому первые d элементов отсортированного мультимножества находятся слева, остальные справа. После выбора стороны меняется только локальный номер искомого элемента. Инвариант «ищем k-й элемент текущего мультимножества» сохраняется до листа.

Сжатие занимает $O(n \log n)$ времени, построение префиксов — $O(n \log m)$ времени и узлов, где m — число разных значений. Один поиск занимает $O(\log m)$, а не $O(\log n \log m)$: на каждом уровне читаются две пары детей. При $m = 1$ сразу возвращается единственное значение.

Полная сборка префиксов и два запроса

Следующая программа связывает сжатие координат с интерфейсом класса. Она хранится отдельно в `code/08-prefix-example.cpp`. Массив непустой; если исходные данные допускают пустой массив, нужно обработать его до создания дерева с размером 0. В учебном примере все координаты и номера версий помещаются в `int`.

```
#include "08-persistent-segment.hpp"
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<long long> a{5,1,5,3,1}, c=a;
    std::sort(c.begin(),c.end());
    c.erase(std::unique(c.begin(),c.end()),c.end());
    PersistentSum tree(static_cast<int>(c.size()));
    std::vector<PersistentSum::Root> prefix{0};
    for (long long x:a) {
        int rank=static_cast<int>(
            std::lower_bound(c.begin(),c.end(),x)-c.begin());
        prefix.push_back(tree.add(prefix.back(),rank,1));
    }
    int l=1,r=5;
    int low=static_cast<int>(
        std::lower_bound(c.begin(),c.end(),1)-c.begin());
    int high=static_cast<int>(
        std::lower_bound(c.begin(),c.end(),5)-c.begin());
    std::cout << tree.sum(prefix[r],low,high)
              -tree.sum(prefix[l],low,high) << '\n';
    int rank=tree.kth(prefix[l],prefix[r],3);
    std::cout << c[rank] << '\n';
}
```

Программа выводит две строки с числом 3. У первого ответа смысл «на позициях от 1 до 4 включительно три элемента принадлежат диапазону значений [1,5)». Это две единицы и тройка; пятёрка исключена верхней границей. У второго ответа смысл «третий элемент после сортировки выбранного диапазона равен 3». Совпадение чисел не означает совпадение операций.

Разберём типичную ошибку: если вместо `prefix[r]` взять `prefix[r-1]`, последний элемент диапазона потеряется. Тогда в первом ответе останутся только одна единица и тройка, получится 2. Если вместо `prefix[l]` вычесть `prefix[l-1]`, добавится элемент перед левой границей. В данном примере это пятёрка, и первый ответ случайно не изменится, поскольку верхняя граница её исключает. Поэтому одного численного запроса недостаточно для проверки индексации: нужны также запрос по всем значениям и поиск последнего элемента.

Полезный инвариант подготовки: `sum(prefix[i],0,m)==i` для каждого префикса. Он проверяет, что каждое добавление увеличило общее число ровно на один. Но он не проверяет распределение по рангам: перепутанные координаты могут сохранить сумму. Для полного теста сравнивают частоты отдельных значений, диапазонные количества и порядковые статистики с прямой сортировкой.

8.7. Какие агрегаты можно вычитать

Разность префиксов работает для аддитивных сведений. Частоты и суммы складываются по непересекающимся частям и допускают обратную операцию. Минимум двух префиксов вычитать бессмысленно: минимумы 1 и 1 не говорят, какие значения находятся между границами.

Если в частотных узлах дополнительно хранить сумму исходных значений, можно вычислять сумму k наименьших на диапазоне. При уходе вправо прибавляем сумму всего левого мультимножества, при уходе влево ничего не прибавляем. В листе добавляем $k \cdot c[rank]$. Счётчик определяет направление, сумма определяет вклад. Оба поля вычитаются между префиксами.

Например, для $[1, 5)$ сумма трёх наименьших равна $1 + 1 + 3 = 5$. При первом уходе вправо забираем сумму двух единиц, затем одну тройку. Отрицательные значения не мешают: порядок координат сохраняется, а частоты остаются неотрицательными. Нужно лишь контролировать переполнение суммы и произведения.

Задача 5003 о реконструкции стены в плане отмечена как углубление. Здесь мы выводим строительный блок «порядок плюс суммы», а не заявляем готовое решение задачи по одному её названию.

8.8. Почему изменение массива не меняет один префикс

Если заменить a_j в исходном статическом массиве, изменятся все частотные префиксы с индексами больше j. Нельзя обновить только p_{j+1} и оставить остальные: их содержимое перестанет соответствовать определению. Построенные префиксы — способ индексировать статические данные, а не бесплатная поддержка онлайн-изменений.

Для динамических запросов по позициям и значениям нужна дополнительная структура: например, дерево по позициям, в узлах которого хранится структура по значениям. Персистентность такого двухуровневого решения требует нового анализа времени и памяти. Оценку $O(\log n)$ для обычной суммы нельзя автоматически перенести на подсчёт по двум координатам.

С отложенными операциями возникает другая опасность. Обычный `push` из § 3 изменяет детей. Если эти дети общие со старой версией, их необходимо скопировать перед изменением. Чтение лучше организовывать без записи, передавая накопленную метку по рекурсии. Порядок композиции остаётся важным. Приведённый здесь код поддерживает только точечные изменения.

Связь с материалами. Копирование пути и ветвление разобраны в Lecture 11 SDS-en. Из видео 30.11.2022, 44:00–46:00, переносится вопрос о памяти на одну версию: для стека это один узел, для дерева — весь изменяемый путь. Этот фрагмент не выдаётся за объяснение частотных префиксов; числовые трассы главы разработаны для учебника.

8.9. Память, проверка и практические ошибки

При $n = 2^{20}$ одно присваивание создаёт 21 узел. Сто тысяч изменений требуют 2,1 миллиона узлов, даже если все они меняют одну позицию. В пуле с двумя `size_t` и `long long` полезная запись на типичной 64-битной платформе занимает 24 байта: около 50,4 МБ только записей, без служебных расходов контейнера и списка корней. Реальный размер проверяется через `sizeof(Node)` и измерение процесса.

В нашем пуле удаление корня не освобождает узлы. Это сознательная модель «храним всё до уничтожения экземпляра». Сборка мусора или подсчёт ссылок могут освобождать недостижимые узлы, но добавляют стоимость. Их нельзя молча включать в оценку времени `set`.

Для проверки версий используйте копии маленьких массивов. Каждый шаг выбирает случайный старый корень, создаёт изменение и затем сравнивает запросы в нескольких старых и новой версиях. Обязательно проверяйте длины 1, 3, 5, пустые запросы, отрицательные значения и повторные присваивания.

Для частот проверяйте каждый диапазон маленького массива прямой сортировкой. Сравнивайте все допустимые k, отдельно повторяющиеся значения и координаты вне диапазона данных. Проверяйте, что `sum` и `kth` не увеличивают счётчик выделенных узлов. Линейная цепочка версий не ловит все ошибки общей памяти: нужны братья в дереве истории.

8.10. Вопросы для самопроверки

1. Чем отличается индекс в дереве версий массива от индекса префикса статического массива?
2. Какие узлы создаст присваивание позиции 5 при длине 8?
3. Почему один нулевой узел может представлять диапазоны разной длины?
4. Какие два утверждения нужно доказать при копировании пути?
5. Почему минимум на диапазоне нельзя получить вычитанием двух префиксных минимумов?
6. Какая проверка, кроме общего количества, нужна для корректности `kth` на произвольных корнях?
7. Как обработать закрытую верхнюю границу без вычисления `y+1`?
8. Почему замена одного элемента исходного массива меняет много частотных префиксов?
9. Как изменится поиск, если нужна сумма k наименьших значений?
10. Какой тест обнаружит изменение общего ребёнка через старую ссылку?

§ 9. Персистентные деревья поиска и кучи

Дерево отрезков имеет заранее заданную форму: высота ограничена независимо от значений. У дерева поиска форма зависит от порядка изменений. У кучи мы вообще не требуем порядка между левым и правым поддеревьями, зато можем ограничить длину пути слияния. Поэтому одна фраза «скопируем путь» ещё не даёт логарифмической оценки.

В этой главе построим неизменяемое множество на BST и персистентную очередь с приоритетом на левосторонней куче. Для каждого интерфейса отдельно докажем логическую корректность и стоимость. Затем сравним левостороннюю и биномиальную кучи и разберём особенность слияния двух версий, имеющих общие узлы.

9.1. Два интерфейса и два разных порядка

Множество поддерживает `contains(root,x)`, `insert(root,x)` и `erase(root,x)`. В этой главе ключи уникальны: повторная вставка не меняет множество, удаление отсутствующего ключа тоже ничего не меняет. Все изменения возвращают корень. Чтение старой версии разрешено после любого числа новых операций.

Очередь с приоритетом хранит мультимножество: одинаковые ключи допустимы и считаются разными элементами. Интерфейс содержит `top`, `push`, `pop` и `meld(a,b)`. Операция `pop` удаляет одно вхождение минимума. `meld` объединяет мультимножества с суммированием кратностей. Чтение минимума или удаление из пустой кучи — ошибка.

В BST все ключи левого поддерева меньше ключа корня, а правого — больше. Это позволяет выбрать одну ветвь поиска по сравнению. В min-куче ключ корня не превосходит ключи обоих поддеревьев. Из свойства кучи не следует, в какой ветви находится заданный ключ; быстрый поиск произвольного значения не входит в её контракт.

Редакторский пример. Из множества $\{2, 5, 8\}$ создадим одну версию вставкой 4, другую — удалением 5. Получатся $\{2, 4, 5, 8\}$ и $\{2, 8\}$. Для кучи со значениями $[2, 5, 8]$ те же действия нельзя описать через `erase(5)`: такого метода у нашего интерфейса нет. Здесь можно удалить только минимум 2.

9.2. Неизменяемые узлы BST

Используем `shared_ptr<const Node>`. Тип `const` запрещает изменение узла через опубликованную ссылку; счётчик ссылок управляет временем жизни. Это альтернативная инженерная модель по сравнению с индексным пулом § 8. Сохраняются только узлы, достижимые из живых корней и временных локальных ссылок.

Ключ и дети определяются в конструкторе. Функция `make` создаёт новый узел, а поиск читает существующие. Листинги ниже составляют полный класс из `code/09-persistent-trees.hpp`; нужны `<memory>`, `<utility>`, `<stdexcept>` и `<algorithm>`.

(c) Aleksey Tolstikov

```
class PersistentBST {
    struct Node;
public:
    using Root=std::shared_ptr<const Node>;
private:
    struct Node {
        int key; Root left,right;
        Node(int k,Root a,Root b):key(k),left(a),right(b) {}
    };
    static Root make(int k,Root a,Root b) {
        return std::make_shared<const Node>(k,a,b);
    }
public:
    static bool contains(Root r,int x) {
        while (r) {
            if (x==r->key) return true;
            r=x<r->key ? r->left : r->right;
        }
        return false;
    }
```

Один и тот же узел может иметь много владельцев в физическом графе памяти. Для конкретного корня его развёрнутое логическое представление всё равно является деревом поиска. Неизменяемость позволяет рассуждать о каждой версии отдельно, не копируя общие поддеревья.

Сохранение только корня достаточно, потому что всё состояние структуры достижимо из него. Если добавить внешний изменяемый словарь «ключ → родитель», эта гарантия исчезнет: словарь тоже понадобится версионировать. Указатели на родителей особенно неудобны, поскольку общий ребёнок может иметь разных родителей в разных версиях.

9.3. Вставка, удаление и сохранение старого порядка

Вставка повторяет обычный поиск места. По возвращении из рекурсии создаёт копии предков. Неизменённый ребёнок сохраняется. Если ключ уже найден, возвращается тот же корень, а проверка равенства ссылок распространяет отсутствие изменений вверх без новых узлов.

```
static Root insert(Root r,int x) {
        if (!r) return make(x,nullptr,nullptr);
        if (x==r->key) return r;
        if (x<r->key) {
            Root a=insert(r->left,x);
            if (a==r->left) return r;
            return make(r->key,a,r->right);
        }
        Root b=insert(r->right,x);
        if (b==r->right) return r;
        return make(r->key,r->left,b);
    }
```

Вставим 4 в дерево с корнем 5 и детьми 2 и 8. Получим новый лист 4 справа от 2, новый узел 2 и новый корень 5. Узел 8 останется общим. Вставка существующего ключа 2 вернёт исходный корень, не создавая новых узлов.

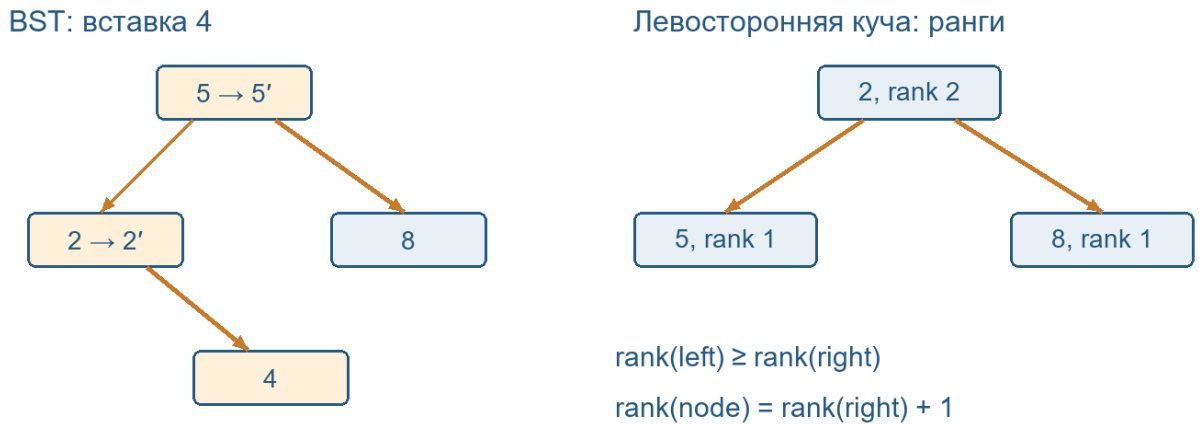


Рисунок 13: BST копирует поисковый путь. Левосторонняя куча контролирует ранг правого пути, а не общую высоту.

Для удаления есть три случая. У листа результат — пустая ссылка. Узел с одним ребёнком заменяется этим ребёнком. Если оба ребёнка существуют, возьмём минимальный ключ правого поддерева, создадим новый корень с этим ключом и удалим одно его вхождение справа. В множестве этот ключ встречается ровно один раз.

```
static Root erase(Root r,int x) {
        if (!r) return r;
        if (x<r->key) {
            Root a=erase(r->left,x);
            if (a==r->left) return r;
            return make(r->key,a,r->right);
        }
        if (x>r->key) {
            Root b=erase(r->right,x);
            if (b==r->right) return r;
            return make(r->key,r->left,b);
        }
        if (!r->left) return r->right;
        if (!r->right) return r->left;
        Root s=r->right;
        while (s->left) s=s->left;
        return make(s->key,r->left,erase(r->right,s->key));
    }
};
```

Почему выбор преемника сохраняет порядок? Все ключи слева меньше прежнего корня, а значит меньше преемника. Преемник — минимум справа, поэтому оставшиеся ключи справа строго больше него. Удаление рекурсивно сохраняет порядок внутри правого поддерева. Старый корень и его прежний преемник не изменяются.

Для нашего примера удаление 5 из исходного дерева создаёт корень 8 с левым ребёнком 2 и пустым правым ребёнком. Оно не затрагивает ветку, в которой ранее вставили 4. Удаление отсутствующего 6 пройдёт поиск и вернёт ту же ссылку, что получила операция.

Корректность доказывается по высоте: рекурсивное изменение строит правильное поддерево, новый родитель восстанавливает тот же порядок, остальные узлы неизменны. Для двух детей отдельно

используется свойство минимального ключа справа. Как и в § 8, доказательство включает сохранность всех старых корней.

9.4. Где появляется логарифм, а где его нет

Если высота исходной версии равна h, поиск занимает $O(h)$, изменение — $O(h)$ времени и новых узлов. Поиск преемника и его удаление проходят один путь дважды, что меняет константу, но не порядок оценки. Повторная вставка и удаление отсутствующего ключа новых узлов не создают.

В обычном BST может быть $h = n$. Последовательность вставок $1, 2, 3, \dots, n$ образует цепочку. Персистентное хранение всех промежуточных версий такого дерева требует $1 + 2 + \cdots + n = \Theta(n^2)$ узлов. Даже если каждый узел неизменяем, структура не становится сбалансированной.

Чтобы получить гарантированную логарифмическую высоту, можно использовать AVL-дерево. Вместе с ключом храним высоту; при возврате из рекурсии создаём новые узлы и выполняем функциональные повороты. При правом повороте вокруг y с левым ребёнком x сначала создаётся новый y с левым поддеревом $x.right$, затем новый x с этим y справа. Старые x и y не меняются.



Поддеревья A, B, C общие. Новые высоты вычисляются снизу вверх. При удалении балансировка может потребоваться на нескольких уровнях, поэтому недостаточно скопировать только два узла у первого поворота. При корректной AVL-балансировке путь и число новых узлов остаются $O(\log n)$.

Альтернатива — декартово дерево с независимыми случайными приоритетами. Приоритет хранится в узле и сохраняется при копировании; нельзя заново случайно назначать его каждому клону. Логарифмическая оценка здесь ожидаемая, а не гарантированная. Приведённый код намеренно показывает базовое BST: никаких скрытых гарантий AVL или treap у него нет.

Рекурсивная реализация может исчерпать стек на длинной цепочке. Для больших небалансированных данных нужен итеративный путь или балансировка. Персистентность не устраняет ограничения глубины рекурсии языка.

9.5. Левосторонняя куча: инвариант короткой правой ветви

У пустой кучи ранг 0. У непустого узла ранг равен единице плюс минимум рангов детей. Дополнительный левосторонний инвариант требует $rank(left) \geq rank(right)$. Поэтому $rank(v) = rank(v.right) + 1$: ранг равен числу узлов на правом пути до пустой ссылки.

У дерева ранга r не менее $2^r - 1$ логических узлов. Доказательство индукцией: при $r > 0$ оба ребёнка имеют ранг не меньше $r - 1$, значит содержат не менее $2^{r-1} - 1$ узлов каждый. Добавление корня даёт $2^r - 1$. Следовательно, правая ветвь имеет длину $O(\log(n + 1))$.

Заметьте, что всё дерево может быть высоким: длинная левая цепочка допустима. Для слияния нужна только правая ветвь, поэтому ограничивать полную высоту не требуется. Смешение ранга с высотой приводит к неправильной оценке и неправильной реализации `make`.

Представление снова неизменяемое. Конструктор `make` получает уже готовые поддеревья, при необходимости меняет их местами и вычисляет ранг. Перестановка не нарушает свойство min-кучи: оно симметрично относительно детей.

```
class PersistentLeftist {
    struct Node;
public:
    using Root=std::shared_ptr<const Node>;
private:
    struct Node {
        long long key; Root left,right; int rank;
        Node(long long x,Root a,Root b,int d)
            :key(x),left(a),right(b),rank(d) {}
    };
    static int rank(Root r) { return r ? r->rank : 0; }
    static Root make(long long x,Root a,Root b) {
        if (rank(a)<rank(b)) std::swap(a,b);
        int d=rank(b)+1;
        return std::make_shared<const Node>(x,a,b,d);
    }
public:
```

9.6. Слияние — единственная содержательная операция

Если одна куча пуста, возвращаем другую без выделения узлов. Иначе выбираем корень с меньшим ключом, пусть это a. Его ключ будет минимумом объединения. Левое поддерево a сохраняем, правое сливаем с b, затем восстанавливаем левосторонний инвариант через `make`.

```
static Root meld(Root a,Root b) {
        if (!a) return b;
        if (!b) return a;
        if (b->key<a->key) std::swap(a,b);
        return make(a->key,a->left,meld(a->right,b));
    }
```

Корректность состоит из трёх утверждений. Во-первых, мультимножество результата равно сумме мультимножеств аргументов: корень a и левое поддерево сохранены, рекурсивный результат содержит правое поддерево и всю кучу b. Во-вторых, $a.key$ не превосходит ключи всех этих элементов. В-третьих, `make` восстанавливает порядок рангов, не меняя состав элементов. Старые узлы только читаются.

Рассмотрим меру $rank(a) + rank(b)$. В каждом непустом рекурсивном шаге мы спускаемся по правой ссылке одного корня, а его ранг уменьшается на 1. Возможная перестановка аргументов сумму рангов не меняет. Значит, до пустой ссылки не более $rank(a) + rank(b)$ шагов. Слияние занимает $O(\log(n_a + 1) + \log(n_b + 1))$ времени и создаёт столько же узлов в худшем случае.

Никакого амортизационного переноса между версиями здесь нет. Каждая отдельная операция ограничена рангами собственных аргументов. Поэтому ветвление истории не ломает оценку, в отличие от повторного разворота очереди из § 7.

Вставка сливает исходную кучу с одноэлементной. Удаление минимума сливает детей корня. Доступ к минимуму просто читает ключ.

```
static Root push(Root r,long long x) {
        return meld(r,make(x,nullptr,nullptr));
    }
    static long long top(Root r) {
        if (!r) throw std::out_of_range("empty heap");
        return r->key;
    }
    static Root pop(Root r) {
        if (!r) throw std::out_of_range("empty heap");
        return meld(r->left,r->right);
    }
```

Класс продолжается диагностическим методом `valid` и закрывающей скобкой в полном заголовке.

Метод `valid` из полного заголовка используется только в проверках: рекурсивно проверяет порядок ключей и ранги. Это полный обход, его не нужно запускать внутри каждой рабочей операции. Иначе быстрые операции будут искусственно превращены в линейные.

9.7. Слияние родственных версий и кратности

Для обычных изменяемых куч слияние часто предполагает непересекающиеся владения памятью. Здесь старые корни сохраняются, поэтому нужно явно определить смысл общих узлов. Наш контракт — сумма мультимножеств. Если a и b содержат общий элемент по происхождению, он всё равно учитывается один раз от a и один раз от b.

Например, $h = [2, 5]$ и $g = push(h, 1) = [1, 2, 5]$. Результат `meld(h,g)` содержит $[1, 2, 2, 5, 5]$. Он не равен множеству $[1, 2, 5]$ и не является объединением уникальных идентификаторов задач. Для такого объединения нужен другой интерфейс и иной алгоритм устранения повторов.

Разрешён даже `meld(h,h)`: он удваивает каждую кратность. Физические узлы результата могут разделяться, но логическое дерево разворачивает каждую ссылку как отдельное вхождение. В этой модели размер n в оценке — количество логических элементов, а не число различных выделенных узлов. После многократного удвоения эти числа могут сильно различаться.

Цикл не возникает. Рекурсивный вызов получает меньший правый путь; новые узлы ссылаются на уже существующие или только что построенные поддеревья, ни один старый узел не получает обратной ссылки. Удаление минимума сохраняет корректные кратности и при общей памяти. Но полный обход для печати или `valid` может быть огромным относительно физического размера графа, поскольку посещает логические вхождения.

Для прикладной очереди задач часто правильнее запретить слияние родственных наборов по контракту. Тогда надо отслеживать происхождение элементов. Сам алгоритм кучи не умеет определять, должна ли одна и та же задача исполняться дважды.

9.8. Биномиальные кучи и осторожность с амортизацией

Биномиальное дерево B_0 состоит из одного узла. Дерево B_{k+1} получается связыванием двух B_k: корень с большим ключом становится ребёнком меньшего. У B_k ровно 2^k узлов, высота k, а ранги детей корня равны $k - 1, \dots, 0$.

Биномиальная куча хранит лес, в котором не более одного дерева каждого ранга. Наличие дерева ранга k соответствует единичному биту числа элементов. Слияние похоже на двоичное сложение: два дерева одного ранга связываются и дают перенос в следующий ранг. Если есть ещё и входящий перенос, одно дерево остаётся, а два других связываются.

При неизменяемом представлении связь создаёт новый корень победителя и новую ячейку списка детей; сами прежние поддеревья остаются общими. Список корней тоже должен быть неизменяемым. Тогда стандартное слияние лесов создаёт $O(\log(n_a + n_b + 1))$ новых ячеек и занимает столько же времени. Детали представления списков принципиальны: полное копирование массива всех потомков может ухудшить оценку.

Вставка одноэлементной кучи может вызвать цепь переносов. Если исходный размер $2^k - 1$, заняты все ранги от 0 до $k-1$, и одна вставка выполняет k связей. В линейной изменяемой истории такие переносы допускают амортизационный анализ. Но если m раз вставлять в одну и ту же старую версию размера $2^k - 1$, каждый раз придётся выполнить k связей: суммарно $\Theta(mk)$.

Поэтому для полной персистентности без специальной ленивой организации обещаем логарифмическую оценку отдельной вставки. Чтобы восстановить более сильные амортизированные гарантии, нужна отдельная конструкция с корректным разделением отложенной работы; обычного списка корней недостаточно.

9.9. Ресурсы и выбор реализации

Представление	Чтение	Изменение	Условие оценки
Обычное BST	поиск $O(h)$	$O(h)$ новых узлов	высота может быть n
AVL с копированием	поиск $O(\log n)$	$O(\log n)$	балансируется каждая версия
Левосторонняя куча	минимум $O(1)$	$O(\log n)$ для push/pop	ранг правой ветви
Биномиальная куча	минимум $O(\log n)$ без кеша	$O(\log n)$	корректные неизменяемые списки

Временные оценки наших методов считают обращения к узлам и выделение новых узлов. Управление временем жизни добавляет важную оговорку. Если удалить последний корень большого уникального поддерева, `shared_ptr` может освободить много узлов одним каскадом. Такая операция не обязана занимать $O(1)$; на глубокой цепочке возможно и большое вложение деструкторов. Для жёстких задержек нужен пул, отложенная очистка или специально управляемое освобождение.

Счётчики ссылок и отдельные выделения памяти дороже компактного индексного пула. Зато они автоматически освобождают недостижимые узлы. Выбор зависит от задачи: сохраняются ли все корни до конца, удаляются ли версии, важна ли предсказуемая задержка, достаточно ли памяти. Ни один вариант не отменяет необходимости считать новые узлы и корни отдельно.

Связь с курсом. Эти структуры соответствуют разделу 3.3 программы. Идея разделения узлов продолжает обсуждение версий и памяти из записи 30.11.2022, 44:00–46:00. Конкретные алгоритмы BST и куч не приписываются этому короткому фрагменту; их доказательства и примеры здесь — самостоятельная редакторская разработка.

9.10. Проверка и вопросы для самопроверки

Эталон для множества — отдельный `std::set` для каждой версии. На каждом шаге выбирайте случайного родителя и случайную вставку или удаление, затем проверяйте присутствие всех ключей небольшого универсума. Для кучи эталон — `std::multiset`; после слияния добавляйте все вхождения обоих аргументов, включая повторы.

Проверяйте не только минимум: последовательно удалите элементы из копии корня и сравните всю отсортированную последовательность. Исходный корень должен остаться прежним. Добавьте пустую

кучу, одинаковые ключи, отрицательные ключи, слияние пустых корней, слияние версии с собой и серию удалений из разных потомков одной версии.

1. Почему BST и min-куча поддерживают разные виды поиска?
2. Сколько узлов создаёт вставка уже существующего ключа в приведённом BST?
3. Почему удаление корня с двумя детьми не нарушает строгий порядок?
4. Приведите историю, в которой память персистентного BST квадратична.
5. Какие узлы создаются при функциональном правом повороте?
6. Чем ранг левосторонней кучи отличается от её высоты?
7. Какая мера убывает при каждом рекурсивном шаге `meld`?
8. Каковы кратности в результате слияния версии с её потомком?
9. Почему переносы биномиальной кучи нельзя бесплатно амортизировать по ветвящимся версиям?
10. Почему удаление корня с `shared_ptr` может вызвать линейную работу?

(c) Aleksey Tolstikov

§ 10. Общие методы персистентности и дерево версий

В предыдущих параграфах мы знали форму структуры и могли явно выписать изменяемый путь. Общий метод должен работать на уровне полей и ссылок: сохранить старое значение, найти нужное состояние при чтении и не потерять связи после копирования узла. Его стоимость зависит от модели памяти и вида истории.

Начнём с полностью проверяемого частного случая — истории одной ячейки на линейной шкале времени. Затем рассмотрим копирование узлов с ограниченным числом входящих ссылок, объясним, что меняется при ветвлении, и построим эйлерову нумерацию дерева версий. Завершим точным форматом сериализации и десериализации корневого упорядоченного дерева. Нумерация готового дерева и поддержание порядка в растущем дереве — разные задачи.

10.1. Сначала фиксируем модель вычислений

Частичная персистентность разрешает читать любую старую версию, но изменять только последнюю. История изменений линейна. Полная персистентность разрешает продолжить любую версию; если каждое изменение имеет одного родителя, история — дерево. Конфлюэнтные операции объединяют несколько версий и дают граф зависимостей. Эйлерова техника этой главы рассчитана на дерево, а не на произвольный граф.

Функциональность — свойство реализации: опубликованные объекты не изменяются. Оно не тождественно одной из трёх форм истории. Неизменяемые узлы могут представлять как линейную историю, так и ветвящуюся, а при аккуратно определённом слиянии — конфлюэнтную. Напротив, частичная персистентность может использовать изменяемые служебные журналы.

Считаем, что каждый узел имеет постоянное число полей фиксированного размера. Чтение одного поля, запись одного поля и работа с указателем занимают $O(1)$. Нужно отдельно указать, ограничено ли число входящих ссылок на узел. Постоянное число детей не означает постоянного числа родителей в общем графе.

Есть ещё два графа, которые нельзя смешивать. Дерево версий показывает происхождение состояний. Граф представления показывает, какие физические узлы используются корнями. Даже когда история является деревом, представление обычно содержит разделяемые узлы и не является деревом.

Редакторский пример. Версии v_1 и v_2 продолжают v_0 разными изменениями. В графе истории у них один общий предок. В памяти они могут разделять десятки поддеревьев, а каждое поддерево может иметь ссылки от нескольких разных корней. Номер версии не заменяет адрес узла и наоборот.

10.2. Полное копирование как эталон

Самый простой метод сохраняет полную копию состояния перед изменением. Для массива длины n это $O(n)$ времени и дополнительной памяти на версию. Чтение не требует поиска истории: выбираем копию и выполняем обычный запрос. При q изменениях получается $O(nq)$ памяти.

Этот способ полезен как тестовая модель и иногда как рабочее решение при очень маленьком состоянии. Его нельзя отвергать только из-за асимптотики: если состояние содержит десять чисел, дополнительная сложность графа узлов может не окупиться. Но при миллионе элементов и точечных изменениях полное копирование расходует слишком много памяти.

Копирование пути уменьшает работу до числа предков изменяемого узла. Для дерева отрезков это логарифм, для списка — расстояние от головы, для несбалансированного BST — его высота. Метод хорош, когда путь известен и все ссылки на изменяемую часть можно восстановить через новый корень.

В произвольном графе у узла может быть много родителей. Простая попытка «скопировать всех родителей» может затронуть большую часть структуры или зациклиться при наличии циклов. Поэтому общий результат требует более сильных предпосылок, чем корректность конкретного дерева.

10.3. Толстая ячейка на линейной шкале времени

Вместо копирования объекта сохраним историю значения: пары `(time,value)` с возрастающими временами. Чтение на момент t выбирает последнюю запись с временем не больше t. Отсутствие новой записи между двумя моментами означает, что значение не менялось.

Возьмём начальное значение 4 в момент 0, затем записи `(3,9)` и `(8,-1)`. Чтение в моменты 0, 2, 3, 7, 8 даёт соответственно 4, 4, 9, 9, -1. Запрос в момент 100 тоже вернёт -1: в контракте этой ячейки значение действует до следующей записи. Контроль существования глобального номера версии остаётся обязанностью вызывающего кода.

```
class HistoricalCell {
    std::vector<std::pair<int,long long>> history;
public:
    explicit HistoricalCell(long long initial):history{{0,initial}} {}
    void write(int time,long long value) {
        if (time<=history.back().first)
            throw std::invalid_argument("time must increase");
        history.emplace_back(time,value);
    }
    long long read(int time) const {
        if (time<0) throw std::out_of_range("negative time");
        auto it=std::upper_bound(history.begin(),history.end(),time,
            [](int t,const auto& entry){ return t<entry.first; });
        return std::prev(it)->second;
    }
};
```

`upper_bound` находит первую запись с временем строго больше запроса; предыдущая содержит ответ. Начальная запись в момент 0 гарантирует, что для неотрицательного времени предшественник существует. Проверка возрастающих времён исключает случай, когда поздняя запись вставляется в середину уже опубликованной истории.

Если у поля было m изменений, чтение занимает $O(\log(m+1))$, хранение — $O(m+1)$. Дописывание в `vector` занимает амортизированное $O(1)$, а отдельное расширение массива может стоить $O(m)$. Для жёсткой гарантии добавления требуется другое хранилище. Время здесь относится к физической линейной истории самого журнала: мы не ветвим его обновления.

Толстый узел хранит такие истории для своих полей. Если обычная операция читала k полей, после преобразования она может потребовать $O(k \log q)$ времени. Это простой общий способ сохранения прошлого, но он не даёт автоматически постоянной надбавки на чтение.

10.4. Ограниченная коробка изменений и копирование узла

Чтобы не искать в длинном журнале, можно хранить в узле исходные поля и небольшую коробку последних модификаций. Для частичной персистентности каждая запись содержит поле, время и новое значение. При чтении старой версии проверяем постоянное число записей и выбираем подходящее состояние. Если коробка ещё не заполнена, изменение дописывается в неё.

Когда коробка заполнена, создаём копию узла в его текущем состоянии и применяем новое изменение к копии. Старый узел остаётся доступен старым версиям. Текущие входящие ссылки нужно

перенаправить на копию. Каждое перенаправление само является изменением поля родителя и тоже может вызвать копирование.

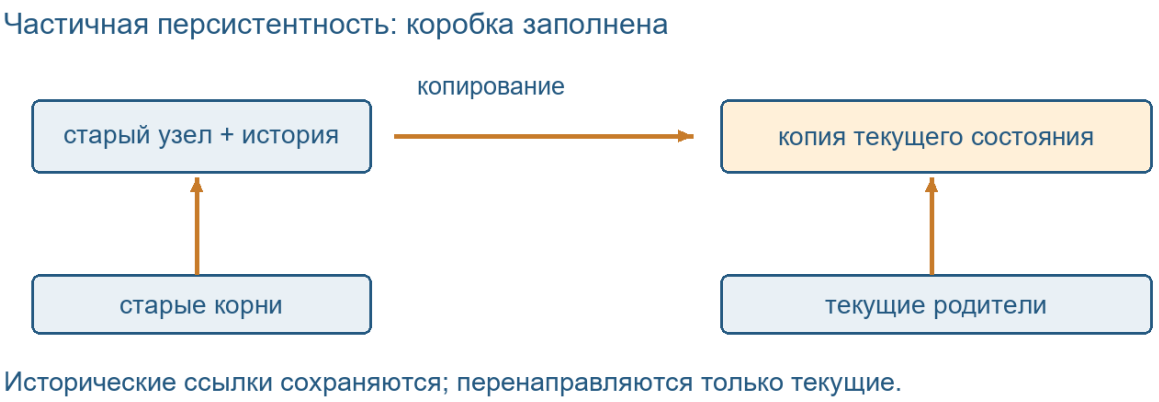


Рисунок 14: Старая версия читает прежний узел и его историю; текущие ссылки переводятся на новую копию.

Важна формулировка «текущие входящие ссылки». Старые корни и исторические связи не переписываются. Для поиска текущих родителей обычно поддерживаются обратные сведения. Если на узел в текущем состоянии указывают p родителей и p не ограничено, одна его копия потребует $\Omega(p)$ перенаправлений. Именно поэтому условие ограниченной входящей степени существенно.

В модели указателей с постоянным числом полей и ограниченной входящей степенью существуют стандартные конструкции частичной персистентности с постоянной амортизированной дополнительной памятью на элементарную модификацию и постоянной стоимостью доступа к полю. Реализация должна правильно выбрать ёмкость коробки, поддерживать текущие обратные ссылки и учитывать каскады копирования. Это утверждение о такой конструкции, а не о любом произвольном «журнале плюс копирование».

Интуиция учёта следующая. Заполненные коробки накапливают выполненные модификации; создание копии освобождает запас места для будущих изменений. Каскад не бесплатен, но его стоимость оплачивается ранее заполненными коробками. Потенциал учитывает занятые места, а постоянный предел входящих ссылок ограничивает число новых перенаправлений от одной копии. Без этих условий нельзя завершить амортизационный аргумент.

В этой главе данный результат используется как граница применимости общего метода. Полная реализация с обратными указателями и полное доказательство теоремы не заменяются несколькими строками псевдокода. Практические листинги главы реализуют более простые и явно ограниченные интерфейсы: линейную историческую ячейку и работу с готовым деревом.

10.5. Почему время создания не описывает ветвление

Пусть в v_0 поле равно 0. Версия v_1 продолжает v_0 и записывает 5. Позже версия v_2 тоже продолжает v_0, но меняет другое поле. В v_2 первое поле должно остаться 0. Линейный поиск «последняя запись с номером не больше 2» ошибочно выберет запись из v_1.

Для полной персистентности модификация видна не всем более поздним версиям, а только своим потомкам, если по пути не встретилось более близкое изменение этого поля. Нужное отношение — предок/потомок, а не сравнение хронологических номеров.

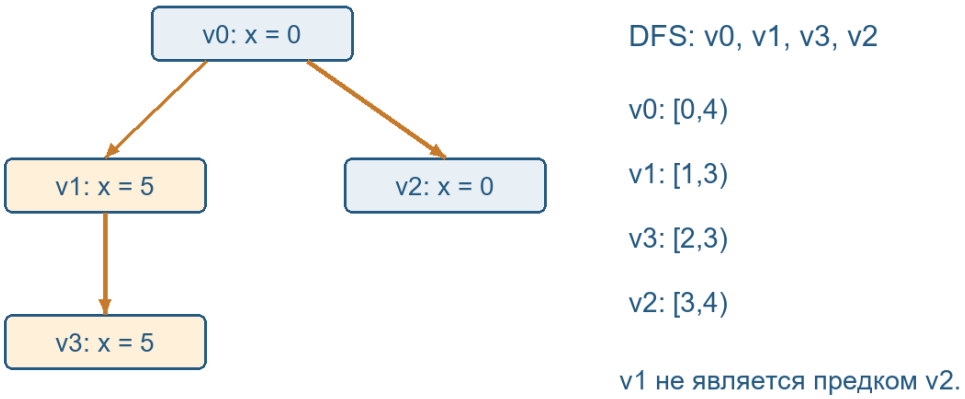


Рисунок 15: Запись в v1 видна v3, но не соседней ветви v2. Порядок создания 0,1,2,3 не кодирует это отношение.

Если для одного поля записаны модификации в нескольких вершинах дерева версий, запрос в вершине v ищет ближайшего предка v среди этих вершин. Это отдельная задача о помеченных предках. Одной проверки «является ли u предком v» недостаточно: она отвечает для выбранного кандидата, но не находит нужного кандидата среди всех записей.

Для офлайн-обработки, когда вся история заранее известна, можно выполнить DFS по дереву версий. При входе применяем изменение, отвечаем на запросы вершины, затем посещаем детей. При выходе откатываем изменение. На текущем стеке рекурсии находятся ровно предки рассматриваемой версии, поэтому значения соответствуют ей.

Это полезный способ решить задачу о запросах ко всем версиям, но не онлайн-структура полной персистентности. Он требует заранее известной истории и изменения порядка обработки. Если следующий запрос зависит от ответа предыдущего, такой перенос может быть невозможен. Откат и персистентность решают пересекающиеся, но разные задачи.

10.6. Эйлерова нумерация готового дерева

При первом посещении вершины v присвоим ей `tin[v]` и увеличим счётчик. После посещения всех детей запишем текущее значение счётчика в `tout[v]`, не увеличивая его повторно. Тогда вершины поддерева v занимают полуинтервал $[tin[v], tout[v])$.

Это одна из распространённых конвенций. Есть и другая, где счётчик увеличивается на входе и выходе. Формулы разных конвенций нельзя смешивать. В нашем варианте у листа `tout=tin+1`, а размер поддерева равен `tout-tin`.

Пусть у 0 дети 1 и 2, а у 1 ребёнок 3. При порядке детей 1, затем 2 получаем:

Вершина	tin	tout	Глубина
0	0	4	0
1	1	3	1
3	2	3	2
2	3	4	1

Условие предка, включая саму вершину:

$$u \text{ — предок } v \quad \Longleftrightarrow \quad tin[u] \le tin[v] < tout[u].$$

DFS заканчивает всё поддерево до перехода к следующему брату. Поэтому между входом и выходом u посещаются ровно его потомки. Это доказывает обе стороны эквивалентности. Условие `tin[u]<=tin[v]` само по себе недостаточно: 1 посещён до 2, но не является его предком.

Рекурсивный обход прост, однако длинная цепочка может исчерпать стек. Следующая реализация использует явный стек пар «вершина, индекс следующего ребёнка». Она принимает уже проверенное дерево с корнем 0.

```
struct EulerIndex { std::vector<int> tin,tout,depth; };
// Precondition: children describes one tree rooted at 0.
inline EulerIndex index_tree(const std::vector<std::vector<int>>& children) {
    int n=static_cast<int>(children.size());
    if (!n) throw std::invalid_argument("empty tree");
    EulerIndex e{std::vector<int>(n),std::vector<int>(n),
                 std::vector<int>(n)};
    std::vector<std::pair<int,std::size_t>> stack{{0,0}};
    int timer=0;
    e.tin[0]=timer++;
    while (!stack.empty()) {
        int v=stack.back().first;
        std::size_t next=stack.back().second;
        if (next==children[v].size()) {
            e.tout[v]=timer;
            stack.pop_back();
        } else {
            ++stack.back().second;
            int u=children[v][next];
            e.depth[u]=e.depth[v]+1;
            e.tin[u]=timer++;
            stack.emplace_back(u,0);
        }
    }
    return e;
}
inline bool ancestor(const EulerIndex& e,int u,int v) {
    return e.tin[u]<=e.tin[v] && e.tin[v]<e.tout[u];
}
```

Один проход занимает $O(V)$ времени и памяти. Каждый ребёнок рассматривается один раз, каждая вершина один раз добавляется в стек и удаляется из него. Запрос предка после подготовки выполняется за $O(1)$.

Полученная нумерация позволяет использовать структуры предыдущих глав: сумма по поддереву становится запросом на отрезке эйлерова порядка. Но это не означает, что сама форма дерева может произвольно изменяться без обновления индексов. Статический порядок фиксирует конкретный обход конкретного дерева.

10.7. Динамический порядок: что нужно общему методу

Добавим нового ребёнка к вершине 1 из предыдущего примера. Его надо разместить внутри интервала поддерева 1, перед вершиной 2. Если номера `tin` — плотные целые числа, вставка сдвинет номера последующих вершин. Простое увеличение глобального счётчика даст номер в конце и нарушит свойство поддерева как интервала.

Вместо плотных чисел можно хранить события входа и выхода в упорядоченном списке. Для нового ребёнка вставляются два события перед выходом родителя. Вложенность событий сохраняется: вход родителя предшествует входу ребёнка, а выход ребёнка предшествует выходу родителя. Сравнения порядка заменяют сравнения постоянных числовых меток.

Задача поддержания порядка предоставляет `insert_after(position,item)` и `before(a,b)`. Она отличается от порядковой статистики: узнать, что стоит раньше, не то же самое, что узнать точный номер позиции. Простой двусвязный список быстро вставляет, но сравнивает произвольные элементы медленно; массив быстро сравнивает позиции, но дорого вставляет в середину.

Назначение дробных меток между соседями работает только до исчерпания точности. Повторные вставки в один промежуток потребуют всё больше битов. Метки фиксированной разрядности с редкими перенумерациями дают полезные конструкции, но частота и стоимость перенумераций требуют доказательства. Нельзя назвать такой способ $O(1)$ только потому, что среднее двух чисел вычисляется быстро.

В общих методах полной персистентности динамический порядок помогает описывать дерево версий и области действия модификаций. Однако он не решает автоматически поиск ближайшего изменения поля и перенаправление ссылок. Эти компоненты должны согласованно поддерживаться. Поэтому из наличия эйлерова обхода не следует универсальная реализация полной персистентности за постоянное время.

Для практического выбора полезно сначала спросить, нужна ли такая универсальность. Если структура — дерево отрезков, копирование пути проще и даёт явные логарифмические оценки. Если история известна заранее, обход с откатом может оказаться существенно проще общего онлайн-метода.

10.8. Сериализация: сначала однозначный формат

Эйлеровым обходом называют несколько разных последовательностей: времена входа, события входа/выхода, список вершин при каждом проходе ребра. Для восстановления дерева нужно выбрать точный формат. Одно перечисление вершин в порядке первого посещения не определяет дерево: последовательность 0,1,2 подходит и цепочке, и корню с двумя детьми.

Выберем формат для **одного корневого упорядоченного дерева без меток**. При входе в вершину пишем `(`, после всех детей — `)`. Порядок детей значим. Для одной вершины код `()`, для корня с двумя листьями — `(()())`, для цепочки из трёх вершин — `((()))`.

Число токенов равно $2V$. Непустая строка должна иметь нулевой итоговый баланс, а баланс каждого непустого собственного префикса должен быть строго положителен. Строгость исключает лес: `()()` сбалансирован, но кодирует две отдельные корневые компоненты. Также запрещены неизвестные символы и закрытие вершины при пустом стеке.

Для восстановления при `(` создаём новую вершину. Если стек непуст, добавляем её последним ребёнком текущей вершины; затем кладём её в стек. При `)` извлекаем текущую вершину. Номера присваиваются в порядке входа. Если нужны исходные метки вершин, формат должен дополнительно хранить их при открытии; текущий код восстанавливает форму и порядок, а не потерянные имена.

```cpp
// One rooted ordered tree, encoded by entry '(' and exit ')'.
inline std::vector<std::vector<int>> decode(const std::string& s) {
    if (s.empty()) throw std::invalid_argument("empty code");
    std::vector<std::vector<int>> children;
    std::vector<int> stack;
    for (char c:s) {
        if (c=='(') {
            if (stack.empty() && !children.empty())
                throw std::invalid_argument("multiple roots");
            int v=static_cast<int>(children.size());
            children.emplace_back();
            if (!stack.empty()) children[stack.back()].push_back(v);
            stack.push_back(v);
        } else if (c==')') {
            if (stack.empty()) throw std::invalid_argument("extra close");
            stack.pop_back();
        } else throw std::invalid_argument("unknown token");
    }
    if (!stack.empty()) throw std::invalid_argument("unclosed node");
    return children;
}
```

Инвариант стека: он содержит путь от корня до вершины, чья запись сейчас продолжается. Открывающая скобка создаёт ровно следующего ребёнка вершины на вершине стека. Закрывающая завершает именно этого ребёнка. Значит, любой допустимый код разбирается единственным способом. Повторная сериализация полученного дерева даст исходную строку.

Время и память равны $O(V)$. Сам текст тоже занимает $O(V)$; потоковый разбор может не хранить его целиком, но для построения списка детей всё равно нужна память под дерево. Итеративный стек позволяет обработать цепочку из сотен тысяч вершин без рекурсивных вызовов.

10.9. Проверка обхода не равна восстановлению

Проверяющий алгоритм отвечает только «допустимо или нет». Восстанавливающий возвращает структуру, на которой можно выполнять последующие запросы. Для скобочного формата проверка баланса проще построения, но её положительный ответ ещё не даёт списков детей.

В плане курса задачи 3768 «Эйлеров обход» и 4084 «Проверка ответа» указаны как мотивация. Перед применением алгоритма нужно сверить их точный формат. Если условие даёт последовательность меток при переходах по рёбрам, нельзя читать её как наши скобки. Если проверяется обход заданного дерева, одной сбалансированности тоже недостаточно: надо сопоставлять события его рёбрам и вершинам.

Проверим редакторский код `((())())`. Он задаёт корень 0 с детьми 1 и 3, у вершины 1 есть ребёнок 2. Сериализация задаёт порядок 0,1,2,3; интервалы поддеревьев равны $[0,4)$, $[1,3)$, $[2,3)$ и $[3,4)$. Теперь номера 1 и 3 являются братьями, а 1 — предок 2. Это та же форма, что на схеме дерева версий, с перенумерацией по входу.

Хорошие отрицательные тесты: пустая строка; `)(`; `((`; `())`; `()()`; строка с посторонним символом. Положительные: одна вершина, длинная цепочка, звезда, случайное упорядоченное дерево. Проверяйте круговой переход «дерево → строка → дерево» с учётом того, что исходные произвольные метки не сохраняются.

Для исторической ячейки проверьте чтение ровно на момент записи, между записями и после последней. Для дерева версий сравните `ancestor` с подъёмом по родителям на всех парах маленьких деревьев. Эти тесты проверяют реализованные интерфейсы; они не являются экспериментальным доказательством общей теоремы о персистентности.

10.10. Итоговый выбор метода и самопроверка

Условия задачи	Подход	Что нужно доказать
Малое состояние	полная копия	стоимость всей копии
Короткий путь изменения	копирование пути	высота и неизменность общих узлов
Линейная история полей	толстые узлы	поиск последней записи
Ограниченная входящая степень	коробки и копирование	учёт каскадов и обратных ссылок
Офлайн-дерево версий	DFS с откатом	правильный текущий путь
Онлайн-ветвление общего графа	специальный общий метод	области действия, порядок, ссылки

Общая тема всех вариантов — явно определить, какое состояние видит запрос. Оптимизация начинается после этого определения. Недостаточно сохранить несколько корней, если из них достижимы изменяемые общие данные. Недостаточно подписать изменения временем, если история ветвится. Недостаточно получить быстрые отдельные компоненты, если их совместное обновление требует полного перестроения.

Источники и границы. Глава следует разделам 3.1 и 3.4 программы и темам лабораторных по сериализации. Подтверждённый видеофрагмент 30.11.2022, 44:00–46:00, служит только связью с обсуждением памяти версий из § 7. Новая расшифровка общих методов по нему не заявляется. Для углублённого доказательства общих преобразований полезна работа Driscoll, Sarnak, Sleator, Tarjan, *Making Data Structures Persistent* (1989): она существенно шире реализованных здесь частных случаев.

1. Чем функциональная реализация отличается от полной персистентности?
2. Почему постоянное число детей не ограничивает число входящих ссылок?
3. Какой ответ даст историческая ячейка между двумя записями?
4. Какая часть операции `write` имеет лишь амортизированную оценку?
5. Почему при копировании узла перенаправляются только текущие ссылки?
6. Постройте контрпример к поиску записи по номеру создания в ветвящейся истории.
7. Докажите формулу предка для выбранной конвенции tin/tout.
8. Почему новый ребёнок может нарушить статическую эйлерову нумерацию?
9. Чем интерфейс поддержания порядка отличается от запроса номера позиции?
10. Почему строка `)(` недопустима в нашем формате, хотя её баланс неотрицателен?

(c) Aleksey Tolstikov